

OPERATING SYSTEMS

Handwritten Notes

VIVEKANAND VERNEKAR

GateWallah Parakram C 2024

(Hinglish Weekday)

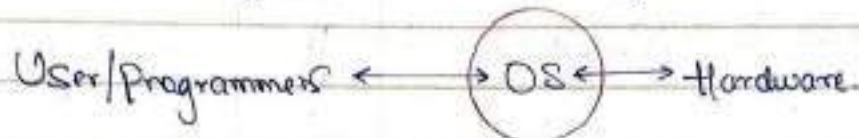
Other Notes Uploaded Below

<https://t.me/pwgatenotes>

Introduction

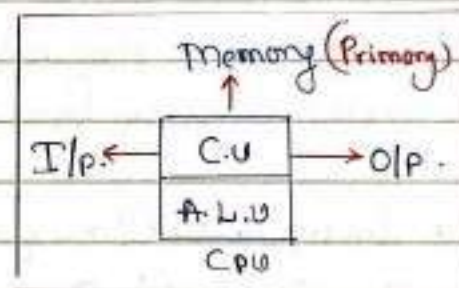
What is an Operating System?

- * Is an interface between user and the hardware
- * Resource managers
- * Set of utilities to simplify the application development
- * Set of program(s) / Software
- * Operating System acts like a government.



1. Control unit (C.U): It is circuitry that directs operations within a computer's processor.

- * Timing / control signals
- * Sequencing / execution of micro-operations.



VON-NEUMANN Architecture.

operations that are carried out on the data, stored in registers (CPU)

2. Arithmetic Logic Unit (ALU): It is a part of CPU that carries out arithmetic and logic operations on the Operands in computer instruction words

Memory:

Primary

Secondary

also called

"Auxiliary memory"

eg: hard disks, pendrive etc.

"Main memory"

eg: Ram, Rom, Cache, registers.

Primary memory

- * Volatile (not Rom)
- * faster to access $\times 10^{-9}$ (ns)
- * Smaller size
- * more expensive

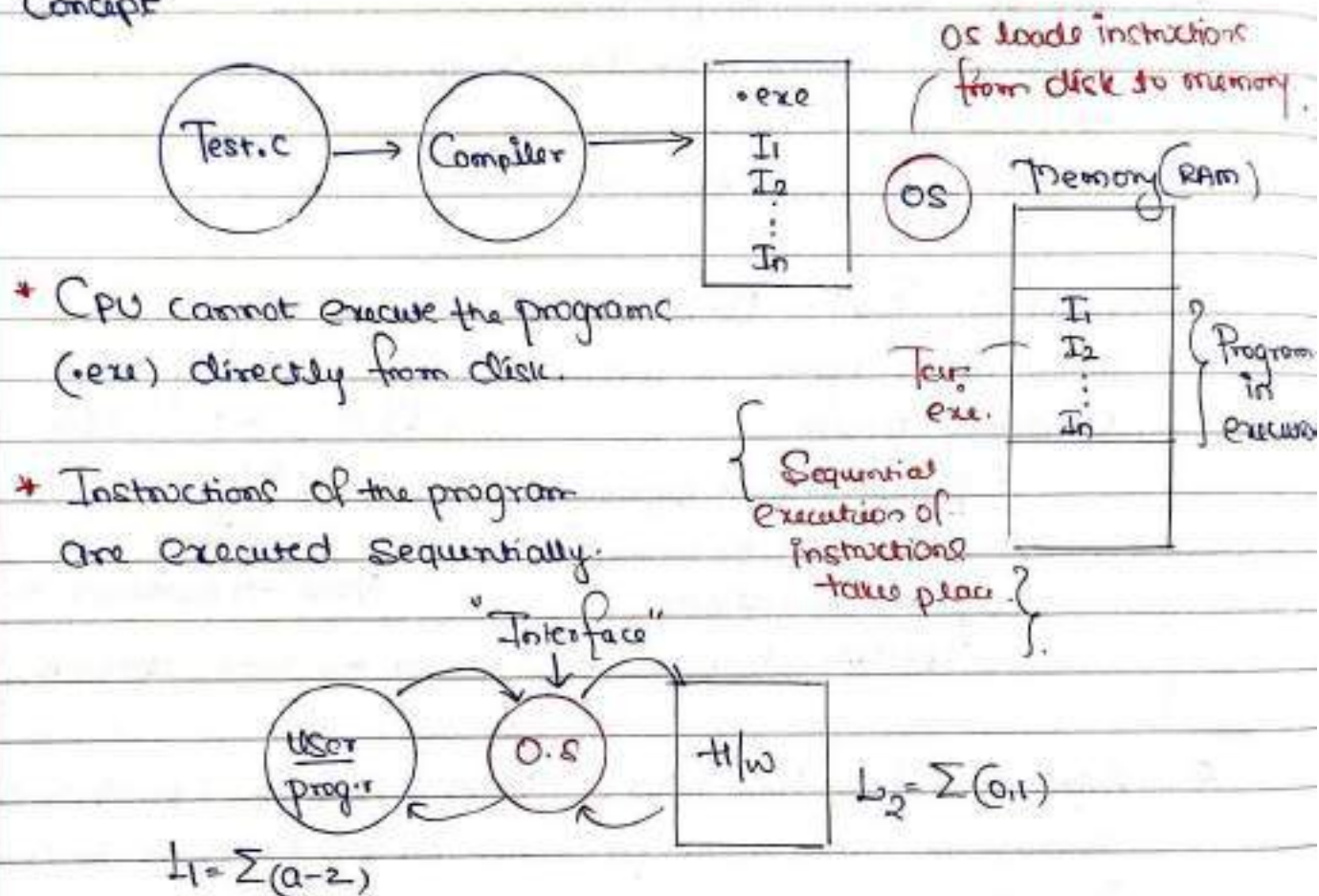
Secondary memory

- * Non volatile
- * Slower to access (ms) $\times 10^{-3}$
- * larger size
- * less expensive

Note: as per Von Neumann architecture, all Secondary storage devices are part of I/O devices.

Q How are programs executed in Von Neumann Architecture?

→ The programs are executed by the CPU by Stored program Concept



Modules of OS:

Process manager (CPU)

Memory manager (memory)

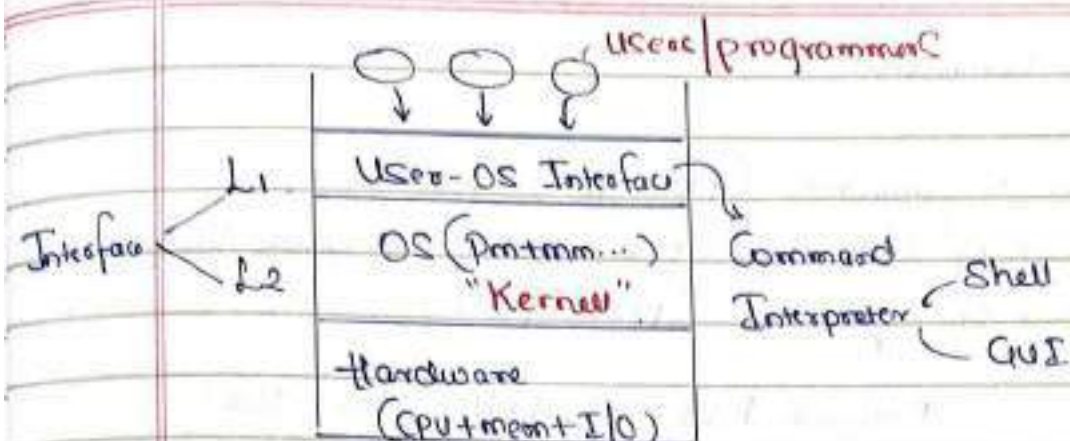
File manager { Device

Devices manager

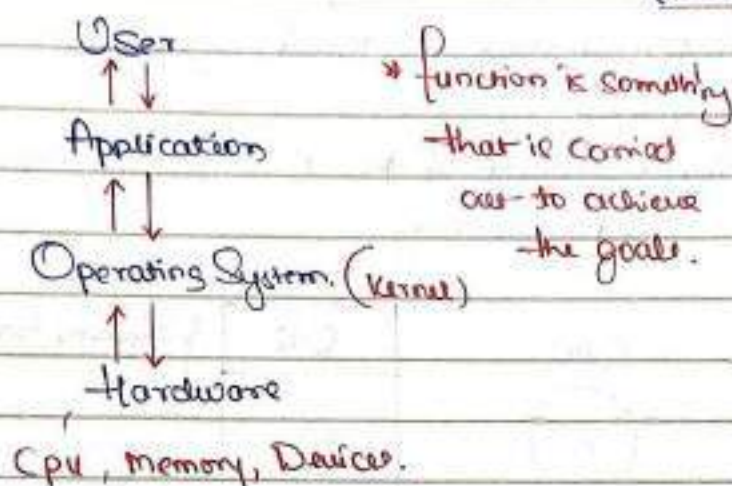
Protection manager (Resource)

Collection is called
"Kernel / Nucleus"

Kernel is a central component of an operating system that manages operations of computer and hardware. It basically manages operations of memory and CPU time.

Functions of OS:

1. Process management
2. Memory mgmt.
3. Error detection
4. Security
5. File Management.

Functions and Goals:

1. Convenience (ease of use)
2. Efficiency
3. Reliability
4. Robustness
5. Scalability
6. Portability

★ Primary (main) goal of an Operating System: Convenience (for personal computing)

★ Convenience is not primary goal of all other computing environment

Other Computing Environment:

(a) Real time Systems (RTS): + Operates in real time (fixed deadlines)

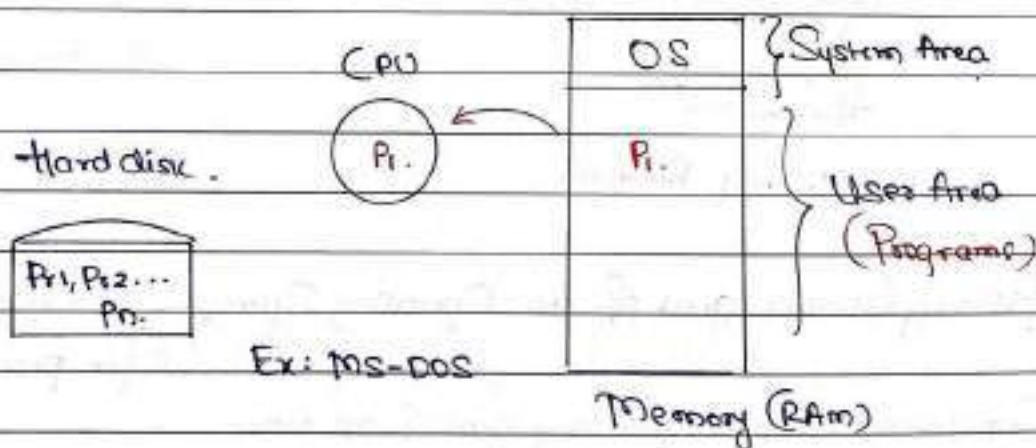
↓
Main goal: efficiency

* Ex: missile control, satellite control
← { Hard - Real time System }

(b) Mobile Operating System main goal: battery efficiency.

Tim Types of OS:

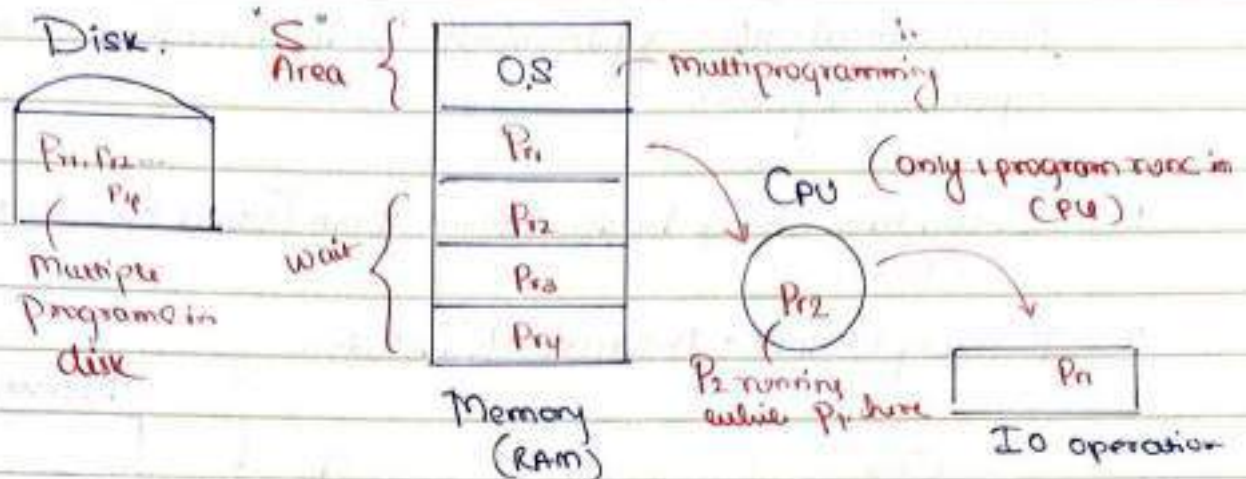
- | | |
|-----------------------|-------------------|
| 1. Batch OS | 4. Real time OS |
| 2. Multiprogrammed OS | 5. Network OS |
| 3. Time Sharing OS | 6. Distributed OS |
- } Advanced

Generations of Computers:1st Generation of Computers (1930-40s): No O.S2nd Generation of Computers (1940-1950s): magnetic Tapes3rd Generation of Computers (1950-1960s)↓
Magnetic Disk (Hard disk, floppy disk)↙
Uni-programmed↘
Multi programmedUni-programming: Ability of the O.S to hold a single program in memoryDrawback:

- * Single program in memory is not enough (sufficient) to keep the CPU always busy.
- * Throughput and efficiency will be less.
 ↑
 no. of programs completed
 Unit time.

Multiprogramming: Ability of the operating system to hold (manage) multiple ready to run programs in memory.Objective:

- * Maximize CPU utilization
- * Maximize throughput.



Multiprogramming: Multiplexing of CPU among different programs in memory.
(another definition)

Types of multiprogramming:

1. Non Preemptive

2. Preemptive

- * No forceful deallocation
- * Running program on CPU will not be forced to leave CPU. It will release CPU voluntarily:
 1. Completed execution
 2. needs I/O Syst. call.
- * Drawback: leads to starvation to other waiting programs.
 - lack of interactivity, responsiveness.

- * Forceful deallocation.
- * Program running on CPU, can get forcefully deallocated from CPU (preemption), so that waiting programs can get their chance to run on CPU.

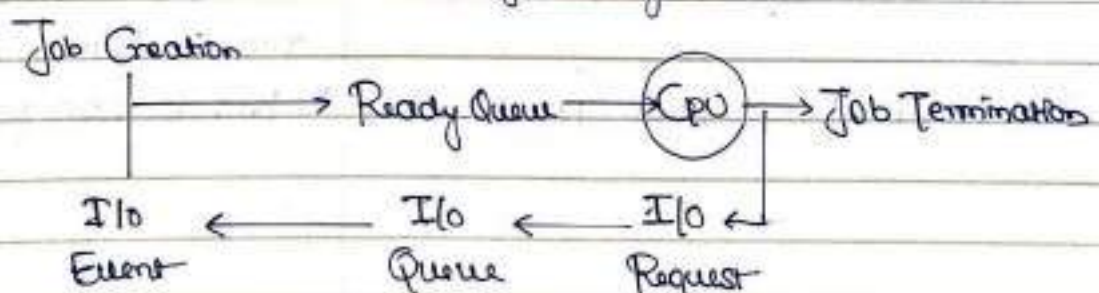
Objective:

- * improve interactivity
- * increase responsiveness

Preemption is done based on:

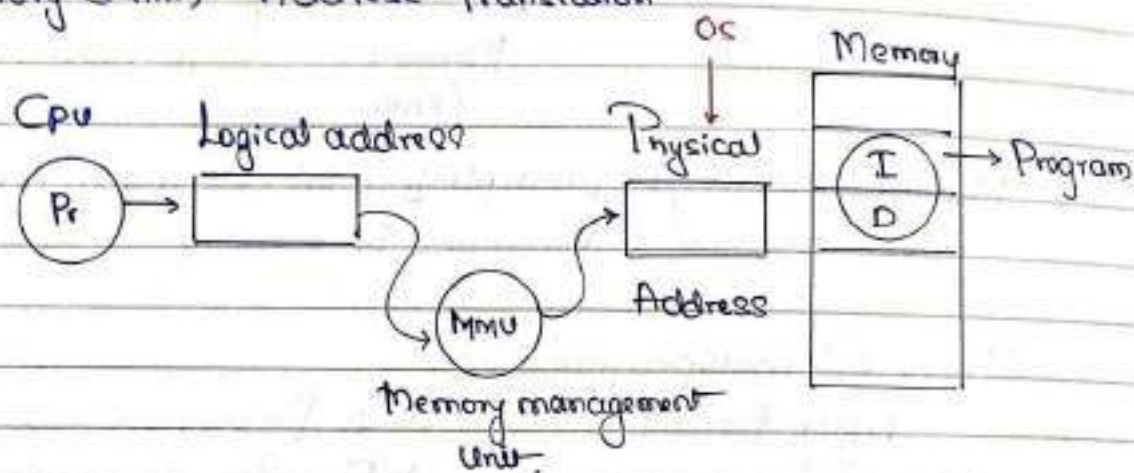
1. Priority
2. Time

Multiprogramming (Schematic View)



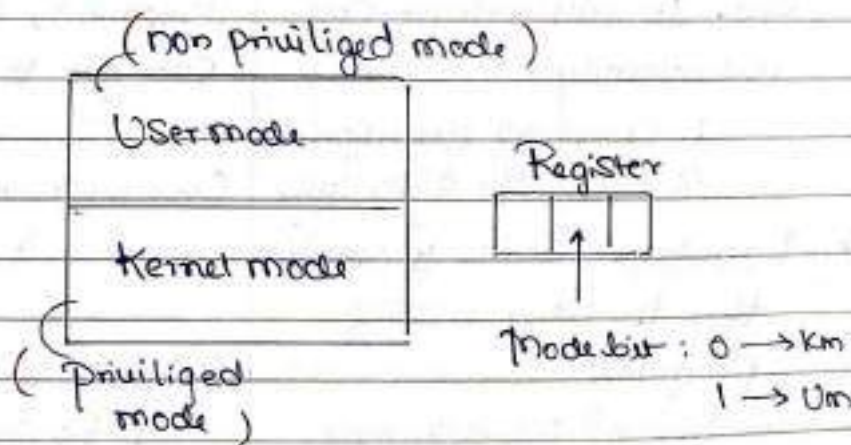
Architectural (H/W) requirements for implementing a multi programmed Operating System:

1. Secondary Storage Device (Io) : DMA [Direct Memory Access]
(Disk)
2. Memory (RAM) : Address Translation

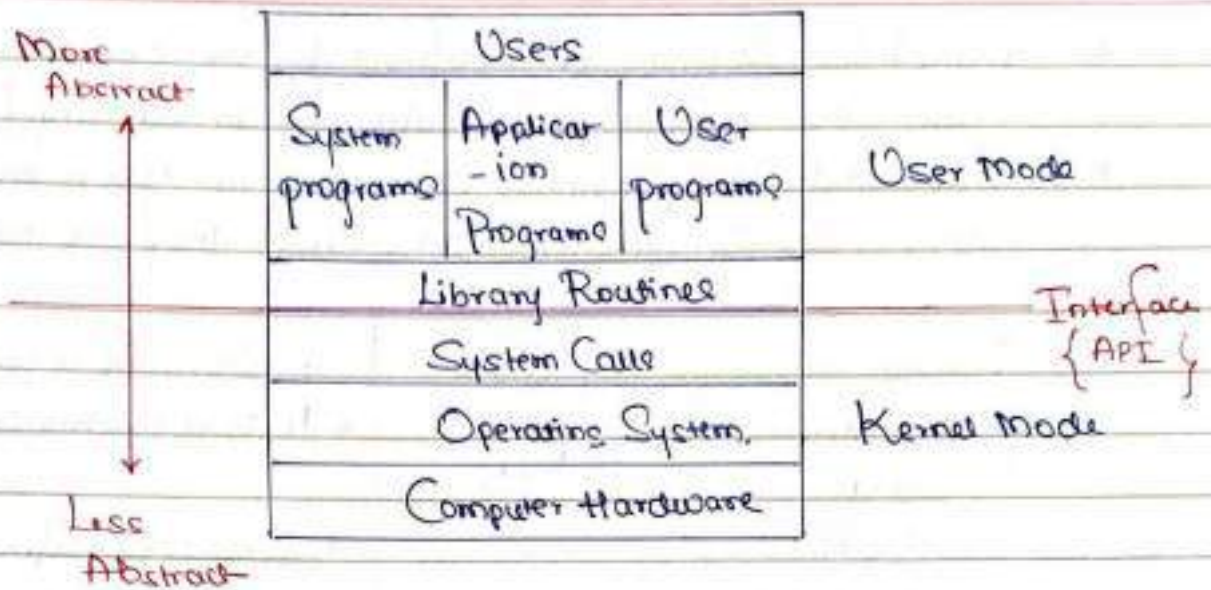


3. CPU (Processor) : Dual mode operation

Every CPU of multi programmed OS should have two mode of execution.



User mode	Kernel mode
* All user Applications run in Um.	* All OS routines programs run in K.m.
* Um is preemptive (Non Atomic)	* Km is non preemptive (Atomic)



- ★ Many a times it becomes necessary to shift the mode from user mode and kernel mode and back from kernel mode to user mode.

Mode Shifting: Shifting of mode from user mode to kernel mode and back is called mode shifting.
 Mode Shifting (Process) is needed to aware of sensor.

API: Application Programmer's Interface / System Call Interface

Implementation of mode shifting process via API/SCI

★	<pre>main() { int a, b, c; b = 1; c = 2; a = b + c; f(a); }</pre>	<pre>f(int k) { k++; printf("%d", k); }</pre> predefined fn. library fun. built-in. * Implementation is in Library file (.lib)	<pre>f();</pre> 'user defined function' * <stdio.h> file Contains function prototype, which is used for type checking
---	---	--	---

- * `printf()`, `scanf()` are defined by the compiler implementor and the compile code will be in library file.
- * User defined functions are run in user mode along with built-in functions because they are written by users.

```

* main()
{
  int a, b, c;
  a = 1;
  b = 2;
  c = a + b;
  f();
  fork(); ← Km.
  printf("%d", c);
}

f(int k)
{
  k++;
}
  
```

Um. (under main)

Um. (under f(int k))

- * `fork()` is a function
- * It is a System call } OS routine
- * `fork()` is defined / implemented in OS Kernel
- * Execution of a `fork()` results in creating a child process

```

* main()
{
  :
  f(); → BSA (privileged inst.)
  :
  fork(); → SVC
  :
}
  
```

* CT: Compile-time

* BSA: branch and save address } Instructions

* SVC: Supervisor Call

`fork();` → SVC

{ Software Interrupt instruction }

* `fork()` is converted into SVC

because it is a not user defined

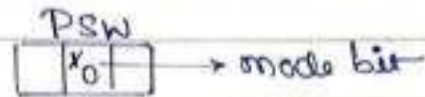
- S/W interrupt at runtime.

{ All pre defined functions are translated into instructions called BSA (non privileged instruction) }

- User mode at runtime.

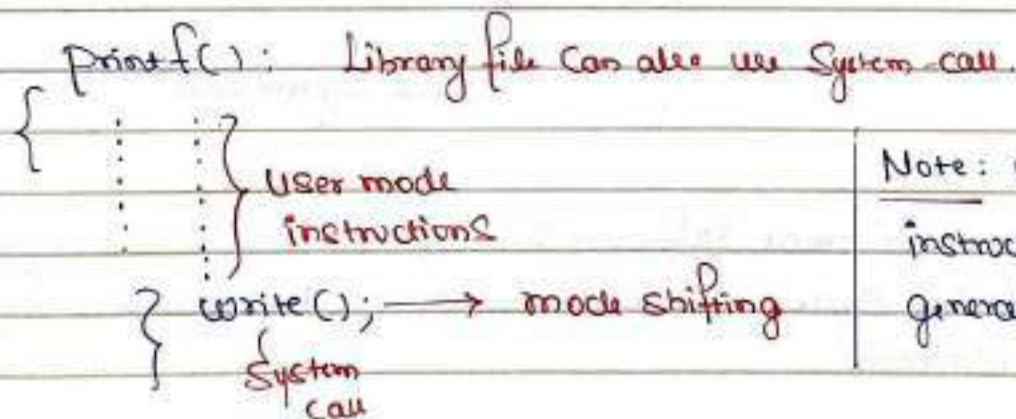
- * Interrupt generated by software instructions are called Software interrupt.
- * ISR: Interrupt Service routine: A program of OS to service the interrupt.

ISR: * ISR will go to the register and convert the mode bit from '1' to '0' to shift from user mode to kernel mode and vice versa.



* ISR examines an interrupt and determines how to handle it, executes the handling and then returns a logical interrupt value (returns the address of fork from the dispatch table).

- * All the Operating Systems - Windows, Linux, Unix follow the same paradigm.
- * The `fork()` will be executed at kernel mode in the dispatch table. All instructions will be executed of the fork and the last instruction will change the mode bit from '0' to '1' after execution and then fork is executed successfully at kernel mode and returned to user mode and then remaining code will run.
- * To change the mode from kernel mode to user mode then no software interrupt is required, interrupt is required for changing modes from UM to KM only.

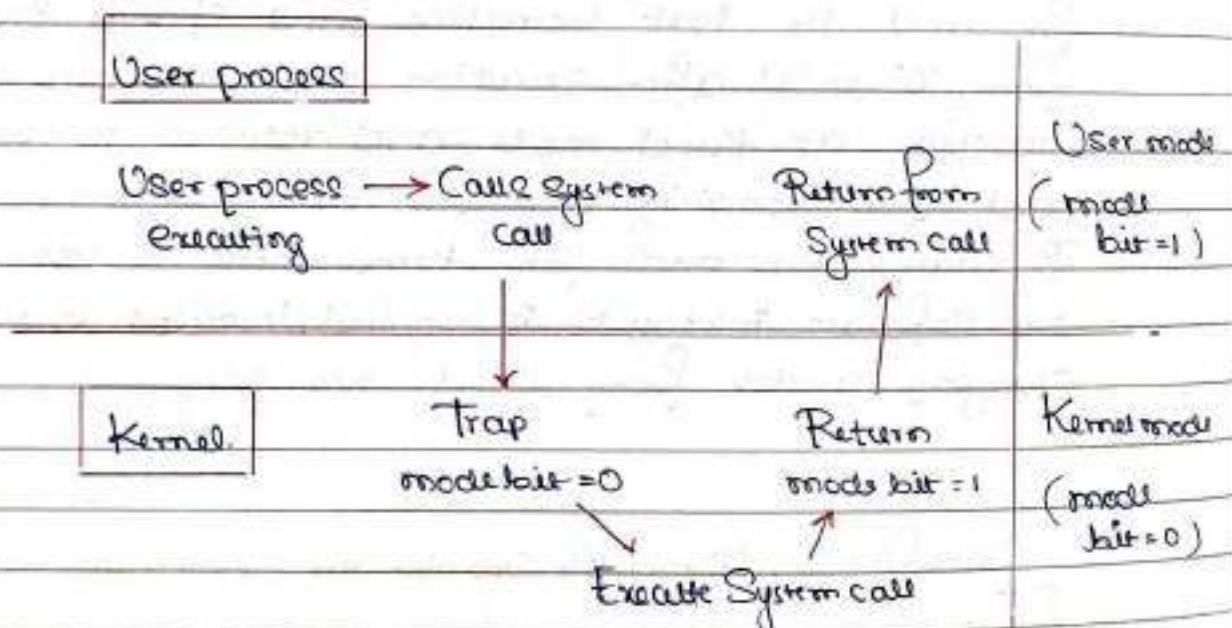


Note: every privileged instruction does not generate interrupt

1. Kernel mode: In kernel mode, the executing code has complete and unrestricted access to the underlying hardware. It can execute any CPU instruction and reference

any memory address. Kernel mode is generally reserved for lowest-level, most-trusted functions of the operating system. Crashes in kernel mode are catastrophic, they will bring the entire PC.

2. User mode: In user mode the executing code has no ability to directly access hardware or reference memory. Code running in user mode must delegate to system APIs to access hardware or memory. Due to the protection afforded by this sort of isolation, crashes in user mode are always recoverable. Most of the code running on your computer will execute in user mode.



Q1. A processor needs software interrupt to
: Obtain system service which need execution of privileged instructions.

Q2. A CPU has 2 modes Privileged & non privileged. In order to change the mode from privileged to non privileged:
: A privileged instruction (which does not generate an interrupt) is needed.

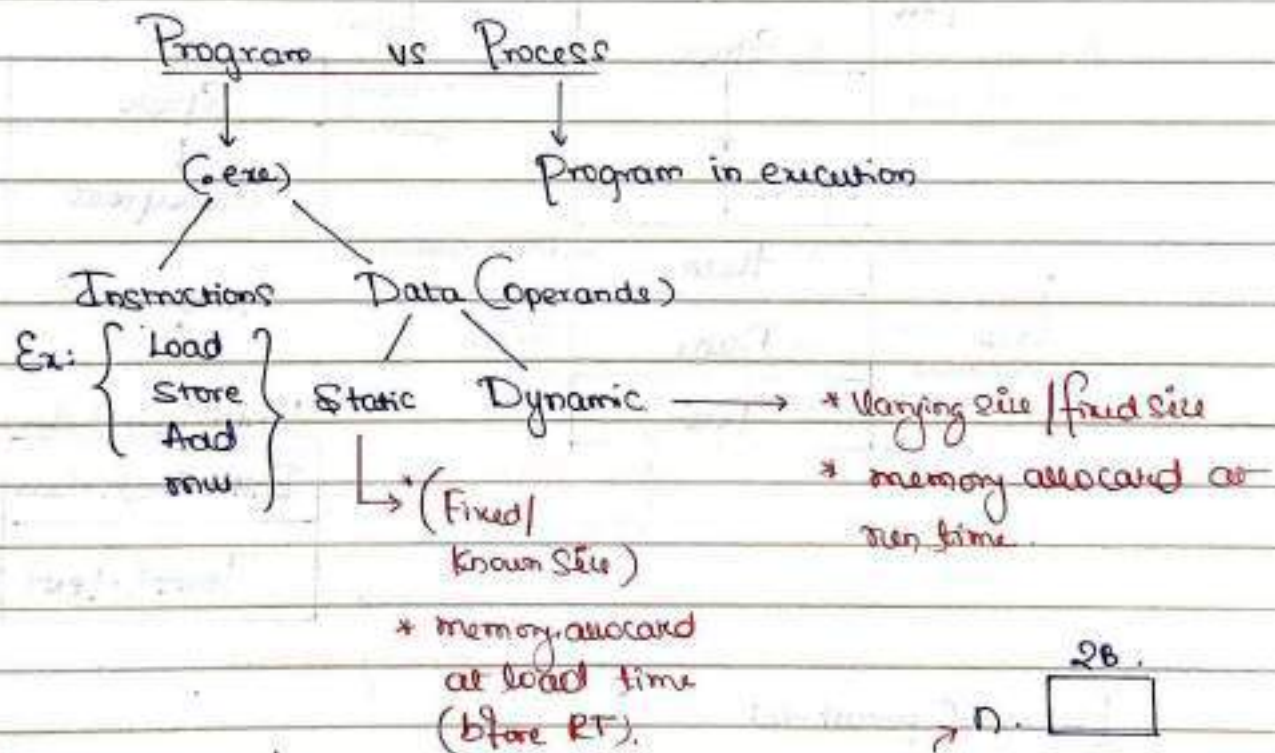
Process Management

Q3. System call are usually invoked by using:
: Software interrupt

Q4. A computer handles several interrupt source of which of the following are relevant for this question:

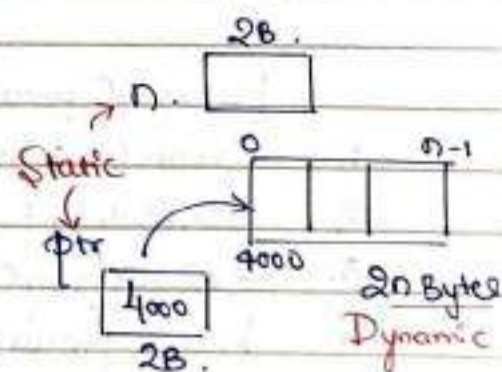
- * Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- * Interrupt from mouse (raises interrupt if it is moved / button is pressed)
- * Interrupt from keyboard (" " key is pressed / released)
- * " " Harddisk (" " disk read is completed)

Which one of these will be handled at the highest priority?
: Interrupt from CPU temperature



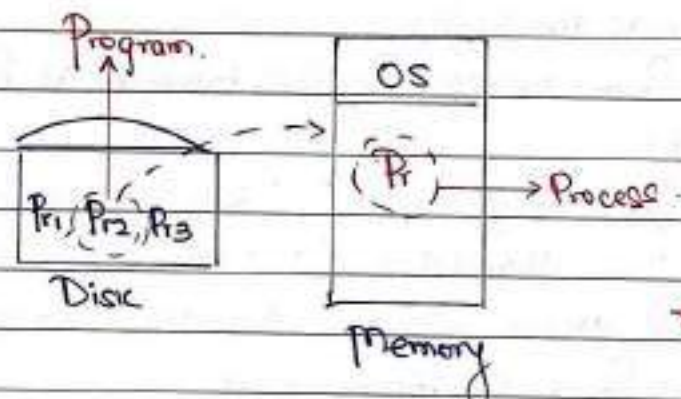
Static and Dynamic

Ex: `int n; int *ptr;`
`scanf("%d", &n);`
`ptr = (int*) malloc(sizeof(int)*n);`



Process: * Program in Execution

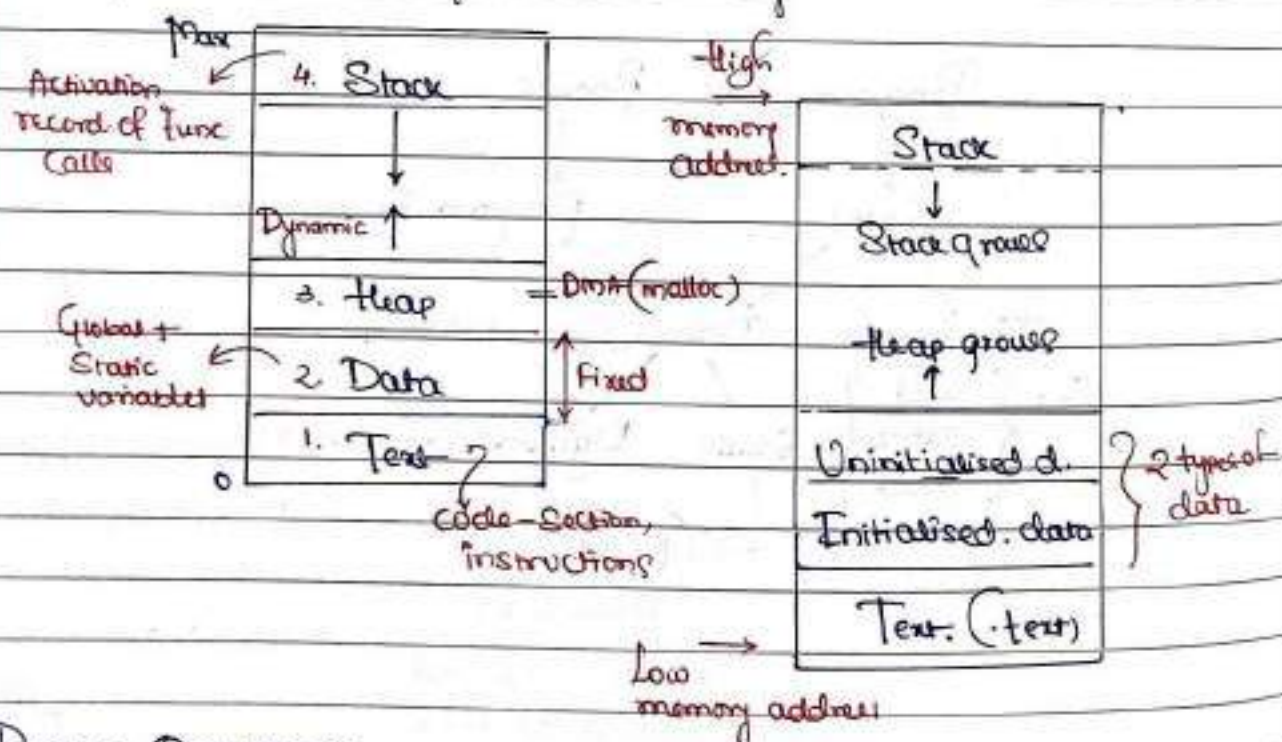
- * When program is loaded from Disk to main memory then program is converted to process.
- * Instance of a program.
- * Locus of control
- * Animated Spirit.



- * Program is passive.
- * Program is always on disk.

* Process is an abstract data type.

Representation of process in memory.



Process Operations:

1. Create(): resource allocation
2. Schedule(): the act of selecting process to run on CPU
3. execute/run(): the act of executing instruction from code section
4. block/wait: for I/O operation / system call
5. Suspend: act of moving process from memory to disk.

6. Resume : act of moving process from disk to memory
7. Terminate : act of resource deallocation.

Process as an entity is associated with several attributes :

1. Identification : $\langle \text{Pid, gid, Ppid} \dots \rangle$
 2. CPU related : $\langle \text{Program Counter, Priority, State, burst time} \dots \rangle$
 3. Memory related : $\langle \text{Size, limits,} \dots \rangle$
 4. File related : $\langle \text{list of files,} \dots \rangle$
 5. Accounting : $\langle \text{resources} \dots \rangle$
- And many more attributes but these are some important ones.

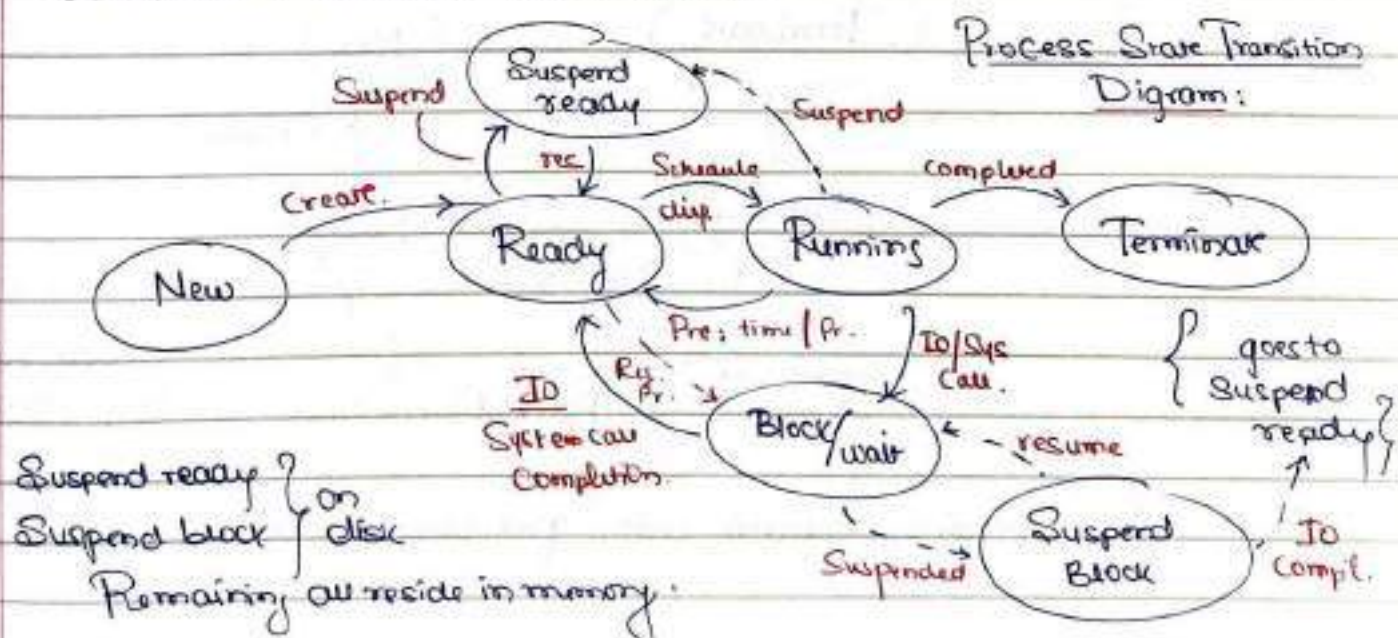
PCB (Process Control Block) :

- * Contains all attributes of process.
- * each process has its own PCB.
- * PCB is stored in memory.

★ Process Content
: content of PCB

Process States

1. new : a process is created and resource is allocated
2. ready : ready to run on CPU
3. running : executing instructions on CPU
4. block/wait : process needs to perform IO/system call.
5. terminate : resource deallocation.



Process State transition diagram of uni programmed O.S.



or just

* just adding a transition from running to ready state, then the system becomes pre-emptive.

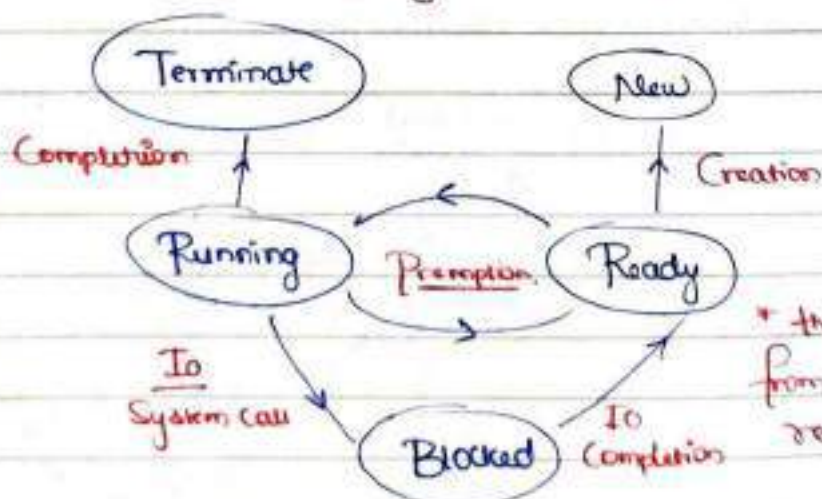
Notes: 1. As long as the process is in ready + running + block states, it is in main memory.

2. There can be many ready, block processes.

3. Maximum number of running processes depends on no. of CPUs.

4. The process may go from ready state to block/wait state because of any missing resource.

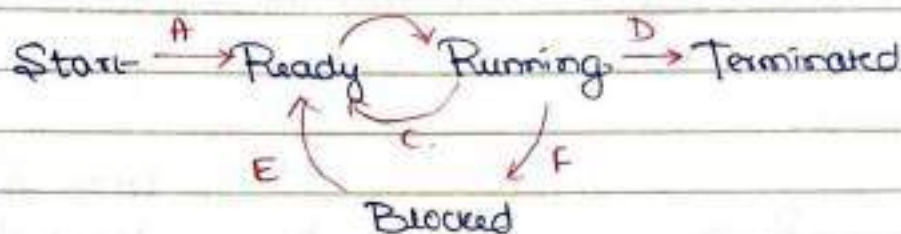
Q1. The process state transition diagram given below is representation of: Ans: An Operating System with preemptive schedule.



Ans: An Operating System with preemptive scheduler.

Q2. In the following process state transition diagram for a Uniprocessor system, assume that there are always some processes in ready state:

- (i) If process makes a transition D, it would result in another process making transition A immediately.
- (ii) A process P₂ is blocked state can make transition E while another process P₁ is in running state.
- (iii) The OS uses preemptive scheduling.
- (iv) The OS uses non-preemptive scheduling.



Which of the above statements are true? → II and III.

Q3. Which combination of the following feature will suffice to characterise an OS as multi-programmed OS?

(a) More than one program may be loaded into main memory at the same time for execution.

Ans:

only 'a'

(b) If a program waits for certain events such as I/O, another program is immediately scheduled for execution.

(c) If the execution of program terminates, another program is immediately scheduled for execution.

Q4. The maximum number of processes that can be in ready state for a computer system with 'n' CPUs:

Ans: Independent of n. (theoretically there can be infinite no. of programs in ready state).

* Ans will be n if 'running state' was asked.

Q5. Consider the following statements about process state transitions for a system using preemptive scheduling.

- I. A running process can move to ready state
- II. A ready process can move to running state
- III. A blocked process can move to running state
- IV. A blocked process can move to ready state.

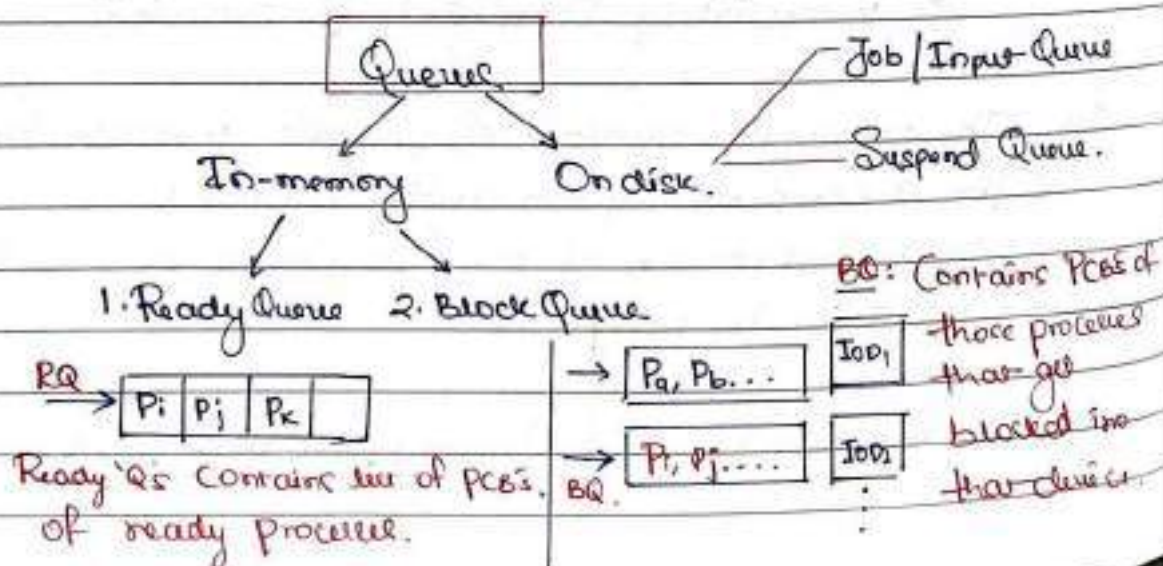
Which of the above statements are true? Ans: I, II, IV

Q6. Consider a system having 'n' CPUs ($n \geq 1$) and 'k' processes ($k \geq n$). Calculate lower bound and upper bound of the number of processes that can be in the ready, running and blocked states.

	L.B.	U.P.	$n = \text{no. of CPUs } (n \geq 1)$ $k = \text{no. of processes } (k \geq n)$
Ready	0	k	
Running	0	n	
Block.	0	k	

	L.B.	U.P.	$n = \text{no. of CPUs } (n \geq 1)$ $k = \text{no. of processes } (k < n)$
Ready	0	k	
Running	0	k	
Block	0	k	

Scheduling Queue and State - queuing diagram

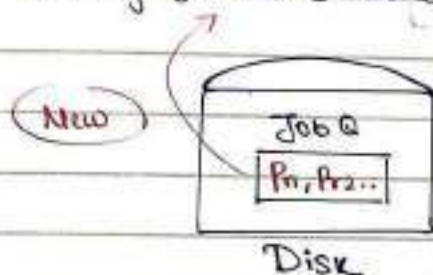


Queue on disk

1. Job Queue

Input Queue.

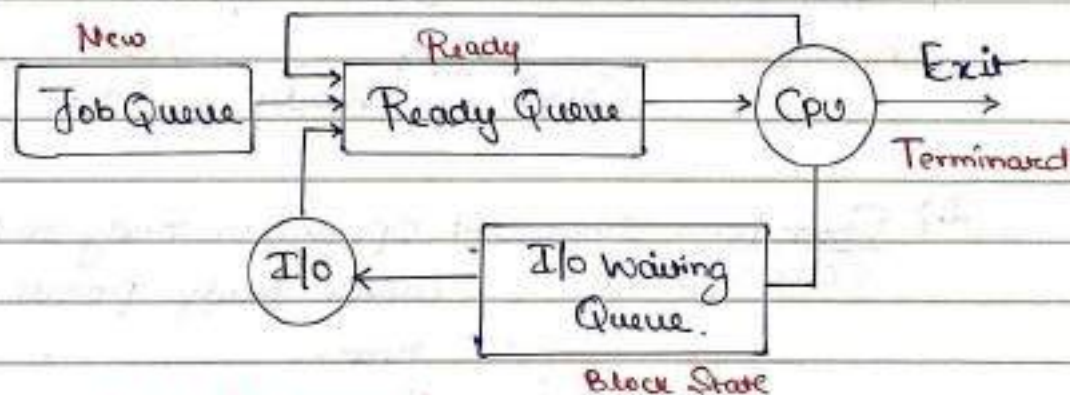
* Contains programs that are ready to be loaded in memory.



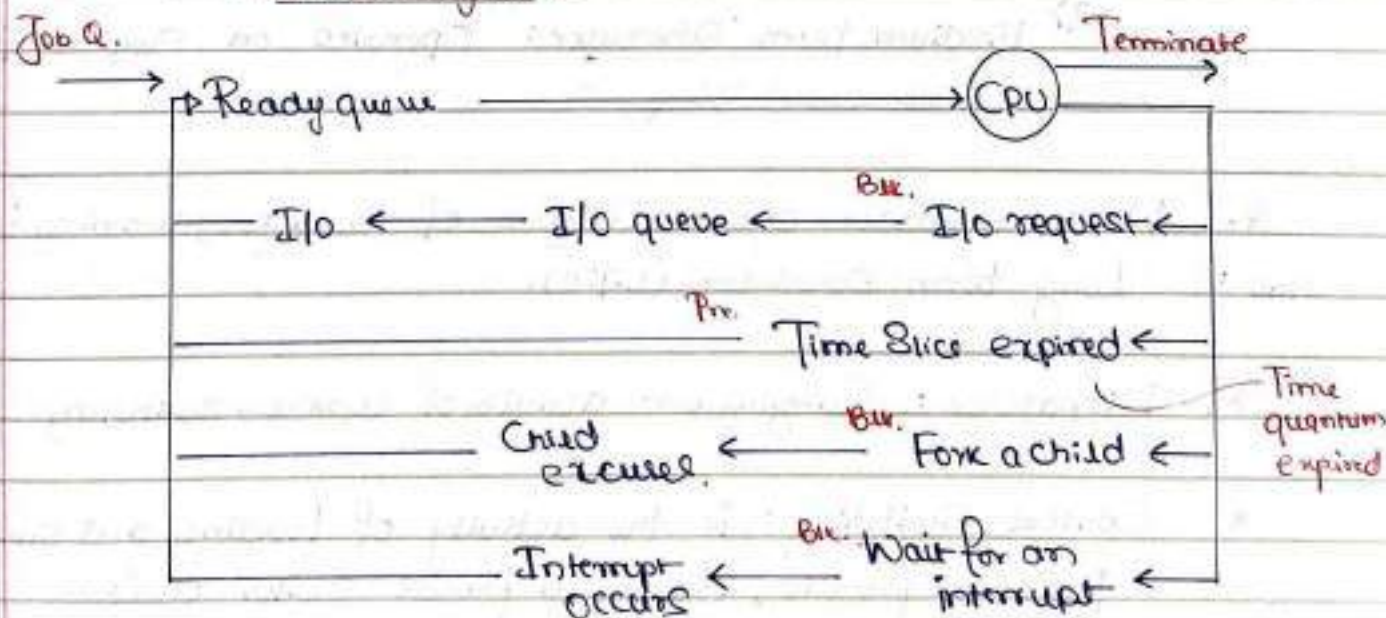
2. Suspend Queue

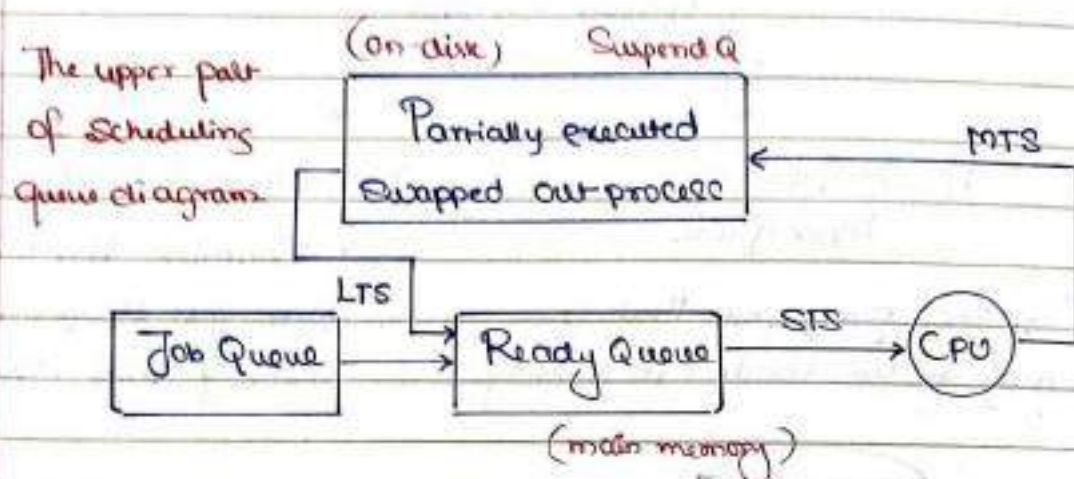
* Contains list of processes that get suspended from memory into disk.

State Queuing Diagram:



Detailed diagram ↓



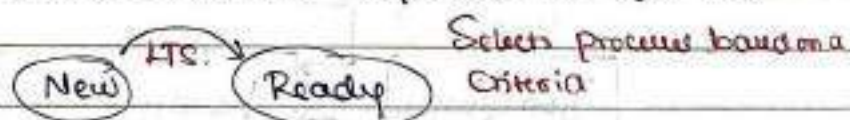


Schedulers and Dispatchers

- * Scheduling means making a decision
- * Scheduler is a component of O.S that makes decisions.
- * There are three types of Schedulers:

1. Long term Scheduler (LTS): Operates on Job-Q.

(CPU Scheduler)



2. Short term Scheduler: Operates on ready to Queue, to decide which ready process should run next.
(CPU Scheduler)

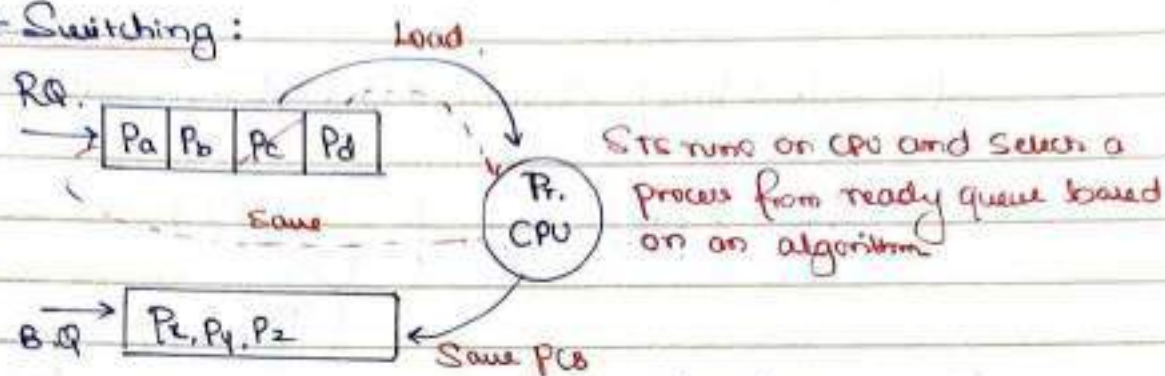
3. Medium term Scheduler: Operates on Suspend Queue.
also called "Swapper".

Q. Which scheduler controls degree of multi programming?

Ans: Long term Scheduler (L.T.S).

* Dispatcher: Carrying out activity of context-switching.

* Content Switching: is the activity of loading and saving the PCBs of processes, during a process switch on CPU.

Context Switching :

- ★ Time taken by dispatcher to load and save the PCBs is known as "Context Switching time" or "CPU Scheduling Overhead" or "dispatch latency".

- Q1. Let the time taken to switch between user and kernel model of execution be t_1 , while the time taken to switch between two user processes be t_2 . Which of the following is true?

Context switching time = t_2

mode shifting time = t_1

$\therefore t_1 < t_2$

* process switching involves mode shifting time.

- Q2. Dispatch latency is defined as:

The time to switch from one process to another on CPU

- Q3. Which of the following is an invalid process state transition?

a. Ready - running ✓

b. Running - terminate ✓

c. Block - ready ✓

d. Ready - suspend ✓

e. Suspend - running ✗

f. block - terminate ✗

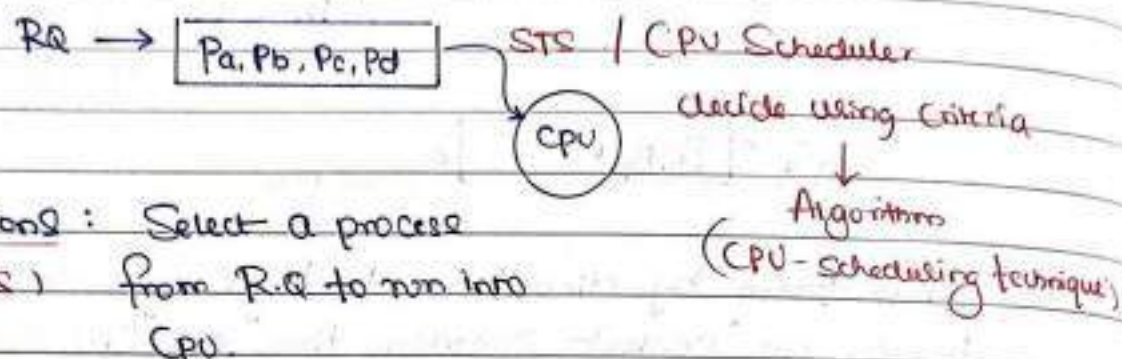
g. Suspend - ready ✓

h. block - new ✗

i. Suspend - terminate ✗

CPU Scheduling

CPU Scheduling: design and implementation of S.T.S.



Functions: Select a process (S.T.S) from R.Q to run into CPU.

Goals (Scheduling Criteria):

- * Maximize CPU utilization (throughput)
- * Minimise waiting time (WT), turn around time (TAT), response time (RT).

Process Times:

1. Arrival time (AT): first time when the process comes from new to ready state is called arrival time.



2. Waiting time (WT): total time spent by the process in ready queue.

3. Burst time (BT): time spent by the process executing in the CPU is called CPU burst time.

4. I/O Burst time (IOBT): time spent by the process in block queue performing I/O operation is called I/O burst time.

5. Completion time: time at which process completes.

6. Turn Around time: the time taken by the process from its arrival time to completion time is called turn around time.

$$TAT = C.T - A.T.$$

7. Waiting time (WT): total time spent by the process in ready queue is called waiting time.

$$WT = TAT - (BT + IOST)$$

Note:

1. n processes $(P_1 \dots P_n)$

2. $AT(P_i) = A_i$

3. $BT(P_i) = X_i$

4. $IOBT(P_i) = Y_i$

5. $CT(P_i) = C_i$

$$a) TAT(P_i) = (C_i - A_i)$$

$$\text{Average TAT} = \frac{\sum_{i=1}^n (C_i - A_i)}{n}$$

$$b) WT(P_i) = (C_i - A_i) - (X_i + Y_i)$$

$$\text{Average WT} = \frac{\sum_{i=1}^n (C_i - A_i) - (X_i + Y_i)}{n}$$

Schedule length: Total time taken to complete all n processes as per schedule

(L)

* No. of schedules with n -processes = $n!$ (non-preemptive)

* No. of schedules with n -processes = ∞ (pre-emptive)

$$L = (\text{completion time of last process}) - (A.T \text{ of last process})$$

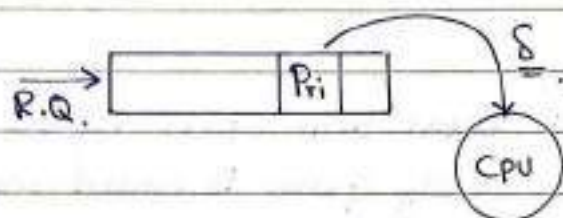
$$\Rightarrow \max(C_i) - \min(A_i)$$

Throughput = no. of processes completed per unit time.

$$\eta = \frac{n}{L}$$

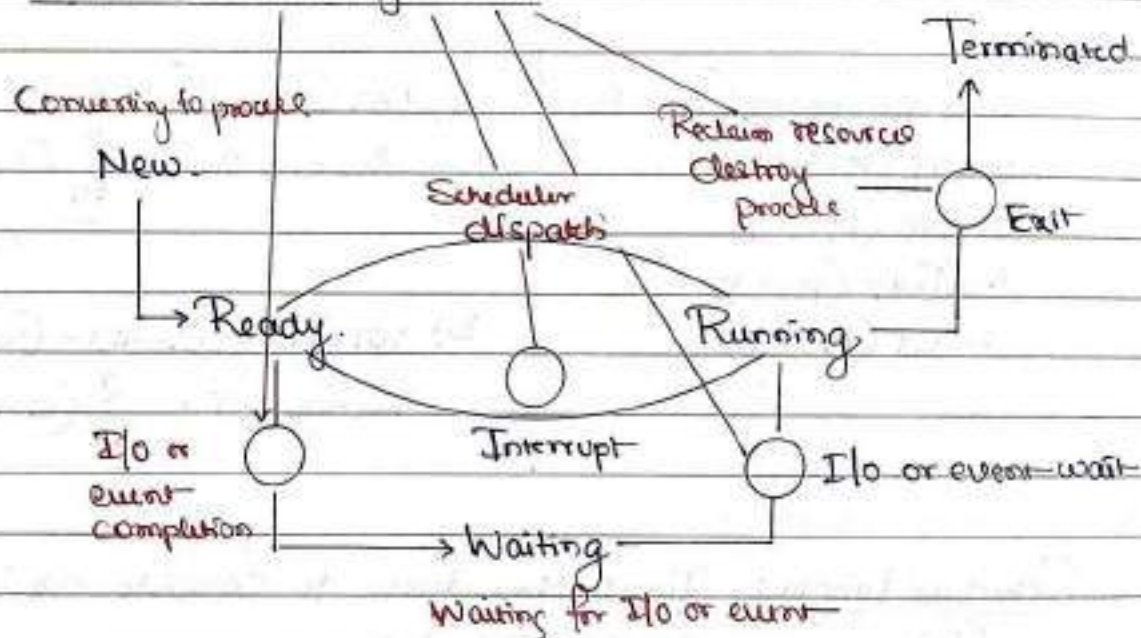
Context-Switch time : CPU Scheduling Overhead

Let CPU scheduling overhead = " S "



Time taken to load Pcs to process from Ready queue onto CPU

CPU Scheduling Occurs



Types of CPU Scheduling :

1. Preemptive
2. Non preemptive

Scheduling Criteria :

Maximize
* CPU utilization

Minimize
* Turnaround time
* Waiting time
* Response time

CPU - I/O Bursts:

Process execution consists of a cycle of CPU execution and I/O wait:

- * different processes may have distributions of bursts.
- * CPU bound processes: perform lots of computations in long bursts, very little I/O.
- * I/O bound process: perform lots of I/O followed by short burst of computation.
- * ideally, the system admits a mix of CPU bound I/O bound processes to maximise CPU and I/O.

Example:

```

      :
Load store      }
add store      } CPU Burst
read from file }

wait for I/O   } I/O Burst

Store increment }
index          } CPU Burst
write of file  }

wait for I/O   } I/O Burst
Load store     }
add store     } CPU Burst
read from     }
file          }

wait for I/O   } I/O Burst
      :
  
```

1. FCFS < First Come First Serve >

- * Selection Criteria: A.T
- * Mode of operation: Non pre-emptive.

* Conflict resolution: Lower process id.

* Assumption:

(i) Time is in clock ticks

(ii) No I/Os

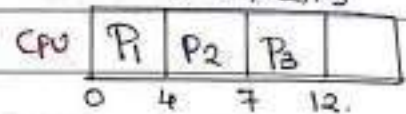
(iii) Scheduling overheads (δ) = '0'.

Ex:1

P.no.	A.T.	B.T.	C.T.	TAT	WT.
1	0	4	4	4	0
2	0	3	7	7	4
3	0	5	12	12	7

RA

→ P₁, P₂, P₃



$$\text{Avg TAT} = \frac{23}{3}$$

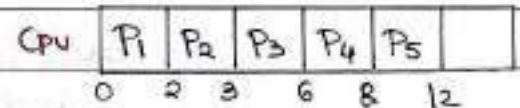
$$\text{Avg WT} = \frac{11}{3}$$

Quant Chant → Time

$$L = 12$$

Ex:2

P.no	A.T	B.T	C.T	TAT	WT.
1	0	2	2	2	0
2	0	1	3	3	2
3	2	3	6	4	1
4	3	2	8	5	3
5	5	4	12	7	3

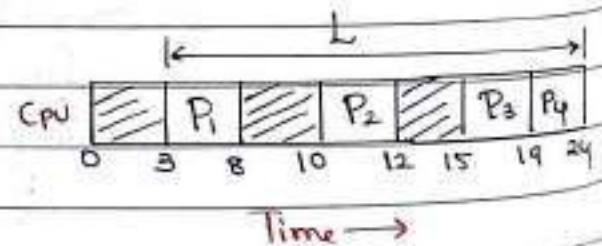


Time →

$$\text{Avg TAT} = \frac{21}{5}, \text{ Avg WT} = \frac{9}{5}, L = 12, \eta = \frac{\eta}{L} = \frac{5}{12}$$

Ex:3

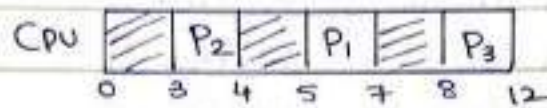
P.no	A.T	B.T	C.T	TAT	WT
1	3	5	8	5	0
2	10	2	12	2	0
3	15	4	19	4	0
4	18	5	24	6	1



$$\% \text{ CPU idleness} = \frac{5}{21}$$

Ex. 4.

P.no	A.T	B.T
1	5	2
2	3	1
3	8	4



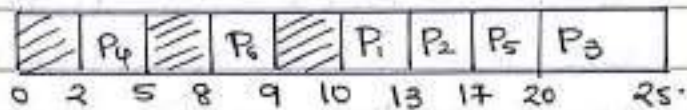
$$L = 12 - 3 = 9$$

Time →

Ex. 5.

P.no	A.T	B.T
1	10	3
2	11	4
3	20	5
4	2	3
5	12	3
6	8	1

→ RQ : P₄ P₆ P₁ P₅ P₃.

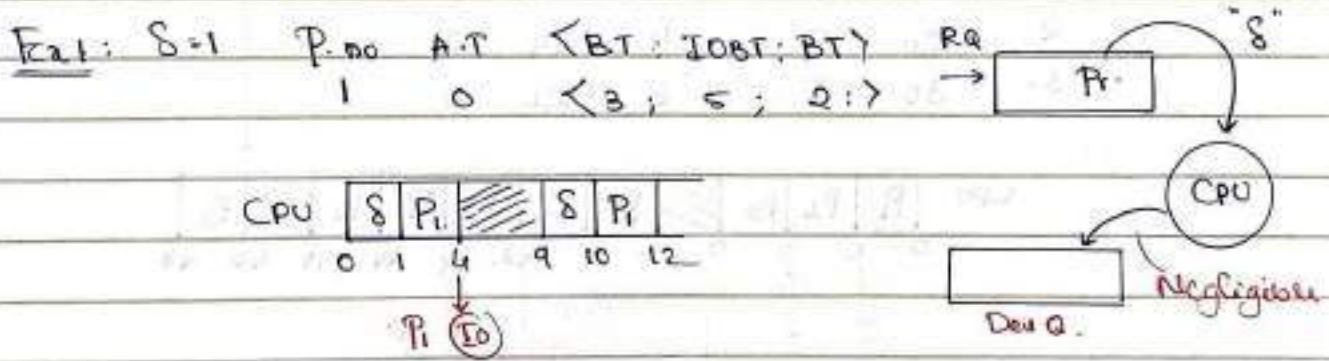


$$L = 25 - 2 = 23$$

∴ CPU idleness = $\frac{4}{23}$

CPU Scheduling Techniques / Algorithms

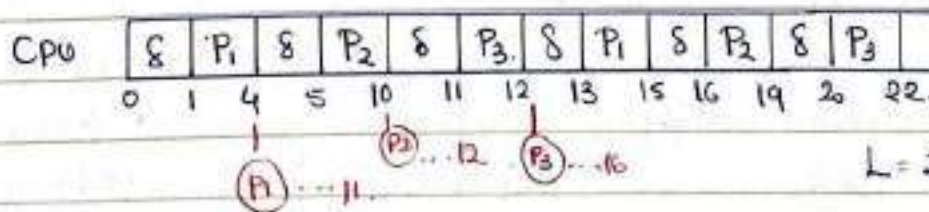
1. FCFS: Scheduling with Io-BT's and CPU scheduling overheads.



Ex. 2: $S=1$.

P.no	A.T	<B.T, IOBT, BT>
1	0	3 7 2
2	2	5 2 3
3	5	1 4 2

System has multiple Io devices.



$$TAT(P_1) = 15 \quad WT(P_1) = 1$$

$$TAT(P_2) = 17 \quad WT(P_2) = 5$$

$$TAT(P_3) = 17 \quad WT(P_3) = 8$$

$$\% \text{ CPU Idleness} = 0\%$$

* With ' $\delta > 0$ ' the formula for WT becomes

$$WT = TAT - (BT + IOBT + n \cdot \delta)$$

* n = no of times process gets scheduled into CPU.

Q1. Consider three processes P_1, P_2, P_3 arriving in the ready queue at time 0 in the order P_1, P_2, P_3 . Their service time requirements are 10, 20 and 30 unit respectively. Each process spends 20% of its service time on I/O followed by 70% of its service time on Computation at CPU and last 10% on I/O before completion. Assuming Concurrent I/O and negligible scheduling overhead, Calculate FCFS Scheduling:

(i) Average TAT (ii) % CPU Idleness.

$$L = 47$$

$$\% \text{ CPU idleness} = \frac{5}{47}$$

$$\text{Avg TAT} = \frac{44 + 44 + 47}{3} = \underline{44}$$

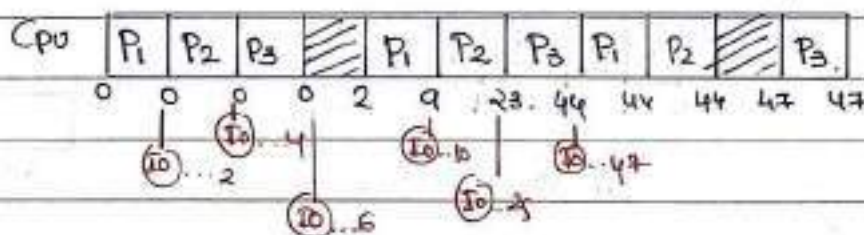
Solⁿ:

P.no S.T AT <IOBT: BT; IOBT>

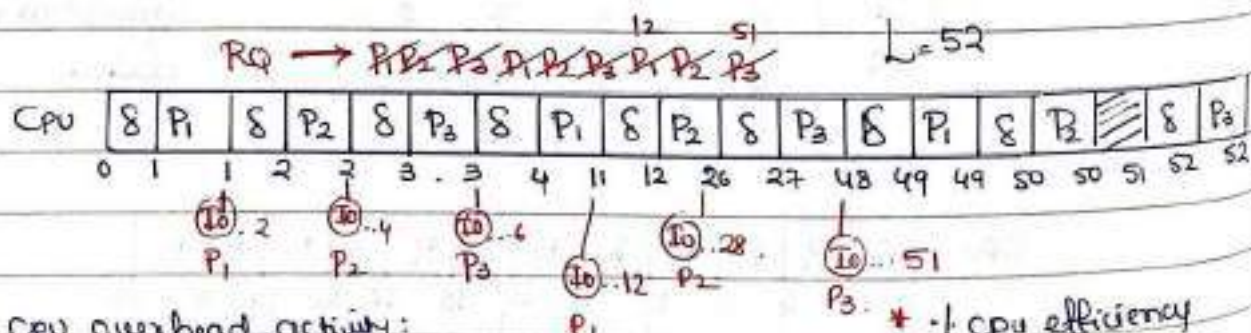
1 10 0 2 7 1

2 20 0 4 14 2

3 30 0 6 21 3



Above problem with $\delta = 1$.



* % CPU overhead activity:

$$\Rightarrow \underline{9/52}$$

* % CPU idleness = $\underline{1/52}$

* % CPU efficiency

$$\Rightarrow 100 - \left(\frac{9}{52} + \frac{1}{52} \right)$$

Homework: Repeat Q 2 and 3 assuming system has only one IO-device.

1)	P.no	AT	BT	IOBT	BT
$\delta=2$	1	0	3	4	2
	2	5	2	0	3
	3	20	1	15	6

2)	P.no	AT	IO	BT	IOBT
$\delta=1$	1	3	5	2	3
	2	8	2	10	4
	3	12	6	12	1

2. Shortest Job First (SJF) / Shortest Process Next (SPN)

- * Selection Criteria: Burst Time
- * Mode of operation: Non preemptive
- * Conflict resolution: Lower process id.
 - : Among the processes present in RQ select the one having least BT and schedule it on CPU.

Ex 1	P.no	AT	BT
	1	0	2
	2	2	3
	3	3	1
	4	5	2
	5	8	3
	6	10	2

RQ: P₁ P₂ P₃ P₄ P₅ P₆

CPU	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
	0	2	5	6	8	11
						13

Ex 2	P.no	AT	BT
	1	10	1
	2	4	2
	3	6	4
	4	2	3
	5	5	2
	6	3	2

RQ: P₄, P₂, P₅, P₃

CPU	P ₄	P ₂	P ₅	P ₃
	0	2	5	7
				9
				13

	P ₄	P ₂	P ₅	P ₆	P ₁	P ₃
	0	2	5	7	9	11
						12
						16

$L = 16 - 2 = 14$

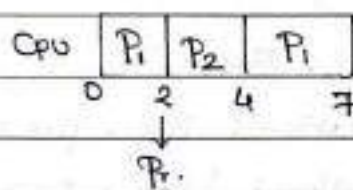
% CPU idleness = 0

3. Shortest Remaining Time First (SRTF) / Pre-emptive SJF

- * Selection Criteria: BT
 - * mode of operation: Pre-emptive
 - * Conflict resolution: Lower process id
- Pre-emption of running process is based on availability of Strictly Shorter process.

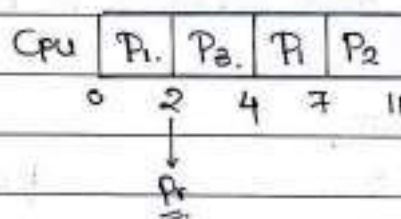
Ex: 1.

P.no	AT	BT
1	0	5
2	2	2



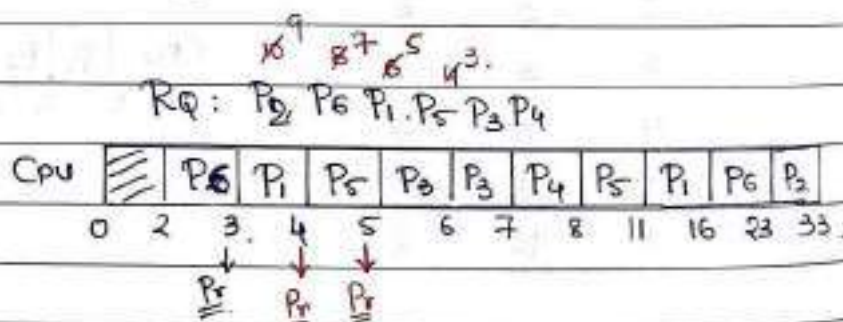
Ex: 2

P.no	AT	BT
1	0	5
2	1	4
3	2	2



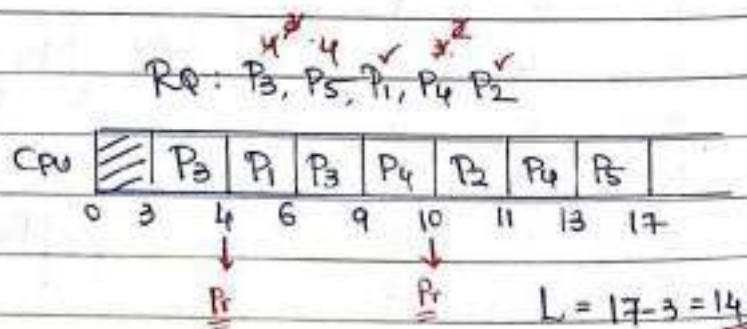
Ex: 3

P.no	AT	BT
1	3	6
2	2	10
3	5	2
4	6	1
5	4	4
6	2	8



Ex: 4

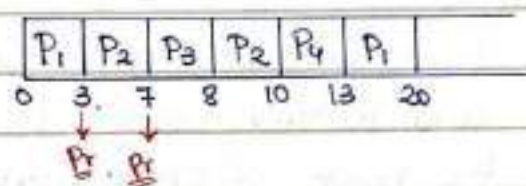
P.no	AT	BT
1	4	2
2	10	1
3	3	4
4	5	3
5	3	4



- Q1. Consider the following processes, with AT and length of the CPU burst given in ms. The scheduling algo used is preemptive SRTF.

Process	AT	B.T
P ₁	0	10
P ₂	3	6
P ₃	7	1
P ₄	8	3

The average turn around time of these processes is 8.25



$$\text{Average TAT} = \frac{20 + 7 + 1 + 5}{4}$$

$$= \frac{33}{4} = 8.25$$

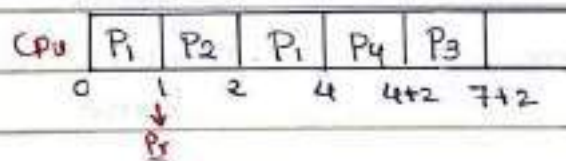
Q2.

Process	P ₁	P ₂	P ₃	P ₄
AT	0	1	3	4
B.T	3	1	3	7 (z)

The processes are run on a single processor using pre-emptive SRTF. If avg WT = 1 ms, then value of z = ?

Average WT = 1 ms (SRTF)

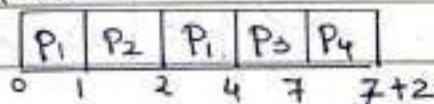
→ Case 1: $z < 3$.



$$\text{Avg WT} = \frac{1 + 0 + (z + 1) + 0}{4}$$

$$1 = \frac{z + 2}{4} \therefore z = 2$$

Case 2: $z > 3$.



$$\text{Avg WT} = \frac{1 + 0 + 1 + 3}{4} = \frac{5}{4} = 1.25$$

Because Avg WT = 1. so z is less than 3.

Performance of SJF/SRTF:

1. It favors shorter process:

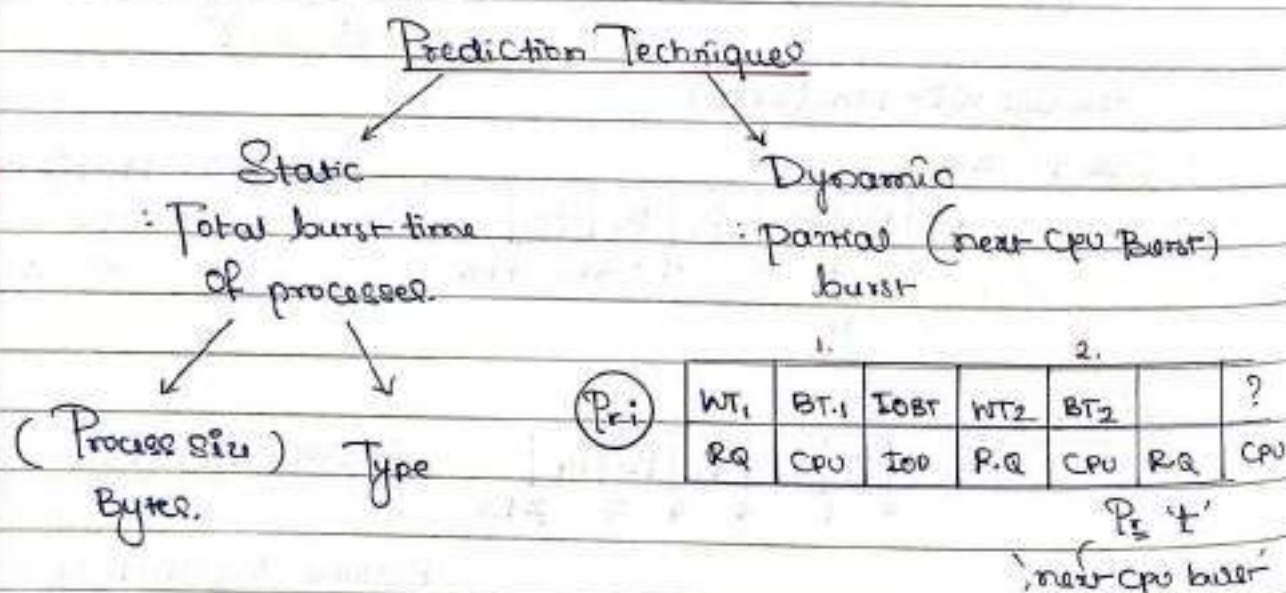
- Advantages: * Complete more processes, more throughput
* minimizing of Avg TAT, Avg WT.
- Drawback: * Starvation to longer processes.

- Note: * SJF/SRTF is considered the "optimal algorithm"
- * Since B.T's of processes are not known a priori, SJF/SRTF is "not practically implementable"!
 - * SJF is used as a benchmark to measure performance of other algorithms.
 - * SJF can be implemented with predicted burst times.

Q3. Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the lengths of CPU's bursts, the min achievable average waiting time for these three processes in non-preemptive scheduler is 12 milliseconds.

Cpu	P ₃	P ₁	P ₂	
	0	10	26	46

$$\text{Avg WT} = \frac{10+26+10}{3} = \frac{36}{3} = 12$$



Exponential Averaging Technique / Aging Algorithm: (To predict near CPU burst)

Let P_i : process
 Let t_i : completed BT
 Let T_i : predicted BT

Let T_{n+1} denote next predicted BT

$$T_{n+1} = \alpha t_n + (1-\alpha) T_n$$

$$0 < \alpha < 1$$

Recurrence Relation:

$$\left. \begin{aligned} T_{n+1} &= \alpha t_n + (1-\alpha) T_n \text{---(I)} \\ T_n &= (\alpha t_{n-1} + (1-\alpha) T_{n-1}) \text{---(II)} \end{aligned} \right\} \text{Back Substitution}$$

$$T_{n+1} = \alpha t_n + (1-\alpha) [\alpha t_{n-1} + (1-\alpha) T_{n-1}]$$

$$= \alpha t_n + \alpha (1-\alpha) t_{n-1} + (1-\alpha)^2 T_{n-1} \text{---(III)}$$

$$= \alpha t_n + \alpha (1-\alpha) t_{n-1} + \alpha (1-\alpha)^2 t_{n-2} + (1-\alpha)^3 T_{n-2} \text{---(IV)}$$

(T_1 = initial guess)

Given the value of ' α ' and T_1 one can predict next CPU B.T.

Q1. Consider a system using exponential averaging technique to predict next CPU BT. Given $\alpha=0.5$ and $T_1=10$. The process B.T.S in previous runs are: 4, 8, 12, 10. Predict the next CPU burst time of process.

Solⁿ:- $T_5 = ?$

4, 8, 12, 10

$\downarrow$
 t_1, t_2, t_3, t_4

$$T_{n+1} = \alpha t_n + (1-\alpha) T_n \text{---(I)}$$

$$T_5 = 0.5 \times t_4 + 0.5 \times T_4 \quad 9.75$$

$$= \frac{1}{2} \times (t_4 + T_4) = \frac{1}{2} (10 + T_4)$$

$$T_4 = \frac{1}{2} (t_3 + T_3) = \frac{1}{2} (12 + T_3)$$

$$= \frac{19.5}{2} = 9.75$$

$$T_3 = \frac{1}{2} (t_2 + T_2) = \frac{1}{2} (8 + T_2) = 7.5$$

$$T_2 = \frac{1}{2} (t_1 + T_1) = \frac{1}{2} (4 + 10)$$

$$T_2 = 7$$

$$\therefore T_5 = \frac{1}{2} (10 + 9.75) = \frac{19.75}{2} = 9.875$$

4. Highest Response Ratio Next (HRRN)

→ : HRRN not only favours shorter process but also limits the waiting time of longer process.

* Selection Criteria: Response ratio : $\frac{W.T + BT}{BT}$

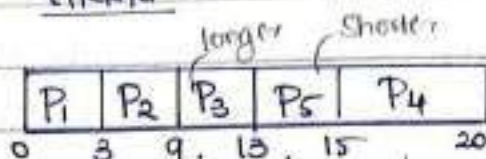
* Mode of operation: Non pre-emp.

Ex 1.

P.no AT BT

HRRN:

1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



$$t_{13} \quad P_4 = 7.15/5 = 1.2/5$$

$$P_5 = \frac{5.12}{2} = 2.5$$

Higher

$$R_{P_2} = 5 + 3 = 8$$

$$R_{P_3} = 1.4$$

$$R_{P_4} = 2.0$$

5. Longest Remaining Time First (LRTF)

→ It is just opposite of longest or smallest remaining time first.

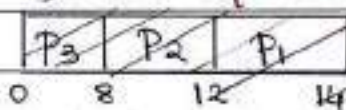
* Selection Criteria: Burst Time

* Mode of operation: Pre-emptive.

Ex 1.

P.no	AT	BT
1	0	2
2	0	4
3	0	8

CPU: (its pre-emp)



TAT:

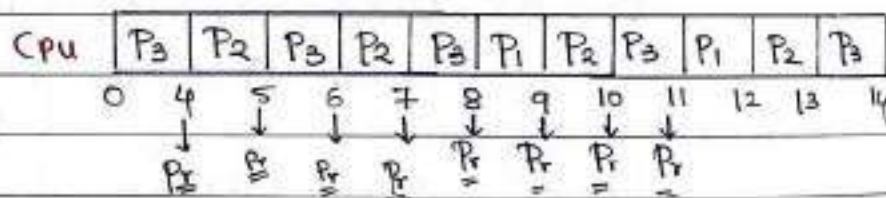
$$P_1 = 14 - 0 = 14$$

$$P_2 = 12 - 0 = 12$$

$$P_3 = 8 - 0 = 8$$

$$\text{Avg T.A.T} = \frac{12+13+14}{3}$$

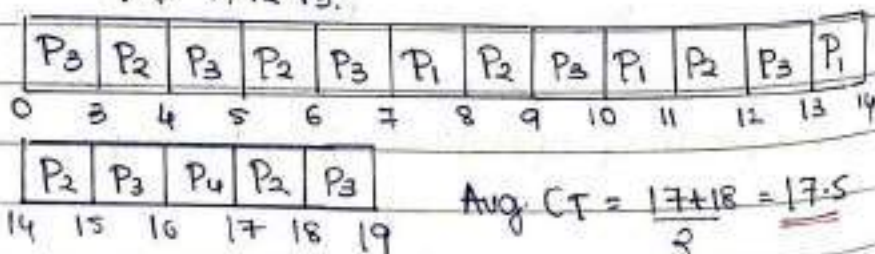
$$\text{Avg T.A.T} = 13$$



Q1. Consider 3 processes A, B, C with compute BT's are 4, 6, 9 units. All the processes arrive at time zero. Consider the longest remaining time first (LRTF). In LRTF ties are broken by giving priority to processes with lowest Pid. Find the avg of completion times of A, B?

RQ: P₁ P₂ P₃

P.no	AT	BT
A	0	4
B	0	6
C	0	9



$$\text{Avg CT} = \frac{17+18}{2} = 17.5$$

Q2. In a system using single processor, a new process arrives at the rate of 10 processes per minute and each such process require 5 seconds of service time. What is % CPU utilization?

Solⁿ :- 60 s $\rightarrow$ 10 processes

$\rightarrow$ (5 sec)

(P₁, P₂ ... P₁₀)

60 sec
cycle.

$$\% \text{ CPU Utilization} = \frac{50}{60} \times 100$$

$$= \underline{\underline{83.33\%}}$$

Q3. Six jobs are waiting to be run. The expected running-times are 9, 7, 5, 2, 1 and x respectively. Where $5 < x < 7$ and avg CT is 13. Find the value of x using SJF algorithm? (Assume all jobs arrive at same time = 0).

6. Priority Based Scheduling

$\rightarrow$ Priority based scheduling works exactly like SJF/SRTF except that it looks at priority instead of burst time.

* Selection criteria: Priority

* Mode of operation: Non preemptive / preemptive

* Tie breaking: Lower process id.

Ex 1.

Prio	Pno	AT	BT
4	1	0	4
5	2	1	3
8	3	2	5

Non
pre-emp

P_1	P_3	P_2	
0	4	9	12

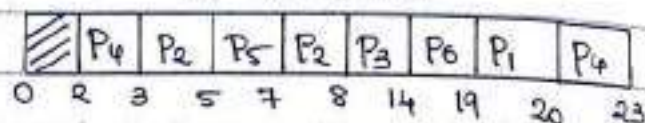
Pre-emp.

P_1	P_2	P_3	P_2	P_1	
0	1	2	7	9	12

Ex 2.

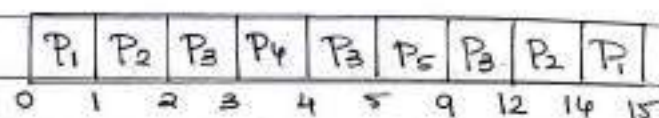
Prio.	P.no	AT	BT
4	1	4	1
6	2	3	3
8	3	8	6
3	4	2	4
7	5	5	2
5	6	3	5

$$L = 23 - 2 = 21.$$



Ex 3.

Prio	P.no	AT	BT
4	1	0	21
5	2	1	3
6	3	2	5
7	4	3	1
8	5	5	4



$$\text{Avg TAT} = \frac{15 + 13 + 10 + 1 + 4}{5} = \frac{43}{5} = 8.6$$

$$\text{Avg WT} =$$

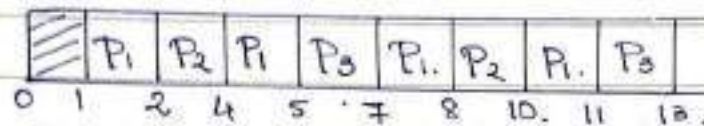
Performance of priority based scheduling:

- * Causes starvation to low priority processes, a solution exists "Dynamic priorities": It increases the priority of the processes at regular intervals of time using an algorithm called "Aging" algorithm.

Q1. Consider a system with pre-emptive priority based scheduling with processes P₁, P₂, P₃ having infinite instances of them. The instances of these processes arrive at regular intervals of 3, 7 and 20 ms respectively. The priority of these process instances is inverse of their periods. Each of the process instance P₁, P₂, P₃ consumes 4, 2 and 4 ms CPU time respectively. The 1st instance of each process is available at 1 ms. What is the completion time of the 1st instance of process P₃?

Prio.	Priod	P _{no}	AT	BT	<Other instances>
1/3	3	1	1	1	<4, 7, 10, 13, 17...>
1/7	7	2	1	2	<8, 15, 22, ...>
1/20	20	3	1	4	<21, 41, 61...>

Rq: $P_1, P_2, P_3, P_1, P_2,$



Completion time of P_3 first instance is 13ms (at the end of 12)

Q2. Consider a System using Preemptive Priority based scheduling with dynamically changing priority. On its arrival a process is assigned a priority of zero and running process priority increases at the rate of ' β ' and priority of the processes in the ready Q increases at the rate of ' α '. By dynamically changing values of α and β one can achieve different scheduling disciplines among the processes. What discipline will be followed for the following conditions

1. $\beta > \alpha > 0$
2. $\alpha < \beta < 0$.

Ans: FCFS for first condition and LIFO for second condition.

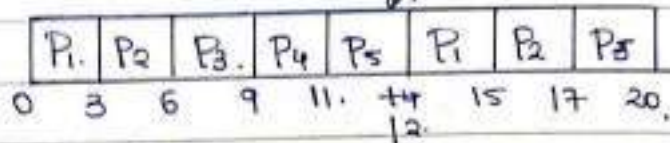
7. Round Robin Scheduling

→ It is used in Pre-emptive multi programmed timeshared Operating systems.

- * Selection Criteria: Arrival time + Time Quantum
- * Mode of operation: Pre-emptive
- * Working principle:
 - each process is allowed a fixed time quantum
 - Pre-emption is based on completion or expiration of time quantum.

Ex 1. P_{no} AT BT TQ=3 RQ: 1, 2, 3, 4, 1, 5, 3, 5.

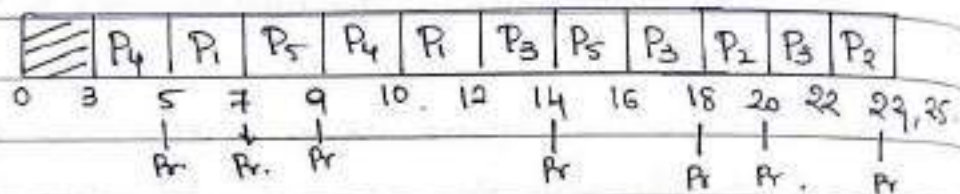
P_{no}	AT	BT
1	0	4
2	1	5
3	2	3
4	3	2
5	5	6



L=20.

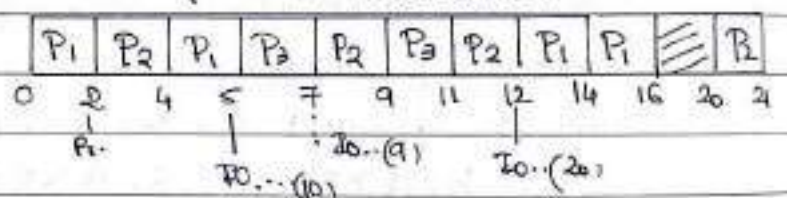
Ex 2. P_{no} AT BT TQ=2 $P_4 P_1 P_5 P_4 P_1 P_3 P_5 P_3 P_2 P_5 P_2$

P_{no}	AT	BT
1	4	4
2	15	5
3	8	6
4	3	3
5	5	4

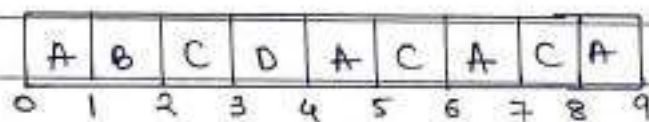
 P_{no} AT BT IOST BT TQ=2

P_{no}	AT	BT	IOST	BT
1	0	3	5	4
2	2	5	8	1
3	3	2	2	2

RQ: P1 P2 P1 P3 P2 P3 P2 P1



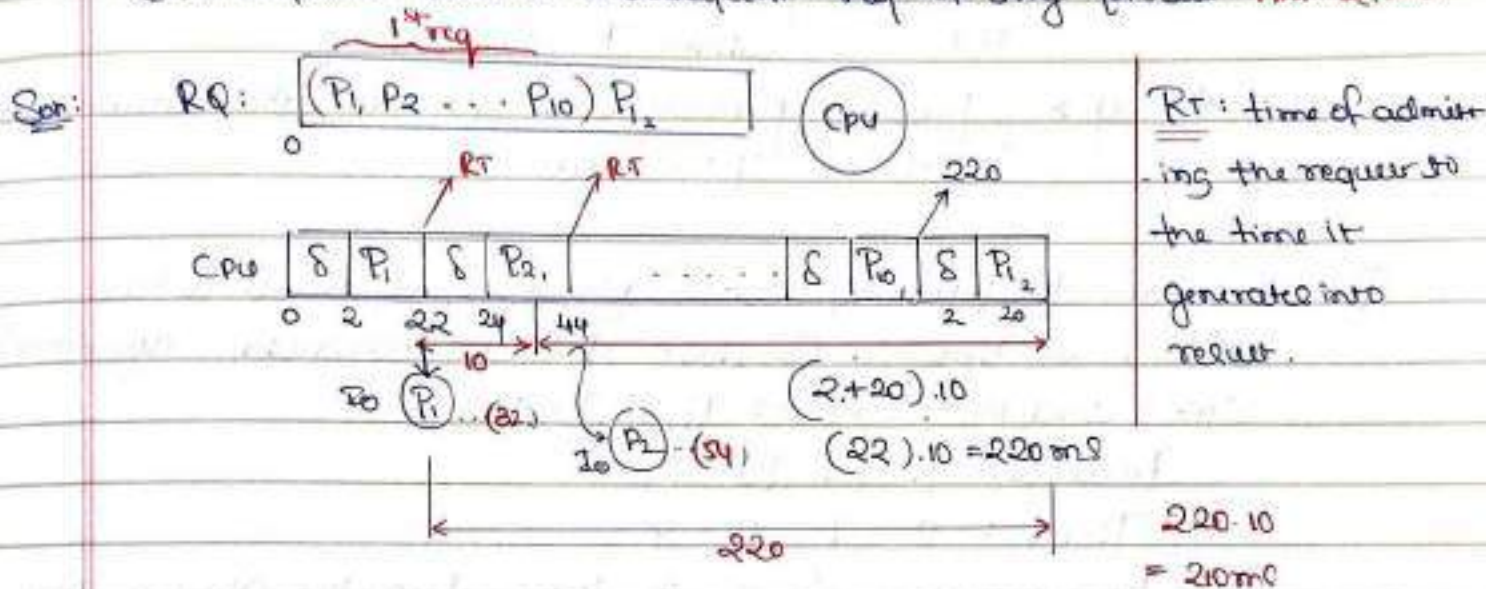
Q1. Consider a set of 4 processes, A, B, C, D arriving in the order at time 0. Their burst time req are: 4, 1, 8, 1 respectively using RR scheduling with TQ=1. The completion time of process A is 9.



Q2. Consider a system using RR Scheduling with 10 processes all arriving at time 0. Each process is associated with 20 identical request. Each process request consumed 20 ms of CPU time after which it spends 10 ms of time on I/O thereafter, initiates subsequent request. Assuming scheduling overhead of 2ms and

TQ of 20ms. Calculate:

- (i) Response time of 1st request of 1st process. - Ans: ~~22~~ 22 ms
- (ii) Response time of 1st request of last process - Ans: 220 ms
- (iii) Response time of Subsequent req. of any process - Ans: 210 ms



Performance of Round Robin

very small:

Efficiency = 0

Time Quantum.

Small

Large

very large: behave like FCFS

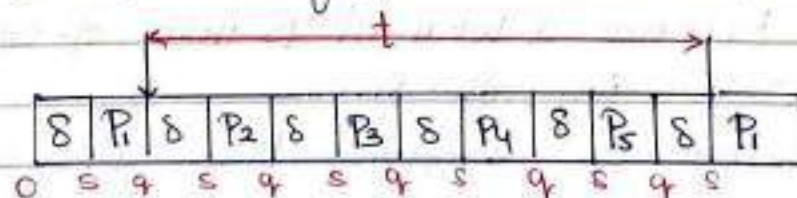
→ More context switching (overhead)

→ Less context switching (less overhead), Poor interactivity, responsiveness.

Makes the system better responsive

Q1. Consider a system with 'n' processes arriving at time 0 with substantially large RT. The CPU scheduling overhead is 's' seconds, Time quantum is 'q' seconds. Using RR scheduling what must be value of time quantum 'q' such that each process is guaranteed to get its turn at CPU exactly after 't' seconds in its subsequent run at CPU.

$$\rightarrow t - n \cdot s = (n+1)q$$



$$\Rightarrow q = \frac{t - ns}{n+1}$$

1) $q = \left(\frac{t - ns}{n-1} \right)$: Process will get into CPU exactly after q 't' seconds.

2) $q < \left(\frac{t - ns}{n-1} \right)$: Process will get into CPU at least once within 't' seconds.

3) $q \geq \left(\frac{t - ns}{n-1} \right)$: Process will get into CPU at least every 't' seconds.

HW

Q2. Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms (SJF) and RR. Order: P_1, P_2, P_3, P_4 .

Processes: P_1, P_2, P_3, P_4

Times: 8 7 2 4

The time quantum for RR is 4 sec, then the absolute value of average turnaround times in SJF and RR is

<u>SJF</u>		<u>RR</u>						
CPU	<table border="1"><tr><td>P_3</td><td>P_4</td><td>P_2</td><td>P_1</td></tr></table>	P_3	P_4	P_2	P_1		<table border="1"><tr><td></td></tr></table>	
P_3	P_4	P_2	P_1					
	<table border="1"><tr><td>0</td><td>2</td><td>6</td><td>13</td><td>21</td></tr></table>	0	2	6	13	21		
0	2	6	13	21				
Avg TAT = $\frac{21+13+2+6}{4}$								
$= \frac{42}{4} = 10.5$								

HW

Q3. Consider processes P_1 and P_2 arriving in RR at time 0 with:
 i) P_1 needs a total of 12 units of CPU time and 20 units of I/O time. After every 3 units of CPU time P_1 spends some on I/O.
 ii) P_2 needs a total of 15 units of CPU time and no I/O. P_2 arrives just after P_1 .
 Compare the completion times of P_1 and P_2 using SRTF & RR.

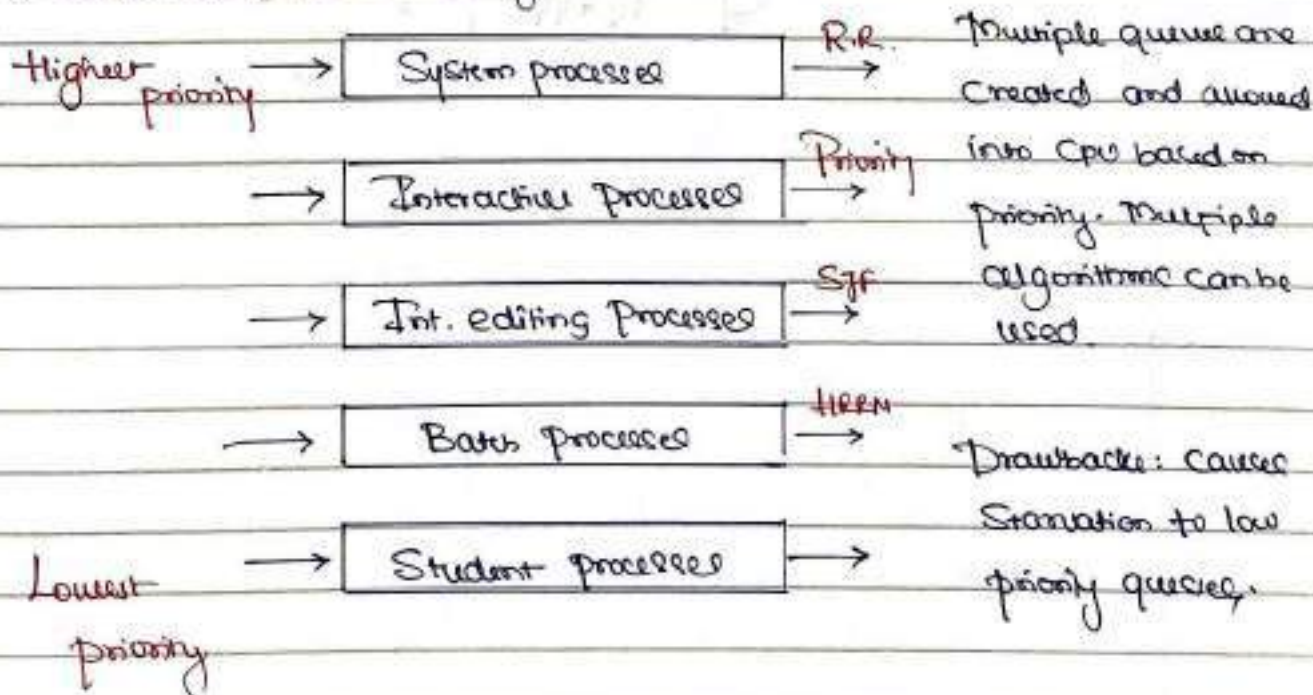
HW

Q4. Consider 4 processes P, Q, R, S scheduled on a CPU as per RR algorithm with $T_q = 4$ units. Processes arrive in order P, Q, R, S all at time $t = 0$. There is exactly one context switch from S to Q, exactly one context switch from R to Q and exactly two context switches from Q to R. There is no context switch from Q to P. Switching to a ready process after termination of another process is also considered a context switch.

Q5. Which of the following statements is/are correct in context of CPU Scheduling?

- A. The goal is to only maximize CPU utilization and minimize throughput
- B. Turnaround time includes waiting time
- C. Implementing pre-emptive scheduling needs hardware support
- D. RR policy can be used even when the CPU time required by each of the processes is not known a priori

8. Multi Level Queue Scheduling:

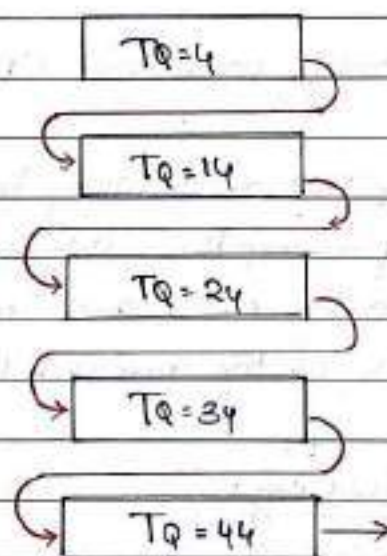


9. Multilevel Feedback Queue Scheduling:

- * Another way to put a reference on short-lived processes, penalize processes that have been running longer.
- * Pre-emptive.

Q6. Consider a system which has CPU bound process, which require the burst time of 80 seconds, the multilevel feedback queue scheduling algorithm is used and the queue time quantum is '4' sec and in each level it is incremented by '10' seconds. Then how many times the process will be interrupted, and on which queue the process will terminate the execution?

Ans: 4, 5.



B.T = 80 sec.

$$4 + 14 + 24 + 34 = \underline{80}$$

Synchronization

★ IPC: Inter process communication is a mechanism that allows processes to communicate with each other and synchronize their actions.

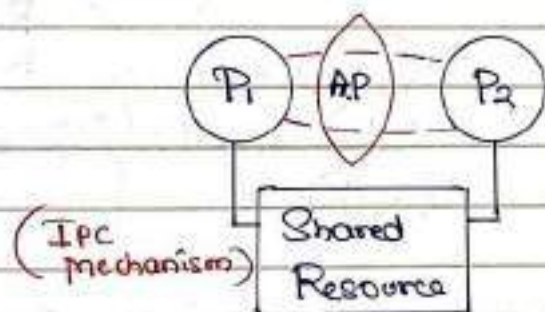
Every communication should be synchronized/co-ordinated

→ Lack of synchronization in IPC environment leads to:

1. Inconsistency (Incorrectness)
2. Loss of data
3. Deadlock (lockup)

Synchronization: It is agreed upon protocol in an IPC environment, to make sure that there is no inconsistency, data loss or deadlock.

IPC Environment:



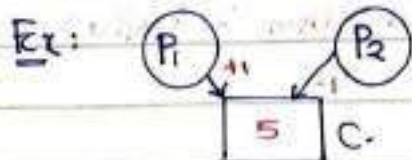
* Synchronization involves the orderly sharing of system resources by processes.

* We can think of this intersection as a system resource that is shared by two resources.

Types of Synchronization

Competitive

Two or more processes are said to be in competitive sync. if they compete for accessibility of a shared resource.

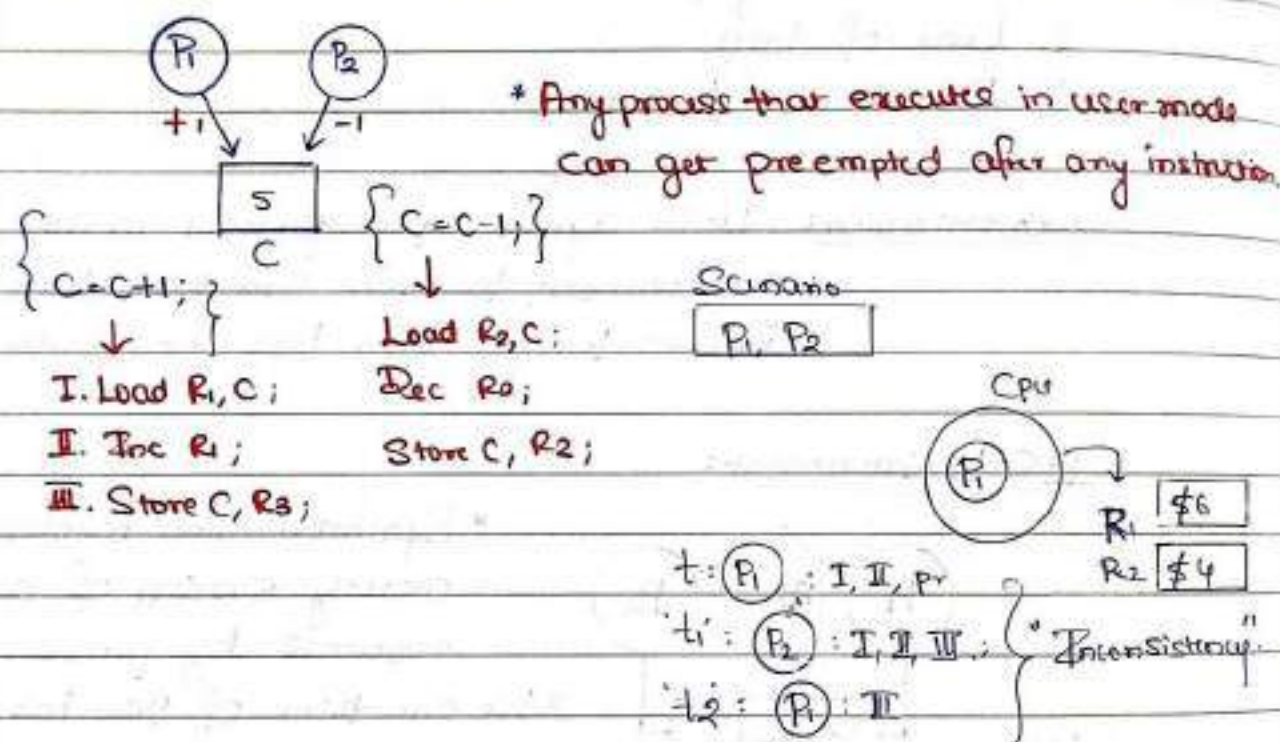


Co-operative

Two or more processes are said to be in co-operative synchronization if they get affected by each other, i.e. execution of one process affects the other process.

Ex: Producer-consumer problem.

- Note :
- * Lack of Synchronization among competing processes may lead to either inconsistency, data loss.
 - * Lack of Synchronization among co-operating processes may lead to deadlock.
 - * An application of an IPC environment may involve either Competition or Cooperation or both type of Synchronization.



- Note :
- * Sometimes we get correct results and other times it is possible to get incorrect results.
 - * As an end user we want a solution, that always gives correct result whether preemption takes place or not.

Implementation of producer consumer problem

```
#define N 100
```

```
int buffer[N];
```

```
int count = 0; <No. of data items in buffer>
```

```

Void Producer (void)
{
  init itemp; in = 0;
  while (1)
  {
    NCS a) itemp = Produce-item();
    CS b) while (count == N); Busy waiting
    CS c) Buffer[in] = itemp;
    CS d) in = (in + 1) % N;
    CS e) Count = Count + 1;
  }
}

```

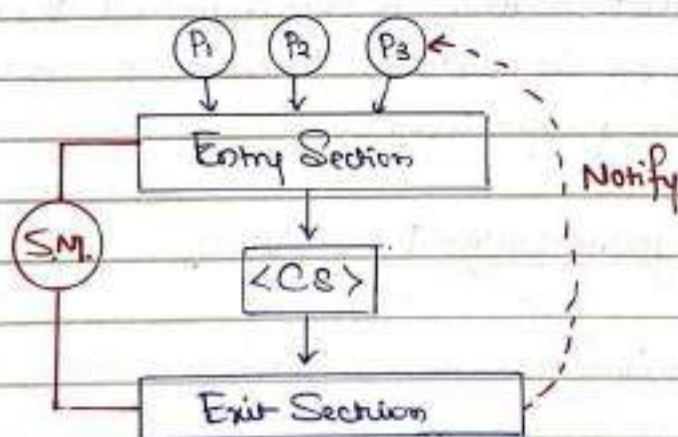
```

Void Consumer
{
  int itemC, out = 0;
  while (1)
  {
    a) while (count == 0); Busy waiting
    b) itemC = Buffer[out];
    c) out = (out + 1) % N;
    d) Count = Count - 1;
    e) Process-itemC;
  }
}

```

Necessary Conditions for Synchronization Problems

1. **Critical Section:** It is that part of the program where shared resources are accessed
2. **Race condition:** Situation where in processes are trying to access critical section and final result depends on the order at which they finish their update.
3. **Preemption:**



```

Do {
  Entry Section
  <Critical Section>
  Exit Section
  <Remainder Sect>
}
while (true);

```

Note: Process running in user mode can get preempted any time and any number of times (after completing any instance).

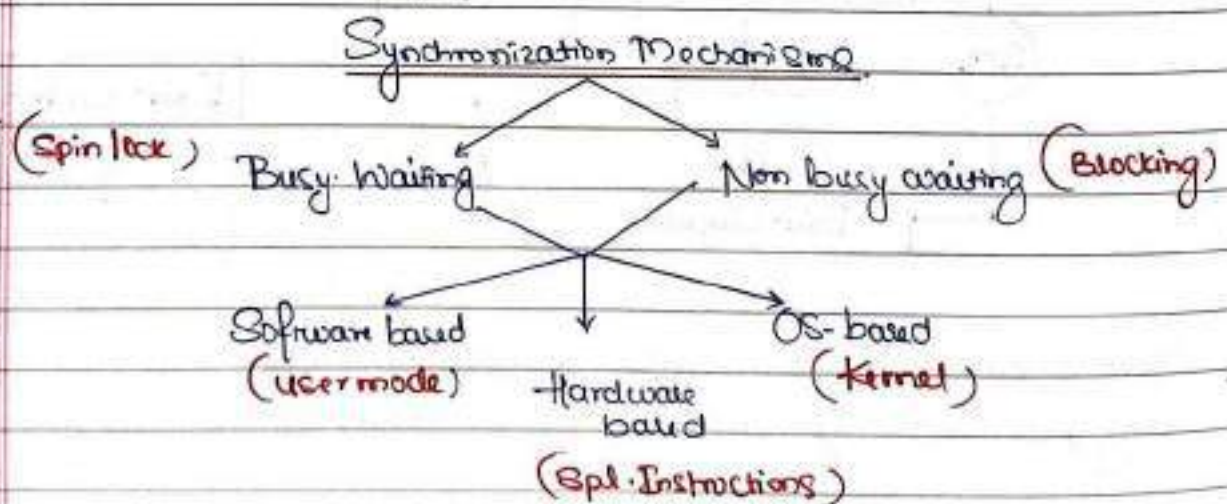
Requirements of C.S. Problems:

1. Mutual Exclusion (M/E): If process P_i is executing in its Critical Section then no other processes can be executing in their Critical Sections.
2. Progress: If no process is executing in its critical section and some processes wish to enter their Critical Sections, then only those processes that are not executing in their remainder Section can participate in deciding which will enter its Critical Section next and this selection cannot be postponed indefinitely.
3. Bounded waiting: There exist a bound, or limit, on the number of times that other processes are allowed to enter their Critical Sections after a process has made a request to enter its Critical section and before that request is granted.

Note: * If mutual exclusion is not guaranteed then inconsistency + data loss may happen.

* If bounded waiting is not guaranteed then starvation.

* If progress is violated then it is an unfair solution (Indefinite postponement).

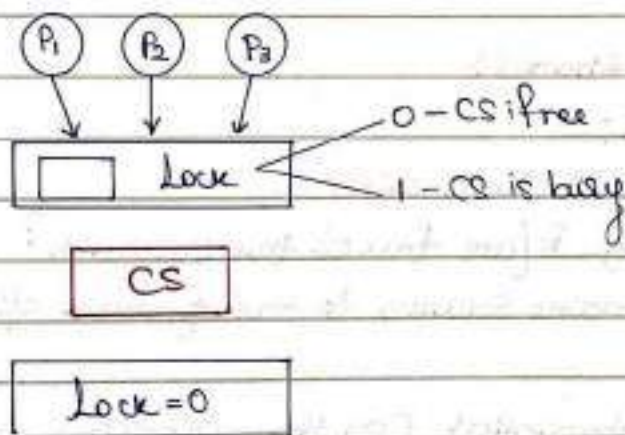


Assumptions:

- * Process enters $\langle CS \rangle$ and come out of it in finite amount of time.
- * When a process is in entry section then it means, it is interested in $\langle CS \rangle$.
- * A process is said to have left CS only when it has executed its Critical Section.
- * A process can get pre-empted from CPU while executing entry + $\langle CS \rangle$ + Exit Section.

1. Lock Variable:

- Busy waiting Solution
- Software Solution implementation at user mode
- Multi process Solution.

Implementation: High L.

```

int lock = 0;
void process (int i)
{
    while (1)
    {
        a) Non-CS
        b) while (lock != 0); } Entry Section
           lock = 1;
        c) CS
        d) lock = 0; } Exit Section
    }

```

Implementation: Low Level.

```

integer lock = 0;
Process (int i):
a) Non-CS
b) Load Ri, Lock:
c) Compare Ri, #0; } Entry Section
d) JNZ step b;
e) Store lock, #1;
f) <CS>.
g) Store lock, #0 } Exit Section

```

* Lock variable does not always guarantee mutual exclusion along with bounded wait.

- * Lock variable guarantees progress always
- * Lock variable fails to satisfy bounded wait.

- * Lock variable fails to satisfy bounded wait.

- * Lock variable is a busy waiting solution leading to wastage of CPU time / cycles (in the loop).

<code unrelated to device use>

```
until busy = false;
```

```
Busy = true;
```

<code to access shared>

Busy = false;

<Code Unrelated to device use>

Which of the following is/are true of this approach?

I. It provides a reasonable solution to the problem of guaranteeing m/e

It may consume substantial CPU time accessing the Busy variable.

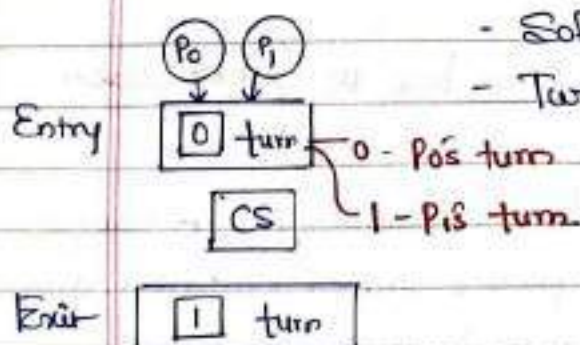
III. It will fail to guarantee mutual exclusion.

2. Strict Alteration:

- Busy waiting

- Software Solution implementable at user mode

- Two process solution $\langle p_i | p_j \rangle$



"Strictly on alternate basis processes take turn to enter CS."

Implementation: High Level.

```
int turn = rand(0,1);
```

```
void process (0)
{
    while (1)
    {
        a) Non-CS();
        b) while (turn != 0)
        c) CS
        d) turn = 1; }
}
```

```
void process (1)
```

```
{
    while (1)
    {
        a) Non-CS();
        b) while (turn != 1);
        c) CS
        d) turn = 0; }
}
```

- * Strict alternation always guarantee mutual exclusion.
- * Strict alternation does not guarantee progress.
- * Strict alternation always guarantee bounded waiting.
- * Like lock variable this is a busy waiting solution which leads to wastage of Cpu time/cycles.

Generalised Implementation of Strict Alternation:

```
int turn = rand(i,j);
void process (int i)
{
    int j = Not(i);
    while (1)
    {
        a) Non-CS();
        b) while (turn != i)
        c) CS
        d) turn = j; }
}
```

Q2 Consider the following two-process synchronization solution.

Process 0

Entry: loop while ($turn \neq 1$);
(critical section)Exit: $turn = 1$;

Process 1

Entry: loop while ($turn \neq 0$);
(critical section)Exit: $turn = 0$;

The shared variable $turn$ is initialized to zero. Which of the following is true?

Ans: This solution violates progress requirement.

Q3. Code Snipped:

```
int turn = 0;
void Process(0)
{
    while (1)
    {
        a) Non-CS();
        b) while (turn == 1);
        c) CS
        d) turn = 1;
    }
}
```

```
void process(1)
{
    while (1)
    {
        a) Non-CS();
        b) while (turn == 1);
        c) CS
        d) turn = 1;
    }
}
```

Does the above code guarantee mutual exclusion?

Ans: No.

* If $turn = 1$; then it will result in deadlock as no process can enter in the Critical Section.

Q4. `int turn = rand(1, j);``void Process (int i)``{ int j = NOT (i);``while (1)``{ a) Non-CS();``b) while (turn != i);``turn = j;``c) CS``d) turn = j; }``}`

i) Does this guarantee M/E?

M/E is not guaranteed.

ii) Progress?

iii) Bounded Wait?

3. Peterson's Solution:

- Busy-waiting

- Software solution implemented at user mode

- Two process solution

- Combination of Lock & Strict alternation.

`#define N 2``#define True 1``#define False 0``int flag[N] = {False};``int turn;``void Process (int i)``{ int j = NOT(i);``while(1)``{ a) Non-CS();``b) flag[i] = True;``c) turn = i;``d) while (flag[j] = True and turn == j);``e) CS;``f) flag = False; }``[i]`Note: Peterson's Solution

guarantees M/E always.

* It guarantees progress

* It guarantees bounded waiting.

Q5. void Dekkers-Algorithm(int i)

```

{
    int j = !i;
    while (1)
    {
        a) Non-CS();
        b) flag[i] = True;
        c) while (flag[j] == True):
            {
                if (turn == j)
                {
                    flag[i] = False;
                    while (turn == j);
                    flag[i] = True;
                }
            }
        d) CS
        e) flag[i] = False;
            turn = j;
    }
}

```

* Deckers algorithm is a variation of Peterson's algorithm and is a correct solution.

Analyse the solution.

Q6. Process P₁ and P₂ hold/use Critical flag in the following routine to achieve M/E. Assume that Critical-flag is initialised to false in main program.

```

get-exclusive-access()
{
    if (Critical-flag == False)
    {
        Critical-flag = True;
        Critical-region();
        Critical-flag = False;
    }
}

```

Consider the following statements

I. It is possible for both P₁ and P₂ to access Critical region Concurrently.

II. This may lead to a deadlock.

Ans. I. is true and II is false.

Q7. Two processes P_1 and P_2 need to access a critical section of the code. Consider the following synchronization construct used by the process.

$/P_1/$

```
while (true)
{
  wants1 = true;
  while (wants2 == true);
  /CS/
  wants2 = false;
}
```

$/\text{Remainder Section}/$

$/P_2/$

```
while (true)
{
  wants2 = true;
  while (wants1 == true);
  /CS/
  wants1 = false;
}
```

$/\text{Remainder Section}/$

Here $wants1$ and $wants2$ are shared variables which are initialised to false. Which of the statements are true.

- I. It does not ensure M/E.
- II. It does not ensure bounded waiting.
- III. It requires that processes enter critical section in strict alternation.
- IV. It does not prevent deadlock but ensures M/E.

Q8. Two processes X and Y need to access a critical section. Consider the following synchronization construct used by both.

Process X

```
while (true)
{
  var P = true;
  while (var Q == true)
  {
    /CS/
    var P = false;
  }
}
```

Process Y

```
while (true)
{
  var Q = true;
  while (var P == true)
  {
    /CS/
    var Q = false;
  }
}
```

Here $varP$ and $varQ$ are shared variables and both initialised to false. Which of statements are true?

- I. The proposed solution prevents deadlock but fails to guarantee M/E.
- II. It guarantees M/E but fails to prevent deadlock.
- III. It guarantees M/E and prevents deadlock.
- IV. It fails to prevent deadlock and M/E.

Synchronization Hardware:

Each processor supports some support instructions (H/W) that are atomic <Execute non-preemptively>

a) TSL

b) SWAP.

- Busy waiting
- Can be used at user mode as spl. instructions
- Hardware category
- Multi-process solution
- Lock based solution.

a) TSL : Test and Set Lock

TSL (&lock); < Process executing TSL, returns the current value of lock and sets the value of lock always to True >

```

Bool TSL (Bool *target)
{
    Bool rv;
    rv = *target;
    *target = True;
    return (rv);
}
    
```

Completely runs on Kernel mode (Atomic)

Solving CS Problem through TSL

```

Bool lock = false;
void Process (int i)
{
    while (1)
    {
        a) NON-CS();
        b) while (TSL (&lock) == True);
        c) <CS>
        d) lock = F; }
    }

```

* This solution guarantees mutual exclusion.

* Does not guarantee bounded wait.

b) SWAP Instruction <ATOMIC>

```

SWAP (Bool *a, Bool *b)
{
    Bool t;
    t = *a;
    *a = *b;
    *b = t; }

```

- * Guarantees mutual exclusion
- * Guarantees progress
- * Guarantees bounded waiting.

User process:

```

Bool lock = false;
void Process (int i)
{
    Bool key = True;
    while (1)
    {
        a) NON-CS();
        b) while (key == True)
            SWAP (&lock, &key);
        c) CS
        d) lock = False; }
    }

```

Q9. The enter-CS(); and leave-CS() functions to implement Critical Section of a process and are realised using test-and-set instructions as follows:

```

void enter-CS(x)
{
    while (test-and-set(x));
}

void leave-CS(x)
{
    x = 0; }

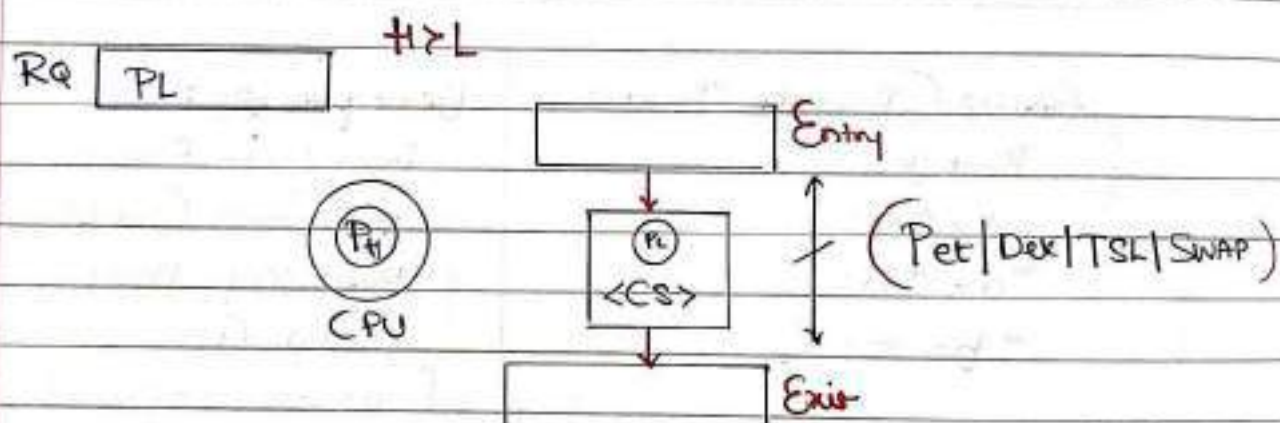
```

In the above solution, X is a memory location associated with the CS and is initialised to 0. Now consider:

- I. The above solution to CS problem is deadlock free.
- II. Solution is starvation free.
- III. The process enters CS in FIFO order.
- IV. More than one process can enter CS at same time.

Priority Inversion Problem

All correct busy waiting software solutions suffer from priority inversion problem. This problem happens on pre-emptive priority based scheduling.



- * If P_n is not interested in CS then there is no problem
- * If P_n is also interested in CS, then it results in "Deadlock"

Imp

Q10. Fetch-And-Add (X, i) is a atomic Read-modify-write instruction that reads the value of memory location X , increments it by the value i and returns the old value of X . It is used in the pseudocode shown below to implement a busy-wait lock. L is an unsigned integer shared variable initialised to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

```

Acquire Lock (L)
{ while (Fetch-And-Add(L, 1)
  L = 1; }
Release Lock (L)
{ L = 0; }

```

This implementation:

- I. fails as L can overflow
- ✓ II. fails as L can take on a non-zero value when lock is actually available
- III. works correctly but many times some process.
- IV. works correctly without starvation.

Analysis:

RQ. P_1, P_2, P_3

$\phi \times 2$ Lock

int lock = 0

P_1

CS

Entry while (Fetch-And-Add (&lock, 1))
L = 1;
<CS>

Exit L = 0;

* If msg asked, I. also correct in some cases

I:

t1: $(P_1) : F.A.A : \rightarrow 0$

t2: $(P_2) : F.A.A : \rightarrow 01$

II:

t1: $(P_1) : F.A.A : 0 : <CS> \dots P_1$

t2: $(P_2) : F.A.A : 1. P_2 <L=1>$

t3: $(P_1) : <CS>$

t4: $(P_2) : L=1, F.A.A=1$

Blocking Mechanisms / Non busy waiting

- to avoid wastage of CPU time in the form of loops.
- all blocking mechanisms are implemented using "if-else-then".

There are three mechanisms:

1. Sleep-wakeup
2. Semaphores
3. Monitors.

1. Sleep() and Wakeup() :

- Blocking Construct
- Multi process solution
- Are OS primitives
- When process executes Sleep() - gets blocked till some other process wakes it up (wakeup())

Producer - Consumer using Sleep() and Wakeup()

```
#define N 100
int buffer[N];
int count = 0;

void Producer(void)
{
    int item, in = 0;
    while (1)
    {
        a> item = produce-item();
        b> if (count == N) Sleep();
        c> Buffer[in] = item;
        d> in = (in + 1) % N;
        e> count = count + 1;
        f> if (count == 1) wakeup(Consumer);
    }
}
```

```
void Consumer(void)
{
    int itemc, out = 0;
    while (1)
    {
        a> if (count == 0) Sleep();
        b> itemc = Buffer[out];
        c> out = (out + 1) % N;
        d> count = count - 1;
        e> if (count == (N - 1)) wakeup(Producer);
        f> Process-item(itemc);
    }
}
```

* When producer and consumer both try to update the value of count, leads to "inconsistency"

Two problems with this implementation

- Inconsistency
- Deadlock.

Semaphores

- * Blocking construct
- * OS based <is an OS resource (S/W)>.
- * General purpose utility
 - <CS>
 - Requirements of classical IPC problems
 - Concurrency mechanisms?
- * It was proposed by E. Dijkstra
- * Semaphore is implemented as an abstract data type.

Definition: Semaphore is a variable
 <ADT: SEM> that takes only
integer values.

Types

1. Binary (mutex)
 <0, 1>
2. Counting (General)
 < $-\infty$ to ∞ >

Operations

down()
 wait()
 P()

up()
 Signal()
 V()

Counting Semaphore < $-\infty$ to ∞ >

Type def

```

struct
{
  int values;
  QueueType L;
} CSME;
```

List of PCBs of those

processes that get blocked while
 performing down operation unsuccessfully.

Um:

```

CSEM S;
S.value = 1;
(S = 1);
Down(S);
<S>
```

Next
 Statement.

Km:

```

DOWN(CSEM S)
{
  1. S.value = value - 1;
  2. if (S.value < 0) // unsuccess
  {
    put this process Pi
    in the S.L(Q) and block
    the process (Sleep); }
  else
    return; } // success

```

eg: CSEM S;

S = 5;

How many Down operations
can be carried out by
process successfully?

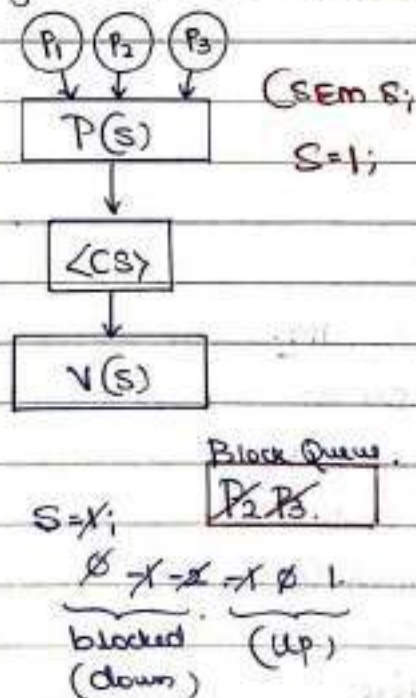
S = ~~4~~ ~~3~~ ~~2~~ ~~1~~ ~~0~~ -1.

| | | | | |

Successful
down op's.

after this process
get blocked

- * Positive value of semaphore indicates that those many down operations can be carried out successfully.
- * After carrying out a series of down operations, if the value of semaphore becomes negative, then the magnitude of the negative value indicates the number of blocked processes.



UP(CSEM S)

```

{
  1. S.value = S.value + 1;
  2. if (S.value < 0)
  {
    Select a process from
    S.L(Q) and wake up(); }
  else
    return; }

```

- * If the value of S is less than or equal to '0' then there are blocked processes in queue.

Q1. CSEM $S = 8$;Operations : 10P, 1V, 15P, 2V, 6P, 3V What is current value?
 -2 -1 -16 -14 -20 -17.Current value of $S = 17$.Q2. CSEM $S = ?$ Operations : 12P, 4V, 6P, 3V, 8P, 1V. Final value is -6.
What should be initial value of S ?Initial value of S should be 12.Q3. Consider a non-negative Counting Semaphore S . The operation $P(S)$ decrements S , and $V(S)$ increments S . During an execution, 20 $P(S)$ operations and 12 $V(S)$ operations are issued in some order. The largest initial value of S for which atleast one $P(S)$ operation will remain blocked is 7.Binary Semaphore $\langle 0, 1 \rangle$

<u>Um:</u>		
<pre> typedef struct { enum value {0, 1}; QueueType L; } BSEM; </pre>	<pre> BSEM S; S = 1; DOWN(S); <Not Stmt> </pre>	<pre> DOWN(BSEM S) { if (S.value == 1) { S.value = 0; // Success return; } else { put-this process (PCB) in S.L(Q) and block it (sleep()); } } </pre>

List of PCBs that
goes blocked while performing
down operation unsuccessfully.

* In binary semaphore we don't go to know the no. of blocked processes because no negative numbers after down operations.

```

UP(BSEM S)
{
    if (SL(Q) is NotEmpty)
    {
        Select a process from
        S.L(Q) and wakeup(); }
    else
        S.value = 1; }

```

Cases of Binary Semaphores

1. BSEM S;
S=1;
P(S);
S=0
Status = Success.

2. BSEM S;
S=0;
P(S);
S=0
Status = Unsuccessful.

3. BSEM S;
S=0;
V(S);
S → 0 : if 'Q' not empty
1 : if 'Q' empty.
Status = Success.

4. BSEM S;
S=1;
V(S);
S=1
Status = Success.

5. BSEM S;
S=0;
V(S); // First operation.
S=1.
Status = Successful.

Note:

As long as there is a process in 'Cs' & processes in block 'Q' then value of BSEM never be '0'.

Q4 BSEM S=1;
Operations: 10P, 2V, 18P, 3V, 4P, 5V;
(i) Final value of S?
(ii) Size of 'Q' [L]?

S = X 0

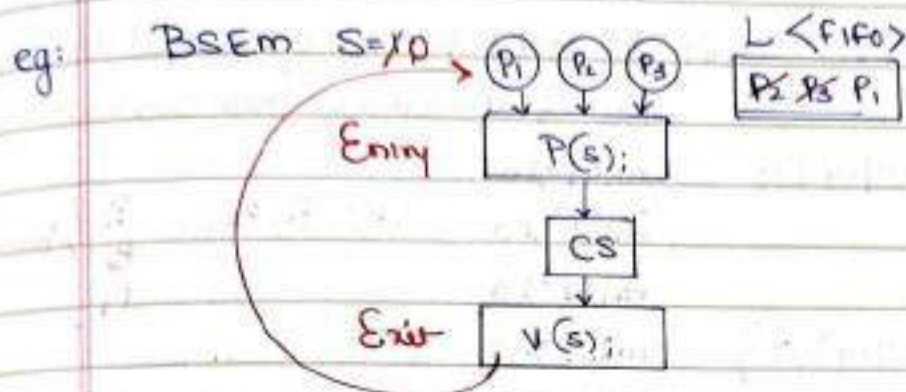
10P, 2V, 18V, 3V, 4P, 5V;

Final value of S = 0

L → 9 7 25 22 26 21

and 21 blocked processes.

Both binary and counting semaphore can be used to solve the problem of CS.



- 1) Mutual Exclusion ✓
- 2) Progress ✓
- 3) Bounded wait ✓

Q1. BSEM $S=1, T=1$;

```

Pr(i)
{
  while (1)
  {
    P(S);
    P(T);
    <CS>
    V(T);
    V(S);
  }
}
  
```

To prevent deadlock

```

Pr(j)
{
  while (1)
  {
    P(T);
    P(S);
    <CS>
    V(S);
    V(T);
  }
}
  
```

Does it guarantee M/E? and deadlock?

Yes it guarantees ME but it is not deadlock free.

To prevent it from deadlock Swap down operations in one process. Both processes down opns. should be same.

Q2. BSEM $S=1, T=0$

```

Pr(i)
{
  while (1)
  {
    P(T);
    Print('1');
    Print('1');
    V(S);
  }
}
  
```

```

Pr(j)
{
  while (1)
  {
    P(S);
    Print('0');
    Print('0');
    V(T);
  }
}
  
```

What will be output?

00110011....

Does it guarantee

1. M/E ✓
2. Progress ✗
3. Bounded wait ✓

This is a implementation of "strict alternation" using semaphores

Q3. BSEM $m[0..4] = \{1\}$; $Pr(i) \ i=0,4$

```

{
  while(1)
  {
    P(m[i]);
    P(m[(i+1)%4]);
    <CS>
    V(m[i]);
    V(m[(i+1)%4]);
  }
}

```

(i) Does it guarantee M/E?

No

(ii) If M/E is not guaranteed then max no. of processes that can be in CS?

 $m[0] = x0$ $m[1] = x0$ $m[2] = x0$ $m[3] = x0$ $m[4] = x0$ CS: P_0, P_2
 P_1
 P_3
 P_4 } blocked
 $\therefore$ 2 processes in CS

* Deadlock can occur.

Q4. BSEM $S=1, T=0, Z=0$

```

Pr(i)
{
  while(1)
  {
    P(S);
    Print(*);
    V(T);
    V(Z);
  }
}

Pr(j)
{
  P(T);
  V(S);
}

Pr(k)
{
  P(Z);
  V(S);
}

```

What is min and max no of * that get printed?

Case 1: Pri, Pij, Pjk, Pri 2 blockedI: $\langle ** \rangle$: Minimum $\rightarrow 2$ Case 1: Pri, Pij, Pri, Pjk, Pri II: $\langle * * * \rangle$: Maximum $\rightarrow 3$ Variation: BSEM $S=1, T=0, Z=0$;

```

Pri
{
  while(1)
  {
    P(S);
    Print('*');
    V(T);
  }
}

Prj
{
  P(T);
  V(Z);
}

Prk
{
  P(Z);
  V(S);
}

```

In this variation the min and max no of stars to be printed = 2;

Q5. Consider the following processes T_1, T_2, T_3 executing on a single processor, synchronized using three binary Semaphore Variables S_1, S_2 and S_3 , Operated upon using Standard $wait()$ and $Signal()$. The threads can be context switched in any order and at any time. Which

which Initialization of Semaphores would print sequence BCABCA...?

T_1	T_2
while (true)	while (true)
{ wait(S_3);	{ wait(S_1);
Print('C');	Print('B');
Signal(S_2); }	Signal(S_3); }

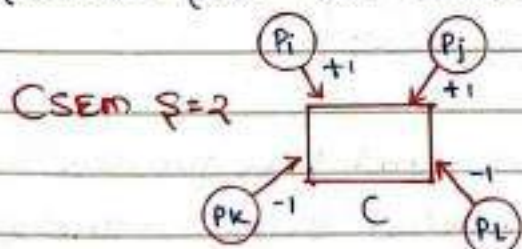
T_3
while (true)
{ wait(S_2);
Print('A');
Signal(S_1); }

Ans: $S_1=1$
 $S_2=0$
 $S_3=0$

$T_2 \rightarrow T_1 \rightarrow T_3$

HW

Q6. Consider a counting Semaphore initialized to 2, there are 4 Concurrent processes P_i, P_j, P_k and P_l , The processes P_i and P_j desire to increment the current value of variable C by 1 whereas P_k and P_l desire to decrement the value of C by 1. All processes perform their update on C under Semaphore Control. What can be the min and max value of C after all processes finish their update?



Assuming initial value of $C=0$

Process (x)

{ P(S);	$x=i, j, k, l$
$C = C \pm 1$	$\begin{matrix} +1 \\ -1 \end{matrix}$
V(S); }	

Q7. Consider three processes using four binary Semaphores a, b, c, d in the order shown below.

Which sequence is a deadlock free sequence?

X: P(b); P(a); P(c); a=b=c=d=1. } Initialized.

Y: P(b); P(c); P(d);

Z: P(a); P(c); P(d);

Q8. Each of a set of n processes execute the following code using two semaphores a and b initialized to 1 and 0 resp. Assume that count is a shared variable initialized to 0 and not used in code section P.

<Code Section P>

wait(a);

count = count + 1;

if (count == n) signal(b);

signal(a); wait(b); signal(b);

<Code Section Q>

What does code achieve?

It ensures that no process execute code section Q

before every process has finished code section P.

HW

Q9. Consider two functions Incr() and Decr() GATE 2023

Incr()	Decr()
$\left\{ \begin{array}{l} \text{wait}(s); \\ x = x + 1; \\ \text{signal}(s); \end{array} \right\}_5$	$\left\{ \begin{array}{l} \text{wait}(s); \\ x = x - 1; \\ \text{signal}(s); \end{array} \right\}_3$

Five processes invoke Incr() and three processes invoke Decr()

x is a shared variable initialized to 10. $(10+5-3)=12$

I₁: S value is 1 (Bin. Sem.) max/min = 12

I₂: S value is 2 (Counting Sem.)

Let v₁ and v₂ be min possible value of implementation of I₁ and I₂ then choose the value of x for v₁ and v₂.

- Q1. Which of the following may result in a process getting blocked?
- a) Down
 - b) Up
 - c) System-call
 - d) Scheduler dispatch.

Classical IPC Problems

1. Producer Consumer Problem:

```
#define N 100
int Buffer[N];
CSEM Empty = N;
<No of empty slots>
CSEM Full = 0;
<No. of full slots (data-items)>
BSEM mutex = 1;
<used b/w P and C to
ensure M/E on buffer>
```

void Producer (void)

```
{
  int itemP, in = 0;
  while (1)
  {
    a) itemP = produce-item();
    b) Down (Empty);
    c) Down (mutex);
    d) Buffer[in] = itemP;
    e) in = (in + 1) % N;
    f) Up (mutex);
    g) Up (Full);
  }
```

void Consumer (void)

```
{
  int itemC, out = 0;
  while (1)
  {
    a) Down (Full);
    b) Down (mutex);
    c) itemC = Buffer[out];
    d) out = (out + 1) % N;
    e) Up (mutex);
    f) Up (Empty);
    g) Process-item (itemC);
  }
```

Q1. Consider the following solution to the P-C sync. problem. Shared buffer size is N . Three semaphores empty, full and mutex are declared with respective initial values of 0, N and 1. Semaphore empty denotes the no. of available slots in the buffer, for Consumer to read from. Semaphore full denotes no. of available slots in the buffer, for producer to write to the placeholder. Variable denoted by P, Q, R, and S in code below can be assigned either empty or full.

Producer:

```
do {
    wait(P);
    wait(mutex);
    // add item - buffer
    signal(mutex);
    signal(Q); }
while (1);
```

Consumer:

```
do {
    wait(R);
    wait(mutex);
    // consume item - buffer
    signal(mutex);
    signal(S); }
while (1);
```

Which of the assignments of P, Q, R, and S yield correct solution?

Ans P: full, Q: empty, R: empty, S: full

Q2. Consider the procedure below for P-C problems use semaphores:

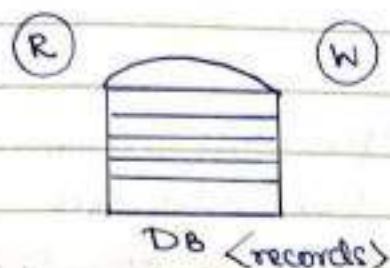
Semaphore $n=0$;

Semaphore $S=1$;

```
void Producer ()
{
    while (true)
    {
        produce ();
        SemWait (S);
        add To Buffer ();
        SemSignal (S);
        SemSignal (n); }
}
```

```
void Consumer ()
{
    while (true)
    {
        SemWait (S);
        SemWait (n);
        remove From Buffer ();
        SemSignal (S);
        Consume (); }
}
```

Ans. Deadlock occurs when/if the Consumer succeeds in acquiring semaphores when buffer is empty.

2. Reader Writer Problem.

BSEM mutex = 1;

```

{
  P(mutex)
  <DB>
  V(mutex);
}

```

First Reader Writer Problem.

t1: r1 and w1 : Anyone

t2: r1; ✓ w1 (wait)

t3: r2; ✓

t4: r3; ✓

r1	r2
r3	

t1: r1 and w1 : Anyone

t2: w1; ✓ r1 (wait)

t3: w2; ✗ (wait)

w1

Problem: Starvation to writers.Implementation of First R-W using Semaphore

int rc = 0;

BSEM mutex = 1;

<used by R's to update>

BSEM db = 1;

<used by R's and W's to
access <DB> as CS>void writer (void)

```

{
  while (1)
  {
    a). DOWN(db);
    b). <DB-write>
    c). UP(db);
  }
}

```

void Reader (void)

```

{
  while (1)
  {
    a). DOWN(mutex);
    b). rc = rc + 1;
    c). if (rc == 1) DOWN(db);
    d). UP(mutex);
    e). <DB-Read>
    f). DOWN(mutex);
    g). rc = rc - 1;
    h). if (rc == 0) UP(db);
    i). UP(mutex);
  }
}

```

Q3. Synchronization in the classical readers and writers problem can be achieved through use of semaphores. In the following incomplete code for readers-writers problem, two binary semaphores

mutex and wrt are used to obtain sync.

wait(wrt)

// waiting is performed

Signal(wrt)

wait(mutex)

readCount = readCount + 1

if readCount == 1, then "S1"

"S2"

// reading is performed

"S3"

readCount = readCount - 1

if readCount == 0 then "S4"

Signal(mutex)

The value of S1, S2, S3, S4 in that order are:

Ans: wait(wrt), Signal(mutex),
wait(mutex), Signal(wrt).

3. Dining Philosophers Problem

'N' - Philosophers ($N \geq 2$)

#define N 5

void Philosopher(int i)

```
{ while (1)
{ a) think(i);
  b) take-fork(i);
  c) take-fork((i+1)%N);
  d) eat(i);
  e) put-fork(i);
  f) put-fork((i+1)%N); }
}
```

If all become hungry
causes "Deadlock".

Without deadlock; for $N=5$ what is the max no. of philosophers that can be eating?

P0: f0, f1 ✓

P1: x

Ans: 2

P2: f2, f3 ✓

P3: f4

P4: f4, f0 x

Prevent/Avoid Deadlock in Dining PhilosophersSemaphore

→ put semaphore control on taking and releasing the forks.

Non Semaphore based

→ set of 'N' philosophers,
 $\text{Let } (N-1) \rightarrow L-R$
 and $1 \rightarrow R-L$

→ Even numbered : $L-R$
 Odd numbered : $R-L$

Q4. A solution to the dining philosophers problem which avoids deadlock is:

Ensure that one particular philosopher picks up the left fork before right fork, and all other philosophers pick up right before left fork.

Q5. Let $m[0..4]$ be mutexes and $P_0 - P_4$ be processes. Suppose each process $P[i]$ executes the following

wait($m[i]$): wait($m[i+1] \cdot 0.4$)

<CS>

release($m[i]$; release($m[i-1] \cdot 0.4$)

Which situation could occur?

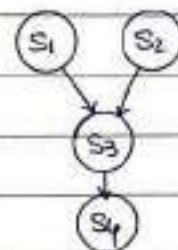
Ans: Deadlock.

Concurrency vs Sequentiality

$\{$
 $S_1: a = b + c;$
 $S_2: d = e * f;$
 $S_3: k = a + d;$
 $S_4: l = k * 10;$
 $\}$

$T_1: \text{Load } R_1, b$
 $T_2: \text{Load } R_2, c$
 $T_3: \text{Add } R_1, R_2$
 $T_4: \text{Store } a, R_1$

* Only S_1 and S_2 can perform concurrently (no dependency)



Precedence Graph (directed)

S_i : High level language statement.

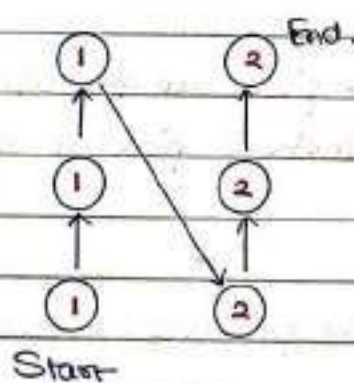
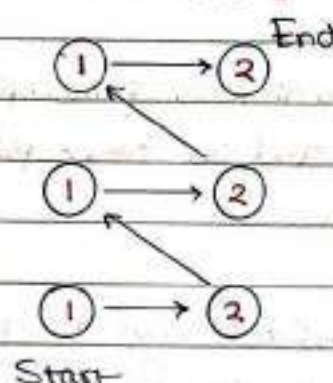
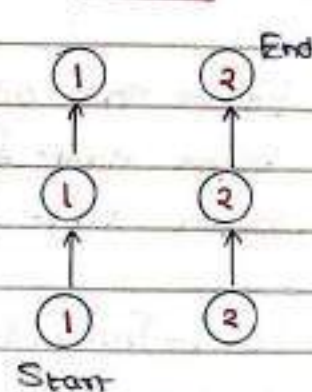
ConcurrencyParallelism

* A system is said to be concurrent if it can support two or more actions in progress at the same time

A system is said to be parallel if it can support two or more actions executing simultaneously

* Concurrency is about dealing with lots of things at once.

Parallelism is about doing lots of things at once.

SequentialConcurrentParallelAn Analogy:

Concurrent: Two queues and one coffee machine

Parallel: Two queues and two coffee machines

Note:

- * In Concurrency process will execute with preemption.
- * Parallelism is possible with multiple CPUs/Cores.
- * Concurrency is about structure, parallelism is about execution.
- * Concurrency provides a way to structure a solution to solve a problem that may (but not necessarily) be parallelizable.

* The modern world parallel it has:

1. Multi Cores
2. Networks.
3. Cloud of CPUs
4. Loads of users.

* Concurrency makes parallelism easy.

Types of Concurrency

1. Pseudo (one CPU)
2. Real/Physical (Parallelism) → Multiple processing units

→ Interleaved execution
Among processes/
Applications

Concurrency Conditions

→ $S_i = a = b + c; \quad S_j = d = e * f;$

* Output of one statement should not serve as input to another

S ← $R(S)$: Read set
 $W(S)$: Write Set

eg: (I) $S: a = b + c$
 $R(S) \rightarrow b, c; \quad W(S) \rightarrow a.$

(II) $S: a += ++b + --c;$
 $R(S) \text{ and } W(S) \rightarrow a, b, c.$

(III) $S: \text{scanf}("%d", \&x);$
 $R(S) \rightarrow \phi$
 $W(S) \rightarrow x$

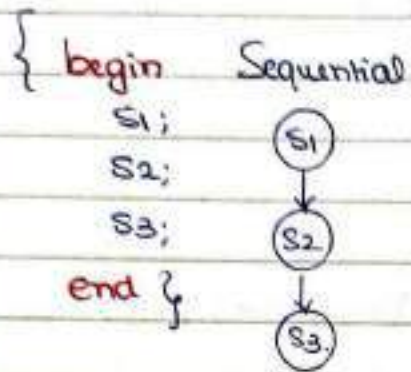
(IV) $\text{printf}(x);$
 $R(S) \rightarrow x$
 $W(S) \rightarrow \phi$

Bernstein's Concurrency Conditions: I. $R(S_i) \cap W(S_j) = \phi$

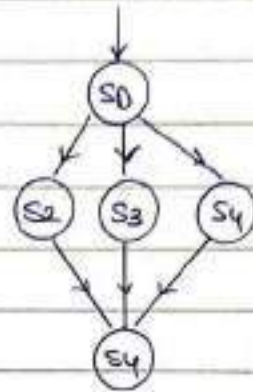
II. $W(S_i) \cap R(S_j) = \phi$

III. $W(S_i) \cap W(S_j) = \phi$

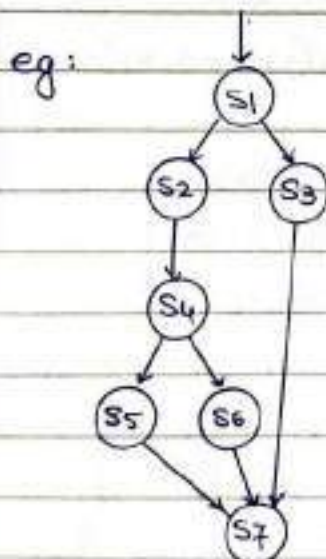
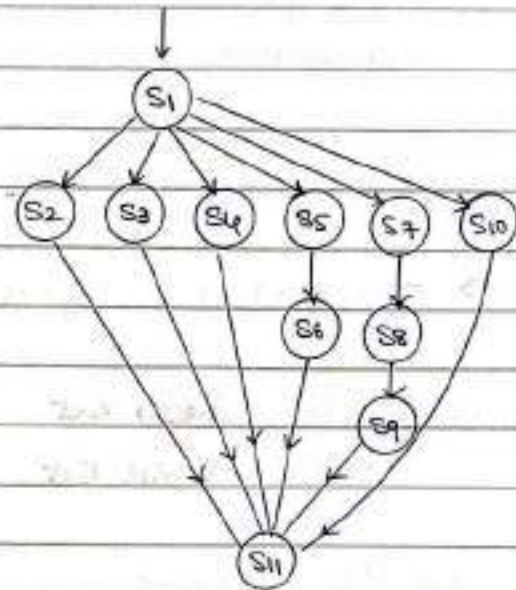
IV. $R(S_i) \cap R(S_j) = \text{may/may not } \phi$

Concurrency mechanisms1. Parbegin - Parend / Cobegin - Coend:

S0;
Parbegin
 S1;
 S2;
 S3;
Parend
 S4;



eg: S1;
Parbegin
 S2; S3;
 S4;
 begin S5; S6; end
 begin S7; S8; S9; end
 S10;
Parend
 S11;

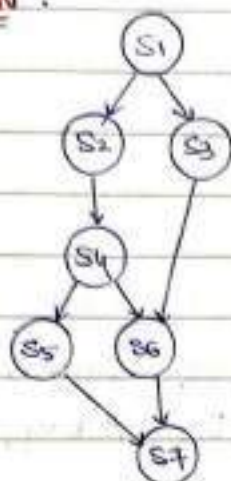


```

S1;
Parbegin
  begin
    S2; S4;
    parbegin
      S5; S6;
    parend
  end
  S3;
Parend
S7;

```

How many times
parallel construct do
be used?

HW :S1: BSEM a, b, c, d, e, f, g = { ϕ }

Cobegin

begin S1; v(a); v(b); end

begin P(a); S2; S4; v(c); v(d); end

begin P(b); S3; v(e); end

begin P(c); S5; v(f); end

begin P(d); P(e); S6; v(g); end

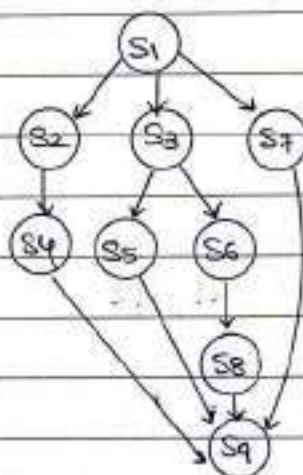
begin P(f); P(g); S7; end

Coend

Q1. Draw the precedence graph for the Concurrent Program.

```

S1;
Parbegin
  begin
    S2; S4; end
  S3;
  Parbegin S5;
    begin S6; S8; end
  Parend
end.
S7;
Parend;
S9;
  
```



Q2. void P(void) void Q(void) void main()

```

{ A;
  B;
  C; }
{ D;
  E; }
{ Parbegin
  P(); Q();
  Parend; }
```

Valid output Sequence: ABCDE
DEABC, ADBEC.

Q3. int x=0; y=0;

Co begin

begin

S1: x=1;

S2: y=y+x;

end

begin

S3: y=2;

S4: x=x+3;

end

Coend.

Final values of x and y

I. x=1, y=2

~~II. x=1, y=3~~~~III. x=4, y=6~~

Statements S3, S4, S1, S2 result in x=1, y=3.

Statements S1, S3, S4, S2 → x=4, y=6

Q4. int x=0, y=20;

Bsem mx=1; my=1;

Final possible values
of x 21, 22

Co begin

begin

P(mx);

x=x+1;

V(mx);

end;

begin

P(my);

x=y+1;

V(my);

end

Coend

HW

Q5. Integer B=2;

P1()

{ C=B-1;
B=2*C; }

P2()

{ D=2*B;
B=D-1; }

main()

{ Parbegin
P1()
P2();
Parend }

The no. of distinct values of B are _____

2. Fork join

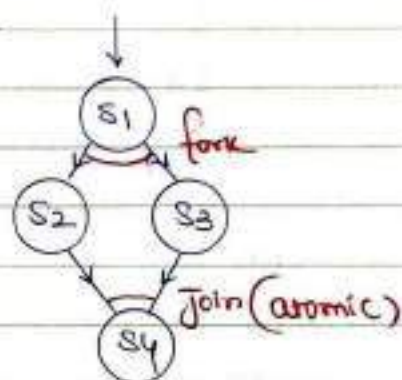
Syntax of fork : fork L;

→ Execution of fork results in starting 2 computation concurrently

I. Which is immediately after fork

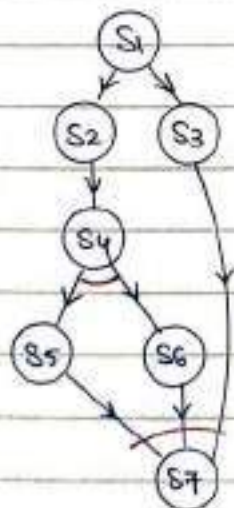
II. which is at Label 'L'.

eg: int Count=2;
S1;
fork L;
S2;
goto X;
L: S3;
X: Join(Count);
S4;



Join (int count)
{
 count = count - 1;
 if (count != 0) then exit;
 else
 return;
}

eg:

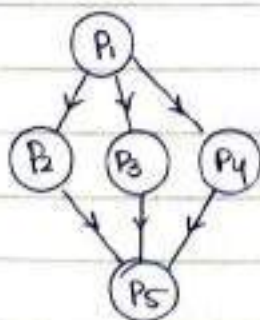


int Count=3;

S1;
fork L;
S2; S4;
fork X;
S5;
goto 2;

L: S3;
goto 2;
X: S6;
Z: Join(Count);
S7;

eg



P1;
fork L;
P2;
L: fork X
P3;
goto 2;

X: P4
Z: Join(count);
P5;

eg.

N=2, M=2

Draw the precedence graph.

fork L3, L4;

S1;

L1: Join N

S3; &

L2: Join M

S5: goto next;

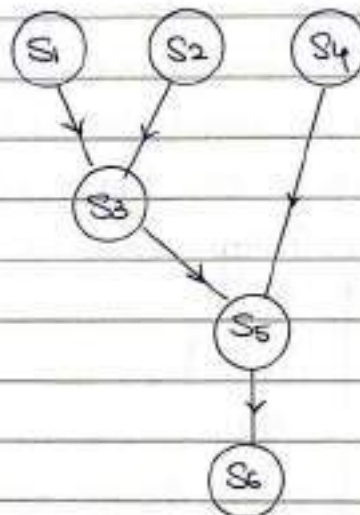
L3: S2;

goto L1;

L4: S4;

goto L2;

next: S6;



Q1. Consider the following pseudocode where S is a Semaphore initialized to five and Counter is a shared variable initialized to 0 in line 1. Assume that Increment Operation in line #7 is not atomic.

int Counter = 0;

Semaphore S = init(5);

void parop(void)

{ wait(S);

wait(S);

Counter++;

Signal(S);

Signal(S);

If five processes execute the function parop Concurrently, which of the following program behaviour is/are possible?

I. there is a deadlock involving all process.

II. Counter = 5 after all processes comp. parop

III. Counter = 1

IV. Counter = 0

Q2. int count=0; void test()

{ int i, n=5;

for (int i=1; i<=n; ++i)

count = count + 1; }

main()

{ Parbegin

test();

test();

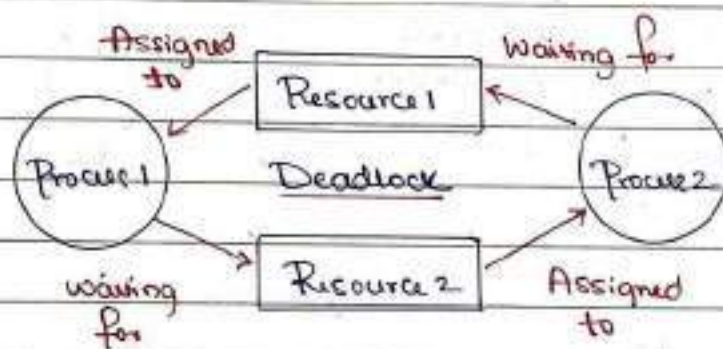
Parend }

Deadlock

Deadlock: Two or more processes are said to be in deadlock if they wait for the happening of an event which will never happen.

Consequences:

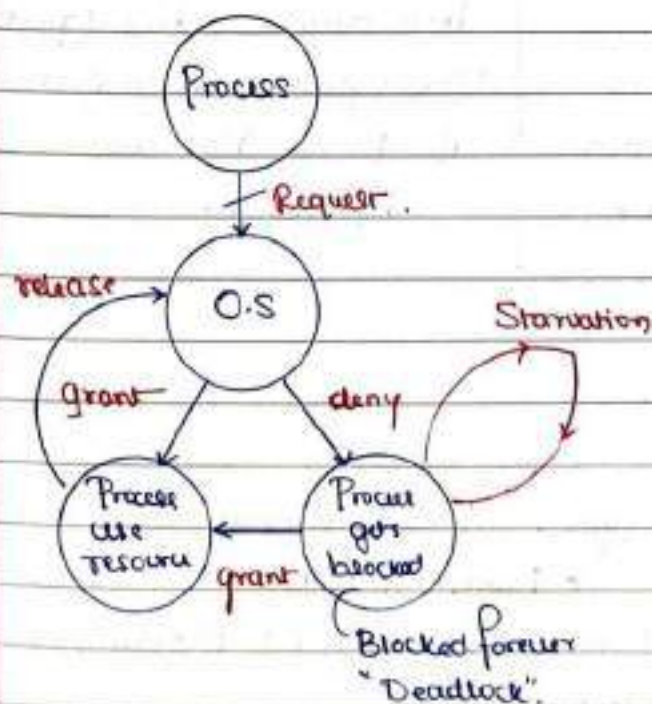
- * Throughput/Efficiency drops
- * Ineffective utilization of resources



System Model:

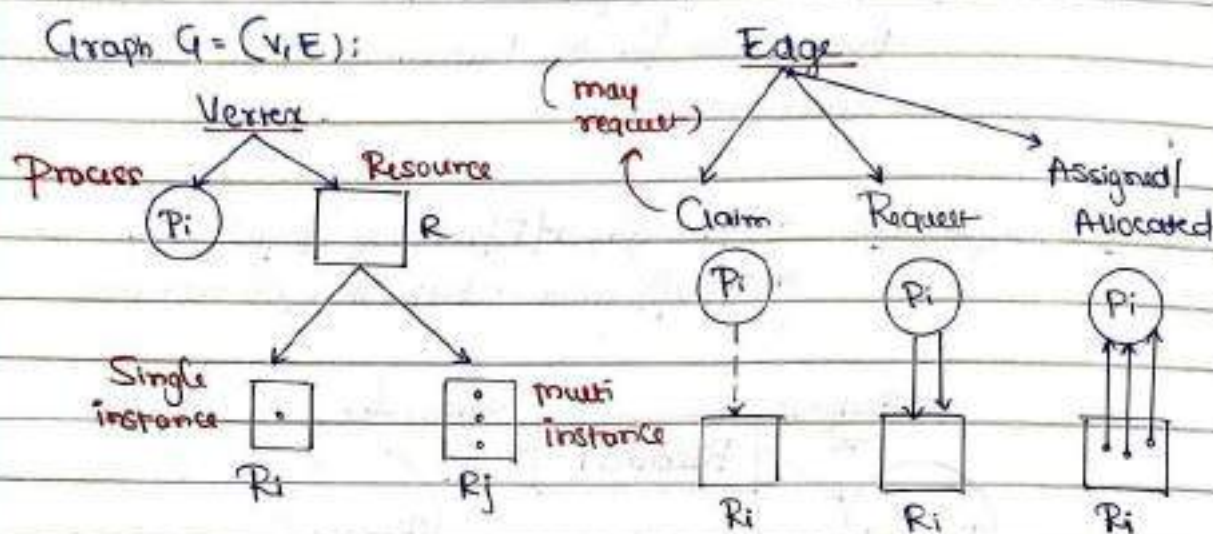
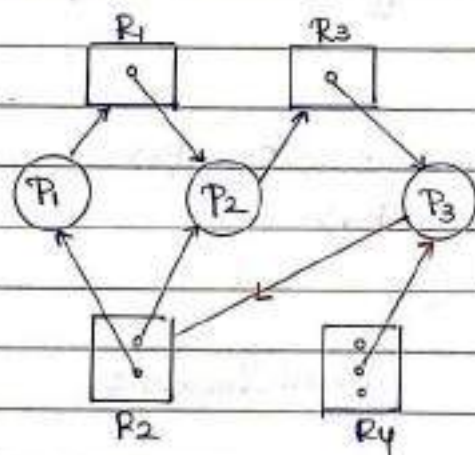
→ n : no. of processes $\langle P_1 \dots P_n \rangle$
 → m : no. of resources $\langle R_1 \dots R_m \rangle$

Single instance Multi Instance

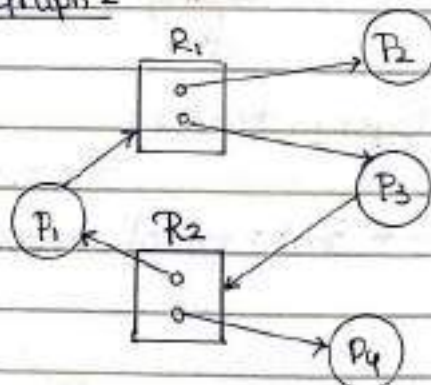


Necessary Conditions for Deadlock:

1. Mutual Exclusion
↳ Shared resource $\langle CS \rangle$
2. Hold and wait
Every thread is not deadlock but every deadlock will have H&W.
3. No pre-emption of resources
4. Circular wait

Resource Allocation Graph (RAG)Graph 1.

It is deadlock free.
After changes it will result in
deadlock ($P_1-R_1-P_2-R_3-P_3-R_2-P_1$)
cycle

Graph 2

It is deadlock free ($P_1-R_1-P_3-R_2-P_4$).
This system will go in deadlock
if P_3 req R_2 and
 P_4 req R_1

Deadlock Handling Strategies

1. Deadlock Prevention } Type 1 (Deadlock never occurs)
2. Deadlock avoidance } Banker's Algorithm
3. Deadlock detection and recovery } Type 2 (Deadlock definitely occurs.)
4. Deadlock Ignorance } Ostrich Algo

I. Deadlock Ignorance (Ostrich Algorithm)

- * Pretend there is no problem (Ignorant Strategy of ostrich)
- * Reasonable if : (i) deadlock occur very rarely
(ii) cost of prevention is high.
- * UNIX and Windows take this approach
- * It is a trade-off between convenience and correctness.

II. Deadlock Prevention

Deadlock Prevention can be done by dissatisfying or negating the necessary conditions of deadlock.

a) Mutual Exclusion : $\rightarrow$ Since every multi programmed Operating System has atleast one shared resource $\therefore$ M/E is non dissatisfiable.

b) Hold and Wait $\rightarrow$ Hold or Wait Implementation

(i) Process must request and be allocated all resources prior to its start.

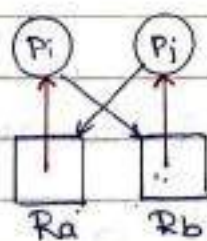
Drawbacks : Causes/leads to starvation, inefficiency

(ii) Process must release all resources before making a fresh/new request.

Drawback : Suffer from starvation

(iii)

c) No preemption : $\rightarrow$ Pre-emption of resources from processes.



forced

Self

(Running process should not block)

(allows other processes to complete first)

Drawbacks : Causes Starvation.

d) Circular-wait : $\rightarrow$ is prevented by a total order relation among all processes and resources.

(i) Assign unique numbers/IDs to each resource

(ii) Never allow a process to request a lower numbered resource than the last one allocated.

Drawback : Starvation possible

Note :

- (1) If the R.A.G. has resource of only multi instance type, then cycle is only a necessary condition
- (2) If R.A.G has resource of both single and multi instance, then presence of cycle is only a necessary condition for deadlock.
- (3) If the R.A.G has resource of only single instance type, then cycle is a necessary and sufficient condition for deadlock.

II

Deadlock Avoidance

1. Single instance resource

<Resource allocation
Graph algorithm>

2. Multi instance of Resource (SI+MI)

Banker's Algorithm

(i) Safety algorithm

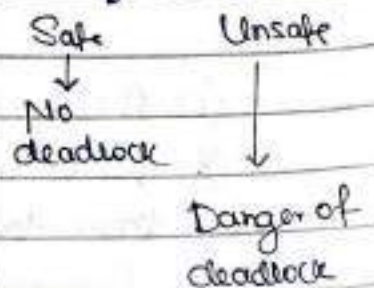
(ii) Resource request algorithm

Note: Both the algorithms are based on Apriori Knowledge/ information.

1. Resource - Allocation Graph Algorithm <Single instance resource>

- * Resource are claimed a priori in the system.
- * If process P_i starts executing then all Claim edges must appear in R.A.G.
- * If P_i 's request resource R_j , then the request is granted only if, converting the request edge to assigned edge does not lead to a cycle in R.A.G otherwise

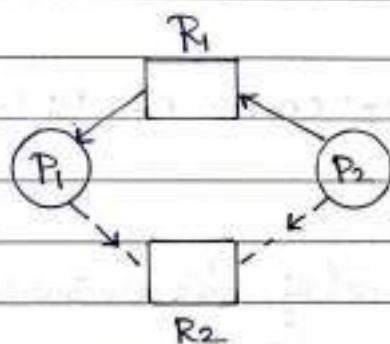
System State



the process is blocked

<No cycle implies Safe State and formation of cycle implies Unsafe State>

- The basic objective of R.A.G algorithm (SI) is to always operate the system is in SAFE state.
- * System is said to be safe, if the conversion of request edge to assigned edge does not lead to cycle in R.A.G. Otherwise if it is leading cycle then it is unsafe mode.



→ Current state: Safe (no cycle)

→ What if P2 also request R2?

System State



Banker's Algorithm (m-Instances)

- (i) Safety algorithm (ii) Resource request algorithm

Data Structures (System state)

1. n : no of processes ($P_1 \dots P_n$)
2. m : no. of resource ($R_1 \dots R_m$)
3. Maximum $[1 \dots n, 1 \dots m]_{n \times m}$:

$$\max [i, j] = k$$

$$P_i \rightarrow k(R_j)$$

Process P_i demands (apriori)
'k' copies of R_j .

4. Allocation $[1..n, 1..m]_{n \times m}$: $Alloc[i, j] = a$
 $P_i \leftarrow a(R_j) \quad [a \leq k]$
5. Need $[1..n, 1..m]_{n \times m}$: Need $[i, j] = b \rightarrow$ Need = Max-allocation
 $P_i \rightarrow b(R_j) \quad [b = k - a]$
6. Request $[1..n, 1..m]_{n \times m}$: $P_i \rightarrow c(R_j)$ @ time 't'.
 $Req[i, j] = c \quad [c \leq b]$
7. Total $[1..m]$
 $Total[j] = 'z' \rightarrow$ There are 'z' copies of R_j in the system.
8. Available $[1..m]$
 $Avail[j] = 'e'$ There are $e(R_j)$ free available @ time 't'.
 $Avail = Total - \sum_{i=1}^n Alloc_i \quad [e \leq z]$

Example : $n=5, m=1, R=22$ (Total)

Pid	Max R	Alloc R	Need R	Avail R
P ₁	10	5	5	3
P ₂	8	4	4	5
P ₃	12	5	7	8
P ₄	5	2	3	13
P ₅	6	3	3	17

$\langle P_4; P_5; P_1; P_2; P_3 \rangle$

↓
 "Safe Sequence"
 (We can have multiple Safe Sequence)

Need = Max - Alloc

<22>

Safety Algorithm:

System is said to be "safe" if the need of all processes can be satisfied with available resources in some order, otherwise it is "unsafe".

Example 2: $\langle A, B, C \rangle = \langle 10, 5, 7 \rangle$ $\langle P_1; P_3; P_4; P_0; P_2 \rangle$

	Allocation	Max	Available	Need	
	A B C	A B C	A B C	A B C	
P ₀	0 1 0	7 5 3	3 3 2	7 4 3	"System is Safe."
P ₁	2 0 0	3 2 2	$\langle 5 \ 3 \ 2 \rangle$	1 2 2	
P ₂	3 0 2	9 0 2	$\langle 7 \ 4 \ 3 \rangle$	6 0 0	
P ₃	2 1 1	2 2 2	$\langle 7 \ 4 \ 5 \rangle$	0 1 1	
P ₄	0 0 2	4 3 3	$\langle 7 \ 5 \ 5 \rangle$	4 3 1	
			$\langle 10 \ 5 \ 7 \rangle$		

(ii) Resource request algorithm

Algorithm Res-request($P_i, \text{Req}_i, \text{Alloc}_i, \text{Need}_i, \text{Avail}$)

- {
1. $\text{Req}_i \leq \text{Need}_i$
 2. $\text{Req}_i \leq \text{Alloc}_i$
 3. [Assume to have satisfied the Req_i]
 - a. $\text{Avail} = \text{Avail} - \text{Req}_i$
 - b. $\text{Need}_i = \text{Need}_i - \text{Req}_i$
 - c. $\text{Alloc}_i = \text{Alloc}_i + \text{Req}_i$
 4. Run safety algorithm
 5. If system is SAFE - then grant the Req_i
 else deny the Req_i and block the process P_i ; }

HW

Processes	Allocation	Max	Available
	A B C D	A B C D	A B C D
P ₀	0 0 1 2	0 0 1 2	1 5 2 0
P ₁	1 0 0 0	1 7 5 0	
P ₂	1 3 5 4	2 3 5 6	
P ₃	0 6 3 2	0 6 5 2	
P ₄	0 0 1 4	0 6 5 6	

Q1. An OS uses the banker's algorithm for deadlock avoidance when managing the allocation of three resources types x, y and z to the processes P_0, P_1 and P_2 . The table given below presents the Current System State. Here the allocation matrix shows the current no. of resources of each type allocated to each process and max matrix shows the maximum number of resource of each type required by each process during its execution.

	Allocation			Max		
	x	y	z	x	y	z
P_0	0	0	1	8	4	3
P_1	3	2	0	6	2	0
P_2	2	1	1	3	3	3

There are 3 units of type x, 2 units of type y and 2 units of z still available. System is currently in safe state. Consider the following independent requests for additional resources in current state.

Req1: P_0 requests 0 units of x, 0 units of y and 2 units of z

Req2: P_1 requests 2 units of x, 0 unit of y and 0 of z.

	Max			Alloc			Need			Avail		
	x	y	z	x	y	z	x	y	z	x	y	z
P_0	8	4	3	0	0	1	8	4	2	3	2	2
P_1	6	2	0	3	2	0	3	0	0	1	2	2
P_2	3	3	3	2	1	1	1	2	2	6	4	2

To: $P_0 \rightarrow \langle 0, 0, 2 \rangle$
 $\langle P_1, \dots \rangle$

System is unsafe
 & Req1 is not granted

T1: $P_1 \rightarrow \langle 2, 0, 0 \rangle$

$\langle 8, 5, 4 \rangle$

Request 2: $\langle P_1, P_2, P_0 \rangle$

$\therefore$ System is safe and request is granted

Only Request 2 is granted and Request 1 is not granted

(iv) Deadlock detection and recovery:

- When to activate/activate (apply detection algo.)?
- Under utilization of CPU
 - majority of processes are blocked.

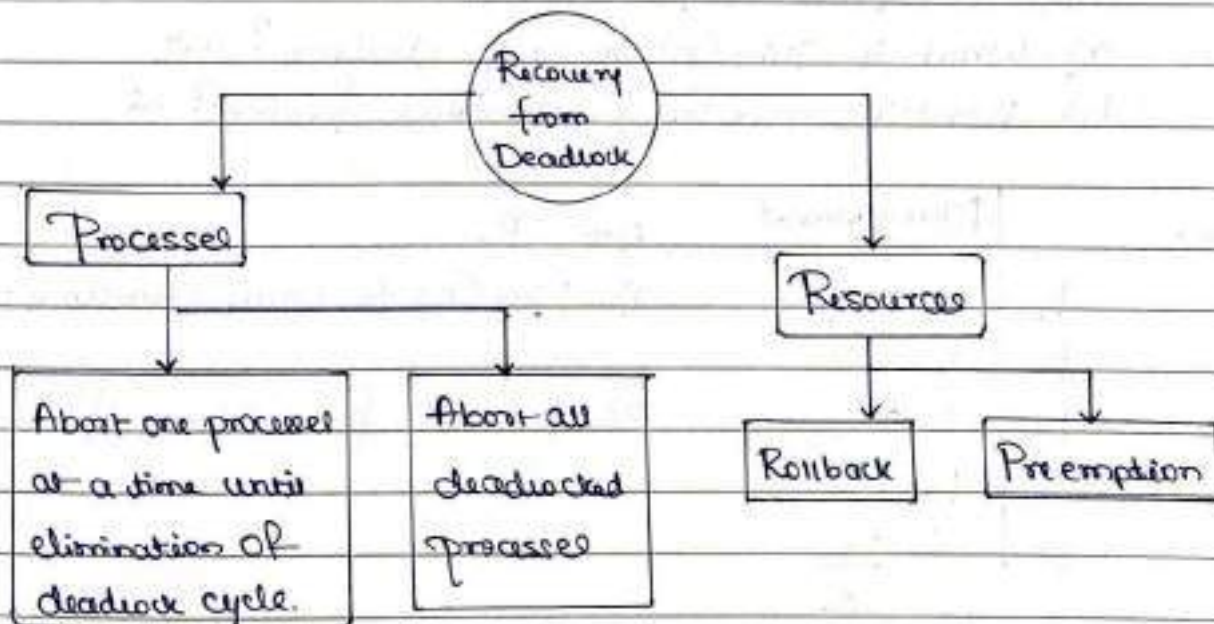
Deadlock detection

For single instance
of every resource

For multi instances of
every resource

Using the wait for
Graph method

Using the safety algorithm
method



1. Prevention } Class 1
2. Avoidance }
3. Detection and Recovery } Class 2
4. Ignorance } Class 3

Conceptual Problems.

- Q2. Consider a system having 'n' processes and a single 'R' having '6' copies. Each process need 2 copies of R to complete.
- a) What is min (n) to cause deadlock? $\rightarrow 6$
- b) What is max (n) to deadlock freedom? $\rightarrow 5$

- Q3. $N=3$, Processes (P_1, P_2, P_3)
 $R=?$
 $P_i = 2(R)$

- a) What is Max (R) to cause deadlock? $\rightarrow 3$
- b) What is Min (R) to free deadlock? $\rightarrow 4$

- Q4. N-processes

$R = 6$ copies. P_i requires: 3(R)

- a) What is Min (n) to cause deadlock? $\rightarrow 3$
- b) What is max (n) for deadlock freedom? $\rightarrow 2$

Q5.

	Max-demand	$N=5, R$
P_1	10 - 9	a) Max(R) to cause deadlock: 36 b) Min (R) for deadlock freedom: 37
P_2	8 - 7	
P_3	5 - 4	
P_4	12 - 11	
P_5	6 - 5	
	<u>36</u>	

- Q6. Which of them is not a valid deadlock prevention scheme?

Ans. Never request a resource after releasing any resource

- Q7. Which is not true of deadlock prevention and deadlock avoidance?

Ans. In deadlock prevention the request for resources is always granted if the resulting state is safe.

Q8. An OS implements a policy that requires a process to release all resources before making a request for another resource. Select the statement.

Ans. Starvation can occur but deadlock cannot occur.

Q9. A computer has 6 tape drives, with n processes competing for them. Each process may need two drives. What is the max value of ' n ' for deadlock freedom?

Ans. 5

Q10. Consider a system having m resource of same type. These resource are shared by 3 processes A, B, C which have a peak demand 3, 4, 6 resp. What value of ' m ' deadlock will not occur?

Ans. Value should be greater than 10.

Q11. A system shares 9 tape drives. The current allocation and max req. of tape drives for:
Which of them best describes the current state of system?

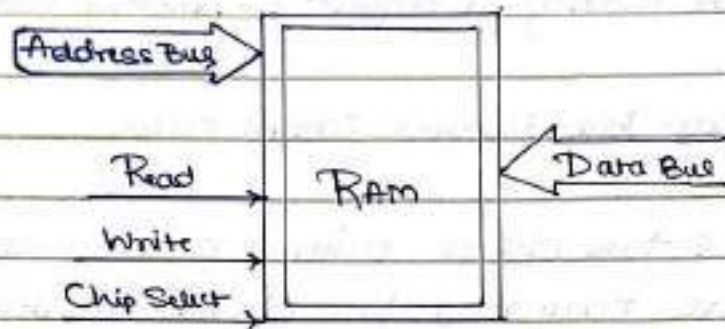
Process	Current Allocation	Max. Req.	Ans: Safe, Not deadlocked
P ₁	3	7	
P ₂	1	6	
P ₃	3	5	

Q12. Which of following statements is/are true w.r.t deadlock?

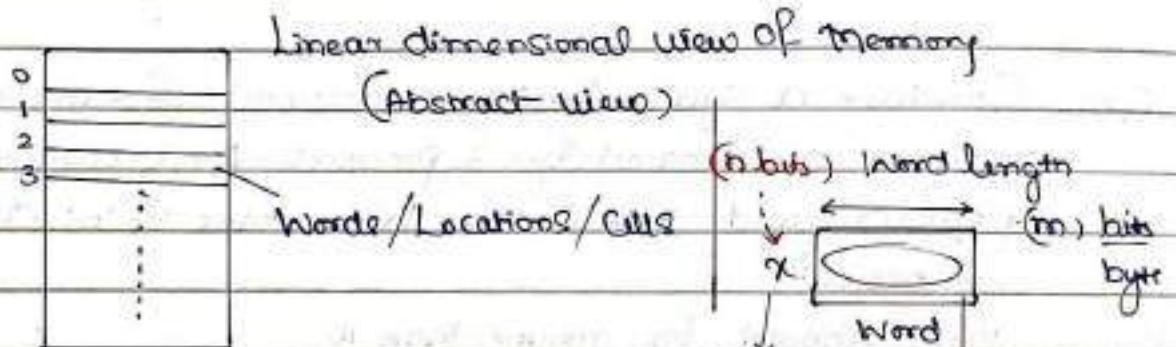
Ans. a. Circular wait is a necessary condition for formation of deadlock.

d. In the resource allocation graph of a system, if every edge is an assignment edge, then system is not in deadlock state.

Memory Management



Block diagram - Primary Memory



Total no. of words
= 'N' Memory Capacity

Fixed length address
You can store instructions + data

Memory: Memory is specified as number of words $\times$ width of words
($N \times m$)

n : Size of address in bit

m : word length / width of word

N : total number of words

I. Relation b/w N & n :

→ Binary bits: (0, 1)

→ Using 3 binary bit 8 combinations are possible

→ ' n ' bits: $\rightarrow 2^n$ combinations possible

eg:



$$N = 8 \text{ words}$$

$$n = 3 \text{ bits}$$

$$2^3 = 8W$$

$$N = 2^n \text{ words}$$

$$n = \log_2 N \text{ bits}$$

Conversions:

$$2^{10} = 1024 = 1K \text{ words}$$

$$2^{20} = 20 \text{ M. words}$$

$$2^{30} = 1G \text{ words}$$

$$2^{40} = 1T \text{ words}$$

$$2^{50} = 1P \text{ words}$$

eg:

$$N = 256$$

$$n = \log_2 256 = 8 \text{ bits}$$

$$n = 6 \text{ bits}$$

$$N = 2^6 = 64 \text{ words}$$

$$* n = \log_2 Z \text{ bit}$$

$$N = \frac{(1024^Z)}{2}$$

$$= \frac{(1024^Z)}{2} \therefore \underline{N = Z \text{ Bytes}}$$

I. $N \propto n \propto m$.

1. $n = 13 \text{ bits}$
 $m = 8 \text{ bits}$
 $N = 8KW$
 $= 8KB$

2. $n = 18 \text{ bits}$
 $m = 16 \text{ bits}$
 $N = 256KW \text{ Word view}$
 $= 512KB \text{ Byte view}$

3. $n = 33 \text{ bits}$
 $m = 64 \text{ bits}$
 $N = 8GW$
 $= 64GB$

4. $N = 64KB$
 $m = 4B$
 $n = \frac{64KB}{4B} = 16K \text{ Words}$

5. $n = 23 \text{ bits}$
 $m = 32 \text{ bits}$
 $N = 2^{23} = 2^3 \times 2^{20} = 8M \text{ Words}$
 $N = 8M \times 4 = 32M \text{ Bytes}$

6. $N = 256MB$
 $m = 128B$
 $N = \frac{256MB}{128B} = 2M \text{ words}$

$n = 21 \text{ bits (W)}$
 $n = 28 \text{ bits (B)}$

7. $N = 8G \text{ bits (Twist)}$
 $m = 64 \text{ bits}$
 $N = 128M \text{ words}$
 $N = 1G \text{ Bytes}$

Loading vs Linking

Loading: Loading refers to the activity of loading the program or bringing the program from disk to memory. The component of the operating system that does this activity is called "loader".

The two approaches of loading are: 1. Static loading
2. Dynamic loading

1. Static loading: The whole program is loaded in memory before starting the execution.

* Advantage: faster execution (time efficient)

* Drawback: possible wastage of space (ineffective utilization of memory)

2. Dynamic loading: The process of loading the program on demand at runtime is called dynamic loading.

* Advantage: no wastage of memory

* Drawback: slow execution (time inefficient)

Linking: Linking is the process of resolving external references in the program

```
#include <stdio.h>
:
main()
{
:
    fun(); BFA
:
    scanf(); BFA
}
```

```
fun()
{ ...
...
... }
```

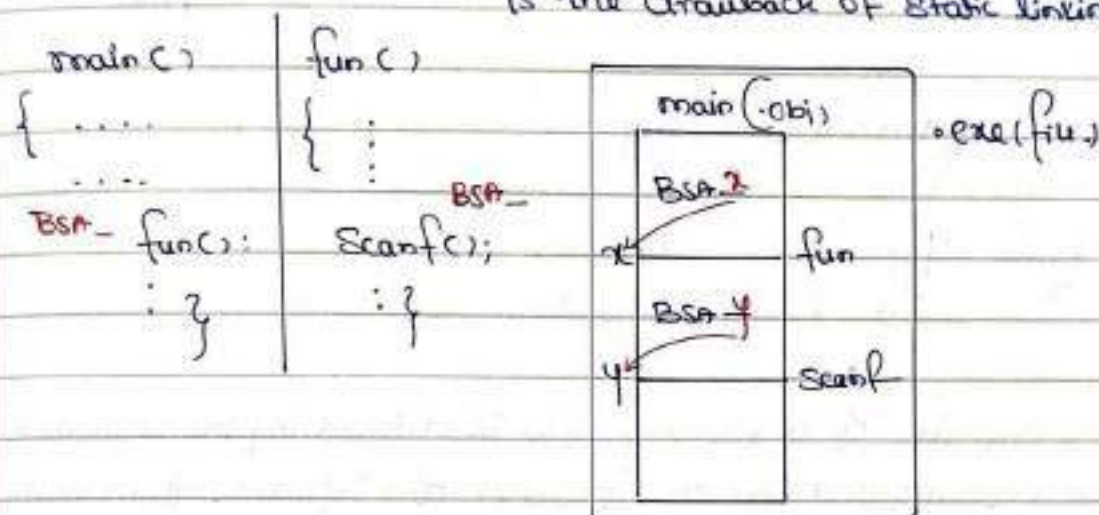
Note: Compilation always starts with / from first line of code

* Execution of program always starts with main()

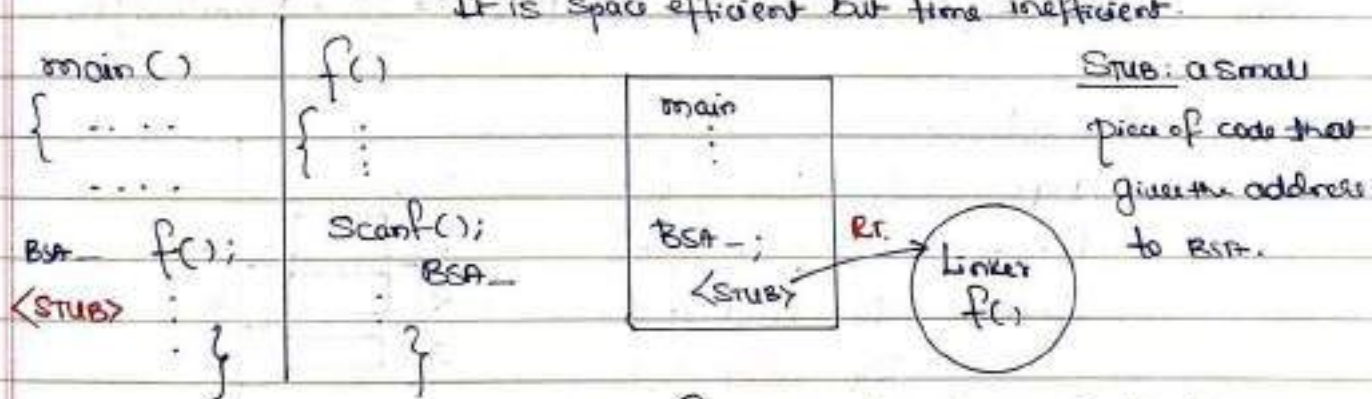
BFA: Branch and save address

The two ways of linking are: 1. Static linking
2. Dynamic linking

Static linking: Linking of program before execution. Space inefficiency is the drawback of static linking.



Dynamic Linking: Linking at runtime is called dynamic linking. It is space efficient but time inefficient.



dll: (Dynamic link library) The libraries linked at runtime using principle of dynamic linking is called DLL

Advantages:

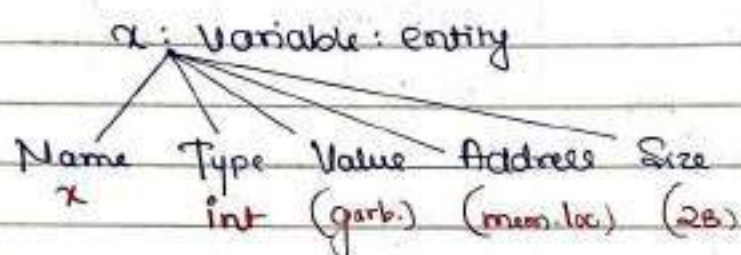
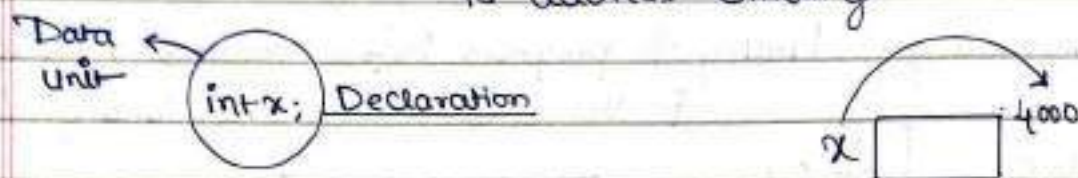
- * Space efficiency
- * Reusability
- * Modification (flexibility) is easy.

Drawbacks:

- * Time inefficiency
- * Less Security (Insecure)

Note: Static linking is more secure

Address Binding: Association of program instructions and data units to memory locations (address) is address binding.



Q1. The capacity of a memory unit is defined by the number of words multiplied by the number of bits/word. How many separate address and data lines are needed for a memory of $4K \times 16$?

Ans $n = \log_2 4K = 12 \text{ bits}$, $m = 16 \text{ bits} = 2B = \text{Data line / bus}$

int a, b, c;

a = 1;	T ₁ : Store a, #1	0		
b = 2;	T ₂ : Store b, #2	1		a
C = a + b;	T ₃ : Load r ₁ , a	2		b
	T ₄ : Load r ₂ , b	3		c
	T ₅ : Add r ₁ , r ₂	4	T ₁	
	T ₆ : Store C, r ₁	5	T ₂	

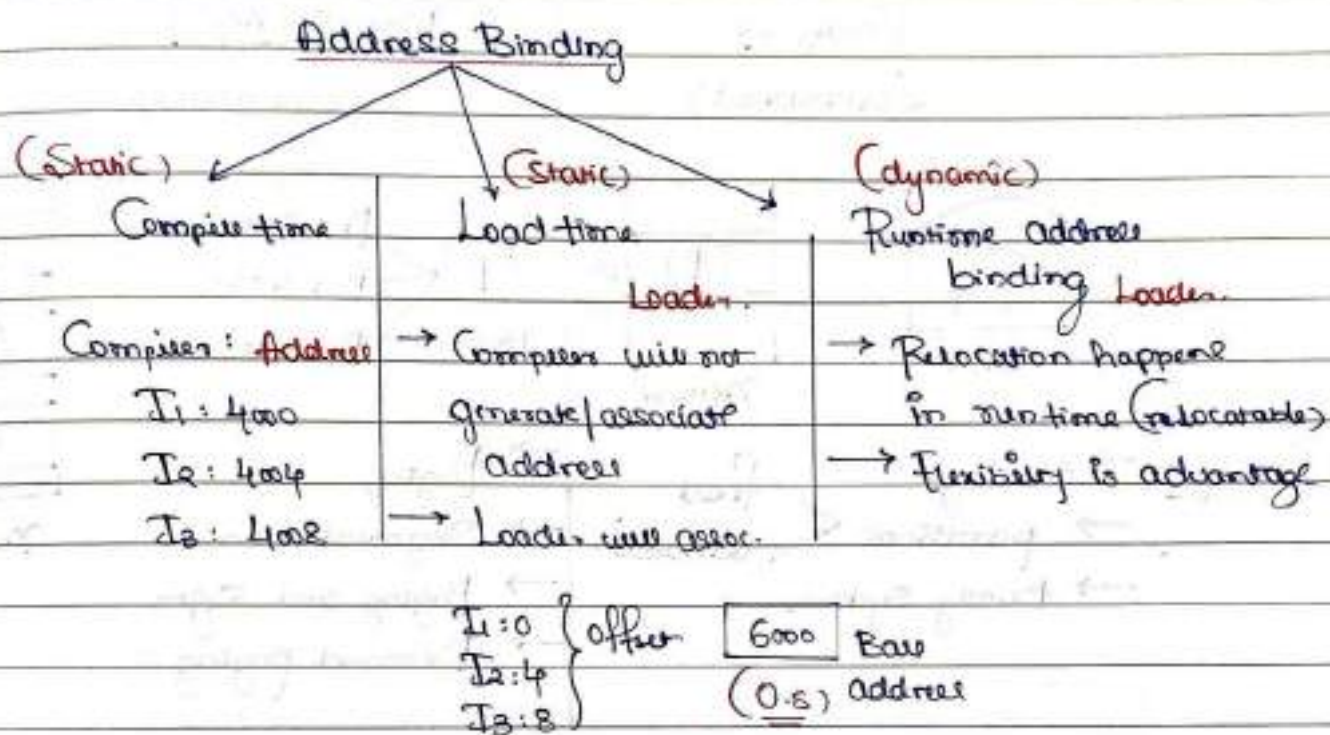
Binding time: Time at which the binding takes place is called as binding time.

int(x);
x = 1;

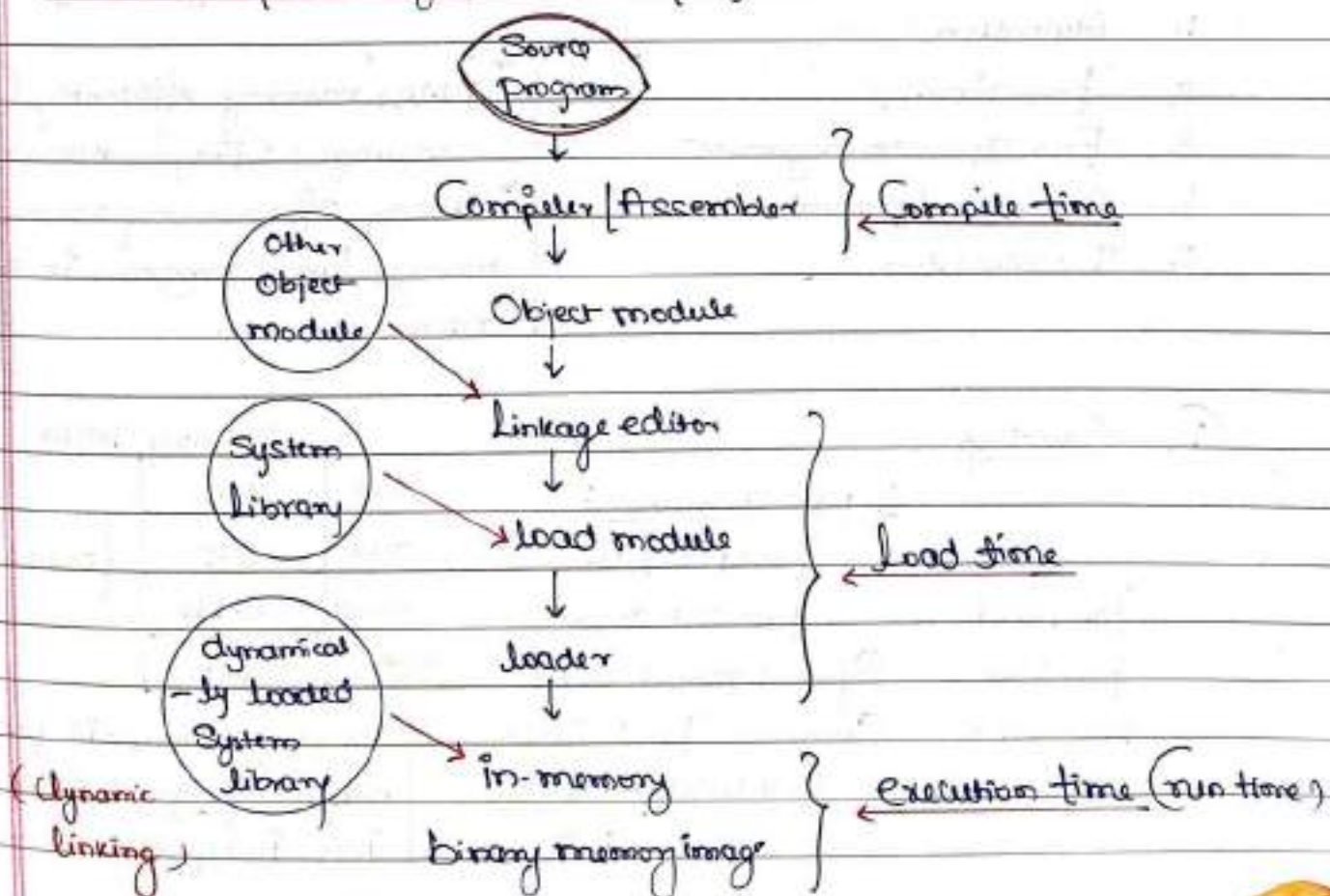
Type : at compile time (static binding)
Address : at load time (st. b.)
Value : run time (address binding)
Size : compile time (st. b.)

Types of binding:

1. Static binding (cannot change)
2. Dynamic binding (can change)



Multistep processing of a user program



Memory Management Technique.Contiguous
<Centralized>Non-Contiguous

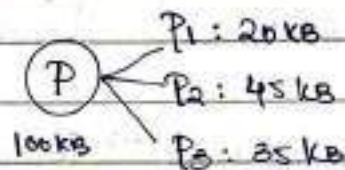
<decentralized>



Disk



Memory



100KB



Memory

Technique

→ Overlay
→ partitions
→ Buddy System

Fixed
Variable

→ Paging
→ Segmentation
→ Paging and Segm.
→ Demand paging

Function of Memory Manager:

1. Allocation
2. Protection
3. Free Space management
4. Address translation
5. Deallocation

Goals of memory manager:

- * Utilize memory efficiently (min wastage) <Fragmentation>
- * Ability of memory manager to manage larger program in small memory areas.

① Overlay.2 pass assembler:

Pass 1: 70KB

Pass 2: 80KB

Symbol table: 30KB

Common Rts: 20KB

O.V loader: 10KB

: 210KB

Memory: 150KB

10KB

20KB

20KB

90KB

O.L

S.T

C. Rts

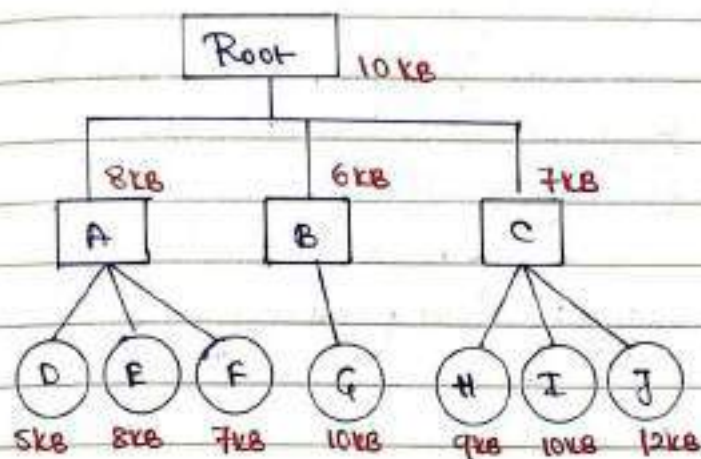
Pass 1.

{ overlay = replace }

Pass 1 and
Pass 2 are
independent.

Overlay concept is possible
only if "program divisible"
into independent modules.

Consider the following program expressed as an overlay.



Prog-Size : 92 KB (on disk)

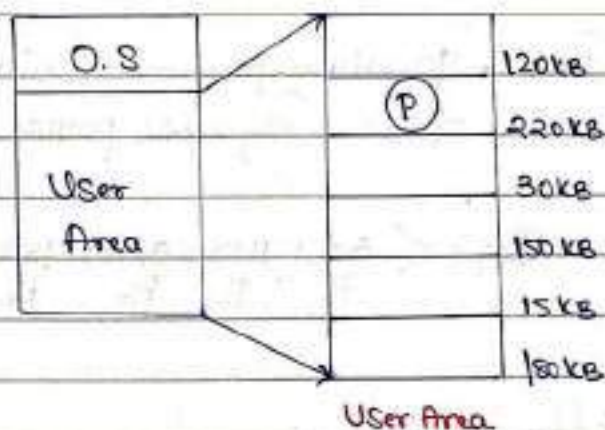
* What is minimum memory required to successfully execute this program using Overlay?

$$\text{Min. Memory Required} = \max \left\{ \begin{array}{l} \text{Path length} \\ \text{from root} \\ \text{to leaf} \end{array} \right\}$$

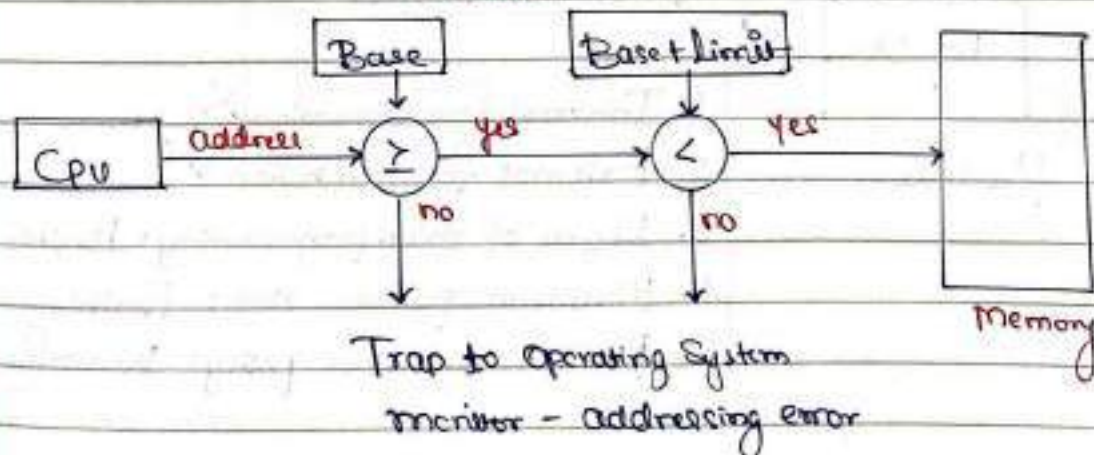
$$= 29 \text{ KB}$$

② Partitions:

(1) Fixed partitions (MFT): Multiprogramming with fixed tasks.



- User area will be divided into fixed no. of partitions
- 1 partition = 1 partition
- partitions maybe of different size.



Partition allocation policies:

1. First fit: first free big enough (Internal fragmentation)
2. Best fit: smallest free big enough
3. Next fit: next fit works like first fit, but searches for free partition beginning from last allocation.
4. Worst fit: largest free big enough.

Performance of fixed partition: (fixed partition)

1. Internal fragmentation ✓
2. External fragmentation X
3. Degree of multi programming: Limited
4. Maximum process size: Limited
5. Partition allocation policy: Best fit (less internal fragment.)

(ii) Variable partitions (movT): Multi programming with variable tasks (dynamic partitioning).

Memory (free hole)	
User	P ₁ 85K
Area	P ₂ 115K
	P ₃ 315K
	P ₄ 15K
	P ₅ 120K

() free hole

Process Req's < 35K; 115K; 315K; 15K; 120K;
P₁ P₂ P₃ P₄ P₅

Performance issue:

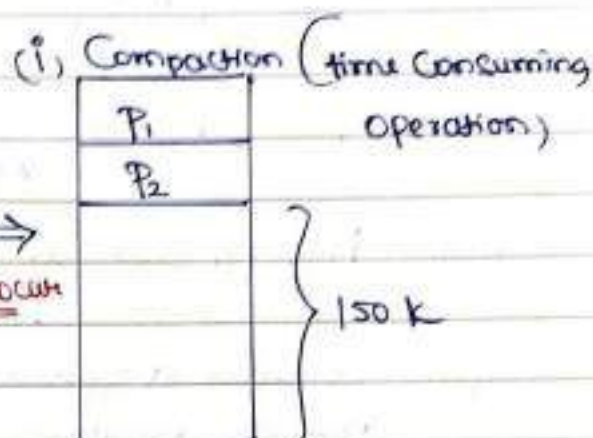
1. Internal fragmentation X
2. External fragmentation ✓
3. Degree of multi programming: Flexible
4. Maximum process size: Flexible
5. Partition allocation policy: Worst fit is better.

External Fragmentation:

Memory map.

Free space: 150k
 Process: P₃ - 85k
 (External frag of 150k)

⇒
relocate



* Compaction is not possible if there is no runtime address binding.

(ii) Non Contiguous allocation:

- * One of the reason of having non contiguous allocation is to avoid the problem of external fragmentation.
- * N.C.G allocation divides the process into smaller part and distributed to free holes.

Note: Adjacent free holes are automatically merged to one free hole.

Q1. Consider a memory system having 6 partitions of size 200k, 400k, 600k, 500k, 300k, 250k. There are 4 processes of size: 350k, 210k, 468k, 49k. Using Best fit allocation policy, what are partitions that remain unallocated?

P ₄	200
P ₁	400
	600
P ₃	500
	300
P ₂	250

600k, 300k remain unallocated

Q2. Consider the following memory map in which blank regions are not in use and hatched regions are in use. Using variable partitions with no compaction.



The sequence of requests for blocks of size 300K, 20K, 125K, 50K, can be satisfied if we use:

Ans: First-fit but not best-fit.

Q3. Consider a system with memory of size 1000Kbytes. It uses variable partitions with no compaction. Presently there are 2 partitions of sizes 200K and 260K respectively.

(i) What is the allocation request of the process which could be always denied?

Ans: 541K

(ii) What is the smallest allocation request which could be possibly denied?

Ans: 181K

Q4. Consider a system having memory of size 2^{46} Bytes, uses fixed partitioning. It is divided into fixed size partitions each of size 2^{24} Bytes. The OS maintains a process table with one entry per process. Each entry has 2 fields: first, a pointer pointing to partitioning in which process is loaded and second, field is process ID. The size of PID is 4 Bytes.

Calculate:

(a) The size of pointer to the nearest Byte: - 38

$$\text{No. of partitions (N)} = \frac{2^{46}}{2^{24}} = 2^{22}, \quad \text{Partition address} = 22 \text{ bits} \\ = \underline{\underline{38 \text{ Bytes}}}$$

(b) Size of process table in Bytes is if system has 500 processes.

Ptr size = 38

Process ID = 48

$$78 \times 500 \rightarrow \underline{\underline{35000 \text{ B}}}$$

Q5. Consider a system with Variable Partition with no compaction

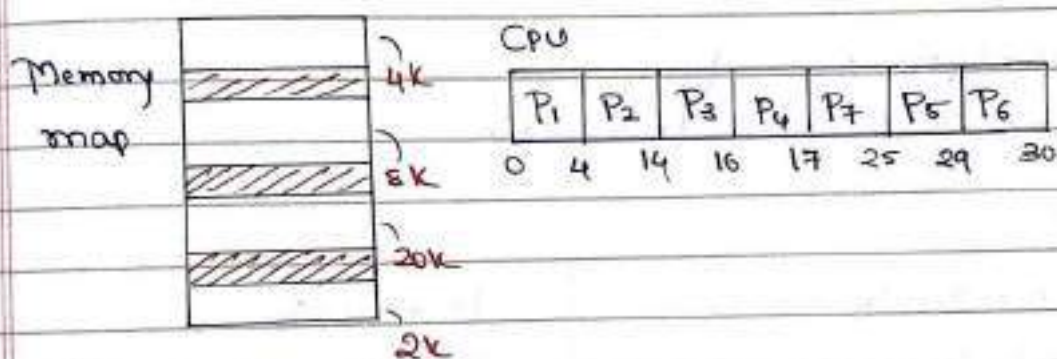
Free hole: 4K, 8K, 20K, 2K.

Program Size: 2K, 14K, 3K, 6K, 10K, 20K, 2K.

Time for execution: 4, 10, 2, 1, 4, 1, 8.

Using best-fit allocation policy and FCFS CPU scheduling, find the time of loading and time of completion of each program.

The burst time in seconds are.



Q6. Consider the allocation of memory to a new process. Assume that none of the existing holes in the memory will exactly fit the process's memory requirement. Hence, a new hole of smaller size will be created if allocation is made in any of the existing holes. Which of the following is true?

Ans - The hole created by best fit is never larger than hole created by first fit.

Non Contiguous Allocation: (to prevent external fragmentation)

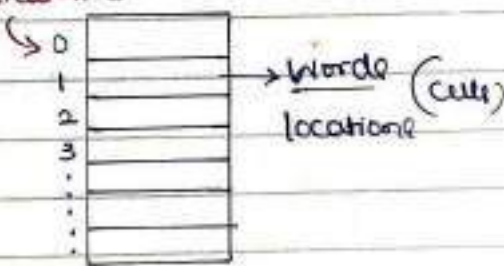
* Address Space: A set of space/words associated with addresses

Logical Address Space

Virtual (A.S)

Physical Address Space.

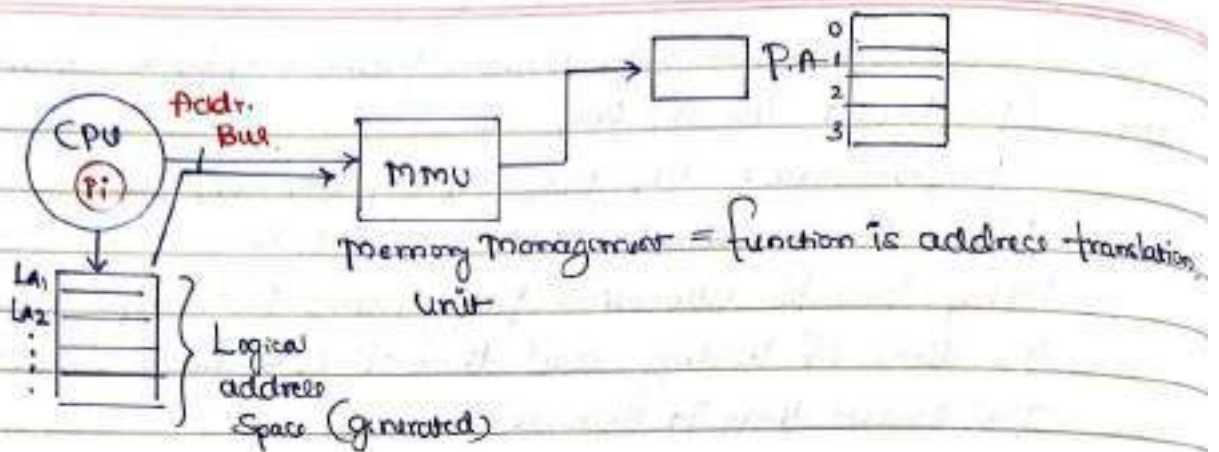
Physical A.S



$$P.A.S = 2^{P.A}$$

$$P.A = \log_2 P.A.S$$

Ram/memory



Eg: ① L.A.S = 64 MB ; P.A.S = 4 MB
 $LA = 26 \text{ bits}$ $PA = 22 \text{ bits}$

② $LA = 33 \text{ bits}$

L.A.S in bytes, if word

Size is 64 bits = 8B.

$$LA_S(B) = 2^{33} \times 8B = 8G \times 8B \\ = 64 GB$$

$PA = 23 \text{ bits}$

P.A.S, W.S = 64 bits = 8B

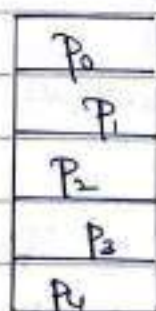
$$8M \times 8B = 64 MB$$

* Logical address: address generated by CPU to access instruction/data unit of program in execution.

* Physical Address (Real/Absolute): address needed to access needed to access the program instructions/data unit in physical memory i.e. address in RAM.

① Simple Paging:

* Organising of L.A.S/V.A.S:



L.A.S: 5KB. $\rightarrow$ L.A.S is divided into equal size units known as pages.

$\rightarrow$ Page size is generally in the power of 2.

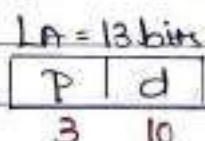
Formulas → No. of page (N) = $\frac{LAS}{PS}$

→ Page offset / displacement (d) = $\log_2 PS$ (bits)

→ Page No (P) = $\log_2 N$
 $N = 2^P$

→ $PS = 2^d B$

L.A. / V.A format



Q1. (i) L.A.S = 32 MB
 P.S = 4 KB = 2^{12}
 $N = 8K [2^{13}]$
 $P = 13$ bits
 $d = 12$ bits
 L.A:

P	d
13	12

$LAS = 2^{25} = 32 MB$

(ii) L.A = 25 bits
 $d = 13$ bits
 What are the no. of pages in the system?
 L.A:

P	d
12	13

 $\xleftrightarrow{25 \text{ bit}}$

$N = 2^{12} = 4K$

(iii) System has 2K pages and LA of 32 bits. What is P.S?

$N = 2K$
 $P = 11$ bits

P	d
11	21

$\xleftrightarrow{32 \text{ bits}}$

$d = 21$, $PS = 2^{21} = 2 MB$

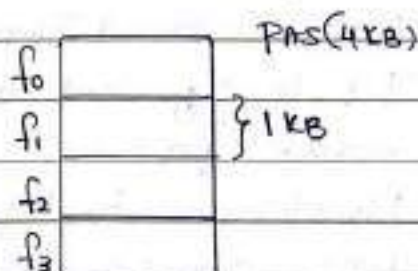
* Organization of P.A.S (4KB):

→ P.A.S is divided into equal size units known as frames [page frames].

→ Frame size = page size

→ Each frame holds one page

→ Any page can be stored in any frame (NCC)



Formulas * No. of frames (m) = $\frac{P.A.S}{PS}$

* frame no. (f) bits = $\log_2 m$
 $m = 2^f$

* frame offset = page offset = d

* P.A format:

f	d
2	10

 12 bits

Q1. L.A = 31 bits, P.A = 20 bits, P.S = 4 KB

$$N = \frac{L.A.S}{P.S} = \frac{2^{31}}{2^{12}} = 2^{19} = 512K$$

$$M = \frac{2^{20}}{2^{12}} = 2^8 = 256, \quad P = 19b, \quad f = 8bits, \quad d = 12bits$$

$$L.A: \boxed{P \mid d} \quad 19+12 = 31 \quad P.A: \boxed{f \mid d} = 20bits$$

Q2. System has 4K pages and 1K frames, Calculate the size of P.A.S, if L.A = 32 bits.

$$P = 12$$

$$N = 4K, \quad M = 1K = f = 10$$

$$L.A = 32bits$$

$$L.A: \boxed{P \mid d} \quad \leftarrow 32bits \rightarrow$$

P.A:

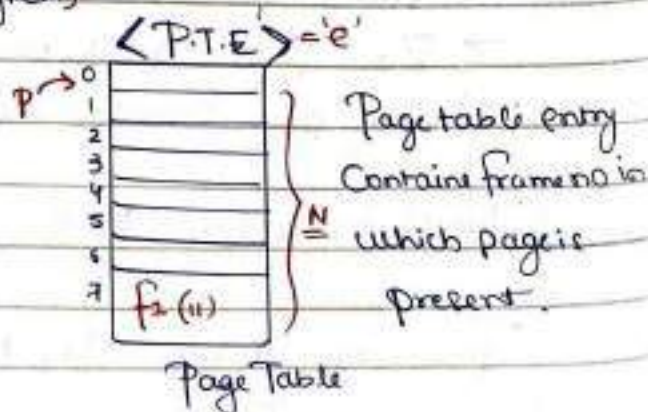
$$\boxed{f \mid d}$$

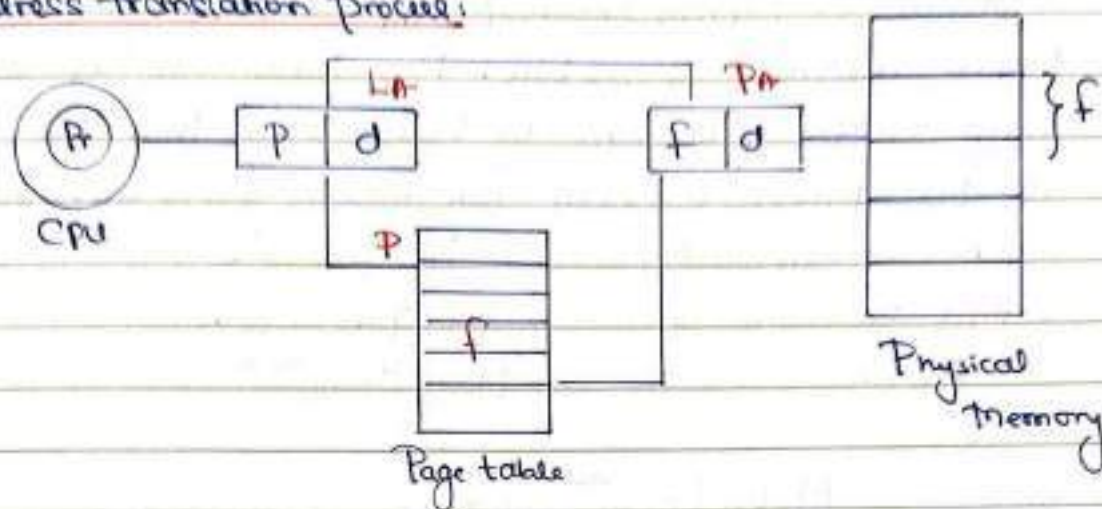
$$P.A.S = 2^{20} = 1GB$$

* Organization of M.M.U [page table/address table]

- Each process has its own P.T
- P.Ts are stored in memory
- P.T is organised as set of entries, known as "Page table entries" (PTE).
- No. of entries in Page table = No. of pages.
- P.T.E contains frame number in which page is present.
- P.T.E is denoted as 'e' (Byte)
- P.T Size = $N \times e$ (Byte)

$$e \geq 18$$



Address translation Process:

Q1. A computer system using paging technique implements on 8KB page with page table of size 24MB. The page table entry is 24 bits. What is the length of virtual address in the system?

Ans

P.S = 8KB	P.T.S = 24MB
d = 13 bits	e = 24 bit = 3B
P.T.S = N * e	
N = $\frac{P.T.S}{e} = \frac{24MB}{3B} = 8m$	
$\therefore 23 \text{ bit}$	

Q2. Consider a computer system with 40-bit virtual addressing and page size of 16KB. If the computer system has a one level page table per process and each page table entry requires 48 bits, then the size of per process page table is megabytes.

Soln

V.A = 'l' bits	P = $\log_2 2$	P.A:
N = '2'	d = (1 - $\log_2 2$) bits	($\log_2^H + 10 \cdot \log_2 2$)
m = 'H'		($1 + \log_2 H - \log_2 2$)
		P.A.S = $2^{1 + \log_2 (H/2)}$
		= $2^1 \times 2^{\log_2 (H/2)}$
		= $2^1 \times \left(\frac{H}{2}\right)$

Q3. Consider a System having/using Simple paging technique with logical address of 32 bits. Page-table entry of 32 bits. What must be the page size in bytes, such that the page table of the process exactly fits in one frame of memory (P.S.)?

Solⁿ

$$L.A = 32 \text{ bits}$$

$$\text{Let } P.S. = 2^x \text{ B}$$

$$P.T.E = e = 32 \text{ bits} = 4 \text{ B}$$

$$P.T.S = N \times e$$

$$P.S. = ?$$

$$N = \frac{2^{32}}{2^x} = 2^{32-x}$$

$$[P.T.S] = 2^{32-x} \times 4 \text{ B}$$

$$= 2^{34-x}$$

$$2^{34-x} = 2^x$$

$$34 = 2x \therefore x = 17$$

Q4. Consider a System using Paging technique with VA = 32 bits and P.S = 4 KB, what must be appropriate P.T.S in bytes, given P.A.S = 64 MB.

Ans

$$P.A.S = 64 \text{ MB}$$

$$P.T.S = N \times e$$

$$VA = 32 \text{ bits}$$

$$M = \frac{2^{26}}{2^{12}} = 2^{14}$$

$$N = \frac{2^{32}}{2^{12}} = 2^{20} = 1 \text{ M}$$

$$P.S = 4 \text{ KB}$$

$$P.T.S = 1 \text{ M} \times 2 \text{ B}$$

$$= 2 \text{ MB}$$

Performance of paging:

① Timing issue:

- * Main memory access time (M_{MAT}) = 'm'
- * Effective memory access time (E_{MAT}) = $2m$
- * Objective: Reduce E_{MAT} from $2m$ to m .

Cache memory: * faster memory access time



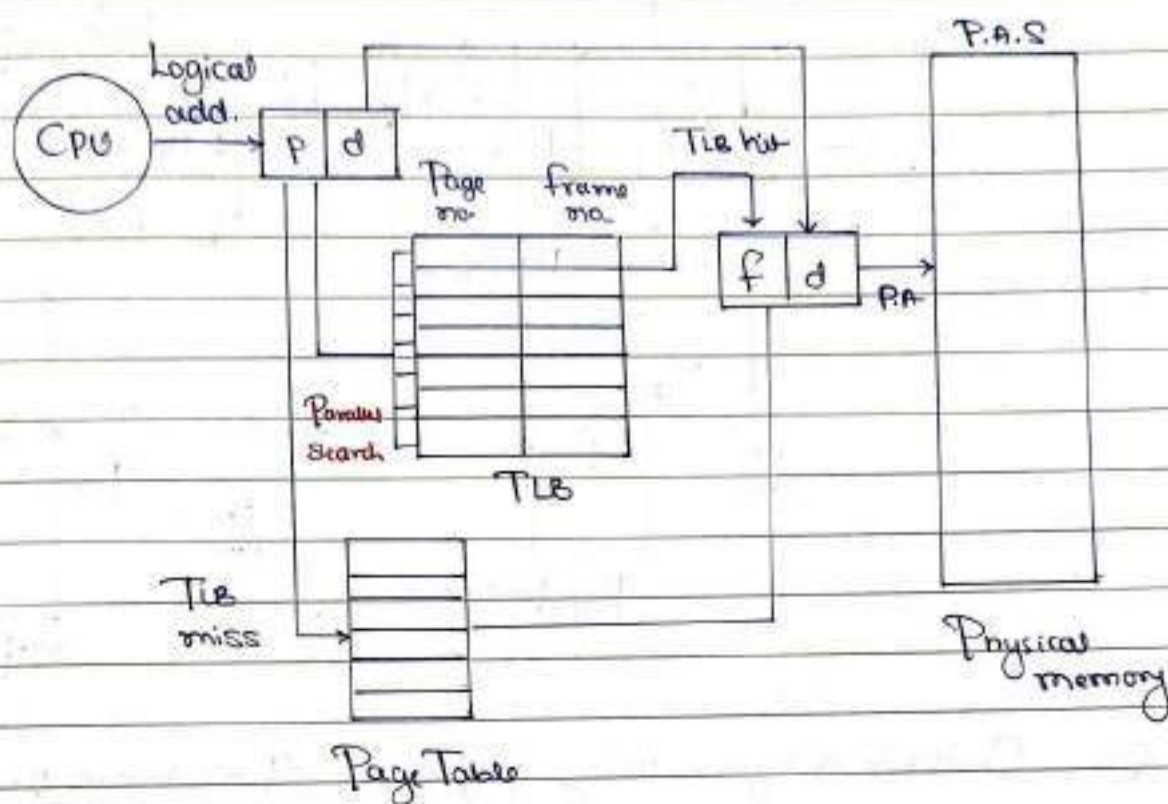
TLB

(Translation Lookaside Buffer)

* costly

* Smaller in size

* Supports parallel search.

Paging Hardware with T.L.B.Formulae

$$* \text{P.M.A.T.} = 'm'$$

$$* \text{T.L.B.A.T.} = C$$

$$* \text{E.M.A.T.} = 2m$$

$$* \text{T.L.B. hit} = 'x' \text{ ratio}$$

$$* \text{T.L.B. miss} = '1-x' \text{ ratio}$$

$$* \text{E.M.A.T.} = x(C+m) + (1-x)(C+2m)$$

Q1. $m = 100 \text{ ns}$

$C = 20 \text{ ns}$ (a) E.M.A.T. with paging: $2m = 2 \times 100 = 200 \text{ ns}$

(b) If $x = 90\%$, what is E.L.B. T.L.B.: $x = 0.9$, $(1-x) = 0.1$

$$= x(C+m) + (1-x)(C+2m)$$

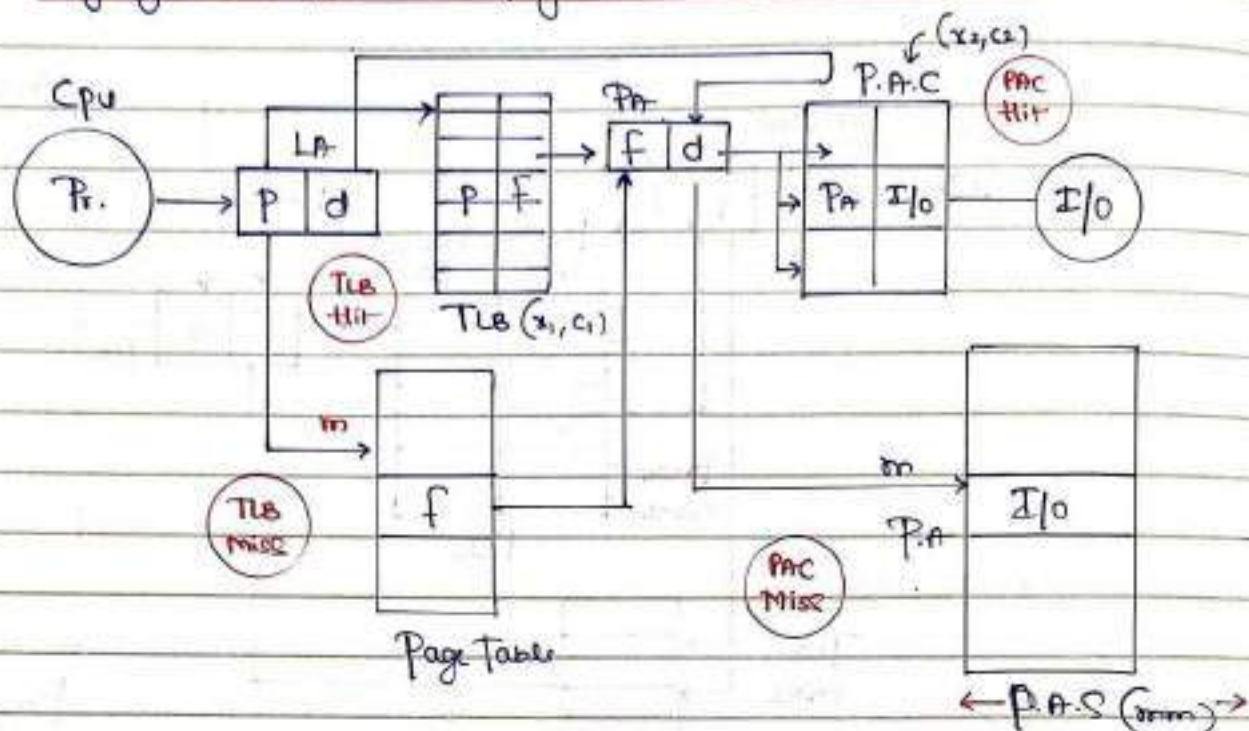
$$= 0.9(120) + 0.1(220) = 108 + 22 = 130 \text{ ns} [200 - 120]$$

(c) If $x = 10\%$, what is E.M.A.T. T.L.B.?

$$= 0.1(120) + 0.9(220)$$

$$= 210 \text{ ns} [200 - 210 \text{ ns}]$$

Paging with TLB and Physical Address Cache (PAC)



- Q2 Consider a system using paging with TLB. What hit ratio is required to reduce the EMT from 'D' to 'Z' using TLB. Assume that TLB access time is 'k' ms.

$$2m = D$$

$$m = D/2$$

$$EMT = x(C + m) + (1-x)(C + 2m)$$

$$'Z' = x \left[k + \frac{D}{2} \right] + (1-x) [k + m]$$

② Space Consumption in Paging

eg: LA = 32 bits, PS = 4KB

Appropriate P.T Size?

$$N = 2^{32} / 2^{12} = 2^{20}$$

$$P.T.S(B_y) = N * e (B_y)$$

$$P.T.S \propto N$$

if $e = 4B$

$$P.T.S = 1m \times 4B = 4MB$$

Objective: Reduce Page table size of processes.

If system has 100 processes, amount of memory required = 100×4
= 400MB.

Reduce P.T Size / Associate Smaller PTS with processes.

$$* P.T.S = N * e$$

$$P.T.S \propto N \quad (i)$$

$$* P.T.S \propto N \propto \frac{1}{PS}$$

$$* N = \frac{LAS}{PS}$$

$$P.T.S \propto \frac{1}{PS}$$

$$N \propto \frac{1}{PS} \quad (ii)$$

(2) Reduce P.T.S by increasing page size.

Optimal page size?

Let V.A.S = 'S' Bytes

Let P.T.E = 'e' Bytes

Let Page Size = 'P' Byte.

$$\text{Total overhead} = \left[\left(\frac{S}{P} \right) e + \frac{P}{2} \right] \quad (1)$$

diff eqn (1) w.r.t P = 0

$$\left[-\frac{1}{P^2} Se + \frac{1}{2} \right] = 0$$

$$(I). P.T.S = \left[\left(\frac{S}{P} \right) e \right] \text{ Bytes}$$

$$\frac{Se}{P^2} = \frac{1}{2} \Rightarrow P^2 = 2Se$$

$$(II). IF = \frac{P}{2} \text{ Bytes.}$$

$$P = \sqrt{2Se}$$

Q. A machine has a 32 bit address space and an 8KB page. The page table is entirely in hardware, with one 32-bit word per entry. When a process starts, the page table is copied to the hardware from memory, at one word every 100 nsec. If each process runs for 100 msec (including the time to load the page table), what fraction of CPU time is devoted to loading the page tables?

$$N \text{ pages} = \frac{2^{32}}{2^{13}} = 2^{19}$$

$$\text{Time to load P.T} = 2^{19} \times 100 \text{ nsec.}$$

$$= \frac{2^{19} \times 100 \times 100}{100 \times 10^6} \Rightarrow 50\%$$

$$\% \text{ fraction of CPU time} = \frac{2^{19} \times 100 \text{ nsec}}{100 \text{ msec}}$$

Q2. Consider a system using paging technique with an address space of 65,536 Bytes. The page size in this system is 4096 Bytes. The program consists of Text, Data and Stack sections as per the space:

Text: 32,768 B

Data: 16,386 B

Stack: 15,870 B

A page of the program contains portion of only one section i.e. either Text or Data or Stack

(a) Does program fit in given address space? No

(b) What is the maximum page size in Bytes such that the program fits in given address space.

$$VAS = 65,536 = 64 \text{ KB}$$

$$\text{Text} = 32,768 / 4096 = 8$$

$$N = \frac{64 \text{ KB}}{4 \text{ KB}} = \frac{2^{16}}{2^{12}} = 2^4 = 16$$

$$\text{Data} = 16,386 / 4096 = 5$$

$$\text{Stack} = 15,870 / 4096 = 4$$

$$\text{Total} = 17 \text{ So you cannot.}$$

(II) Hashed Paging / Paging with Hashing

* Element $(x) \rightarrow b(f(x)) \rightarrow i \rightarrow \text{index}$

* We will design a Page-table using hashing

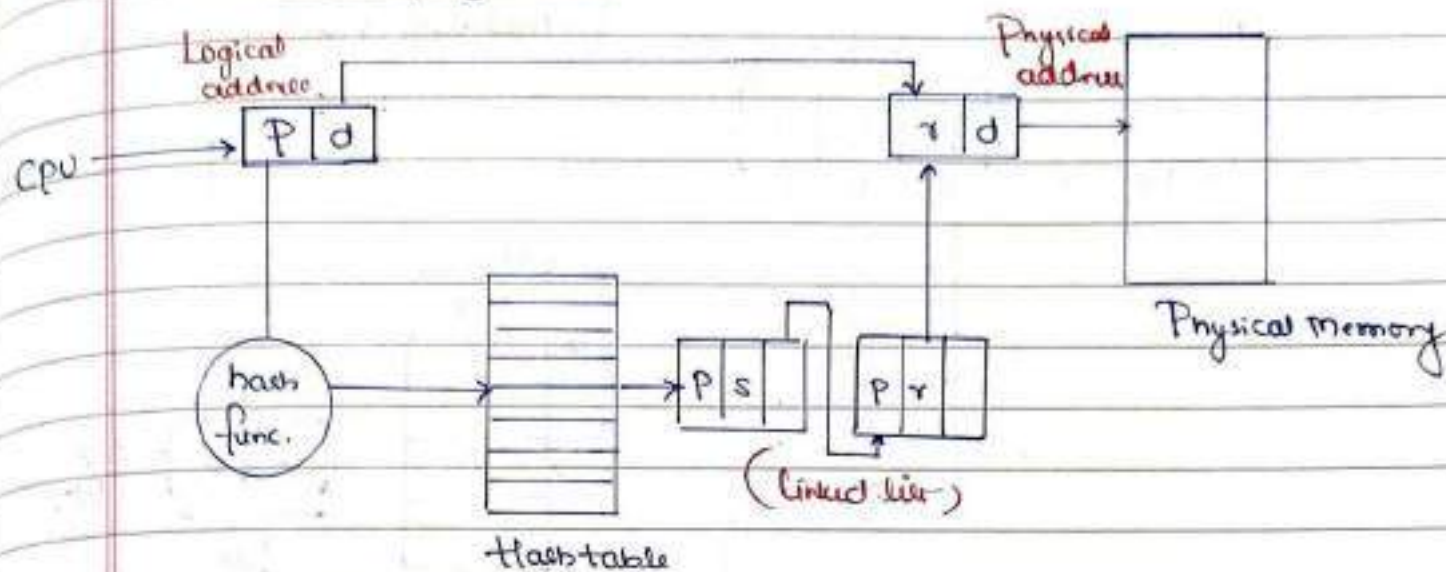
$$[x \% 10] \rightarrow \begin{matrix} 0 \\ \vdots \\ 9 \end{matrix}$$

Collision: Let I_1 and I_2 be two distinct elements
 $f(I_1) = f(I_2) = i$

* To resolve collision - Chaining is used.

* Space optimized, but time inefficient.

Hashed page table



(III) Multi Level Paging / Hierarchical Paging (Recursive paging)

Objective: To reduce Page table size overhead by associating smaller Page table's with process.

eg.: VA = 32 bit

P.S = 4 KB

e = 4B

PTS = 1MB x 4KB

= 4MB

When do we say P.T is small?

: If the page table fits in one frame of memory

Paging as a concept involves 3-steps:

1. Divide the address space into page (chunks).
2. Storing the pages in physical address space
3. Access the pages (chunks) of address space in physical address space through page table

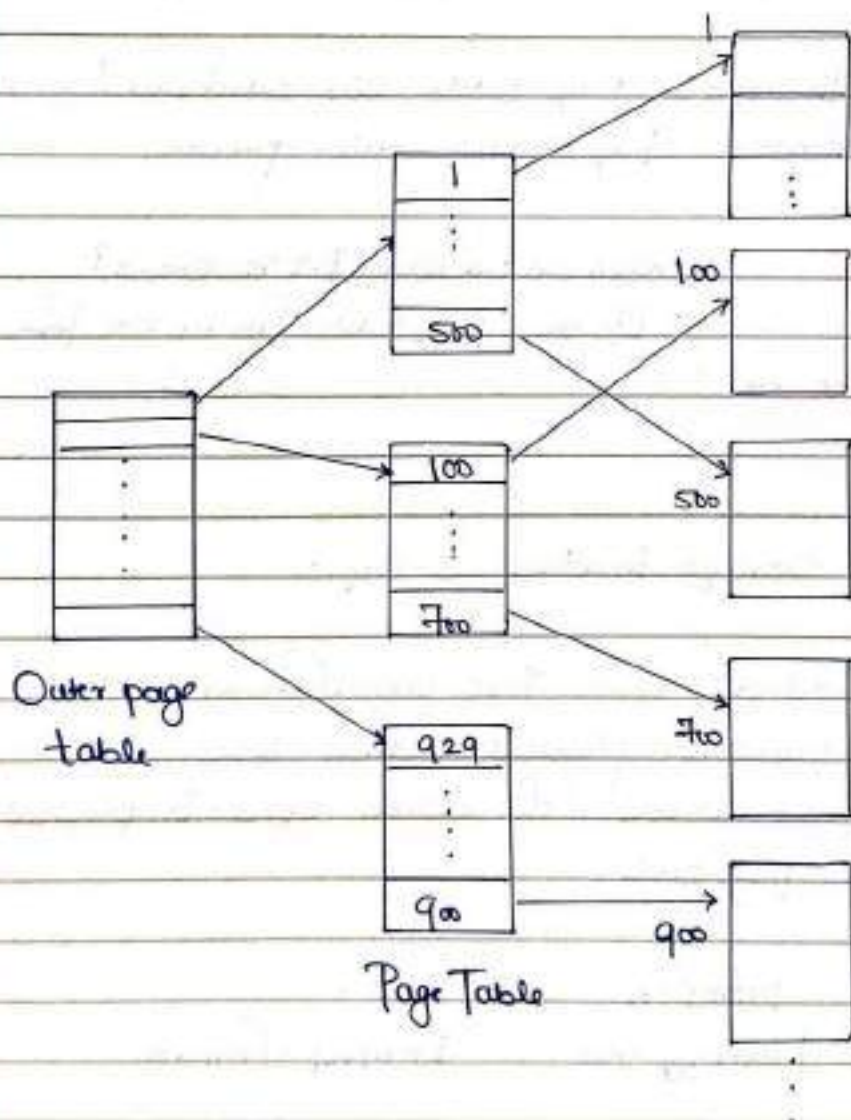
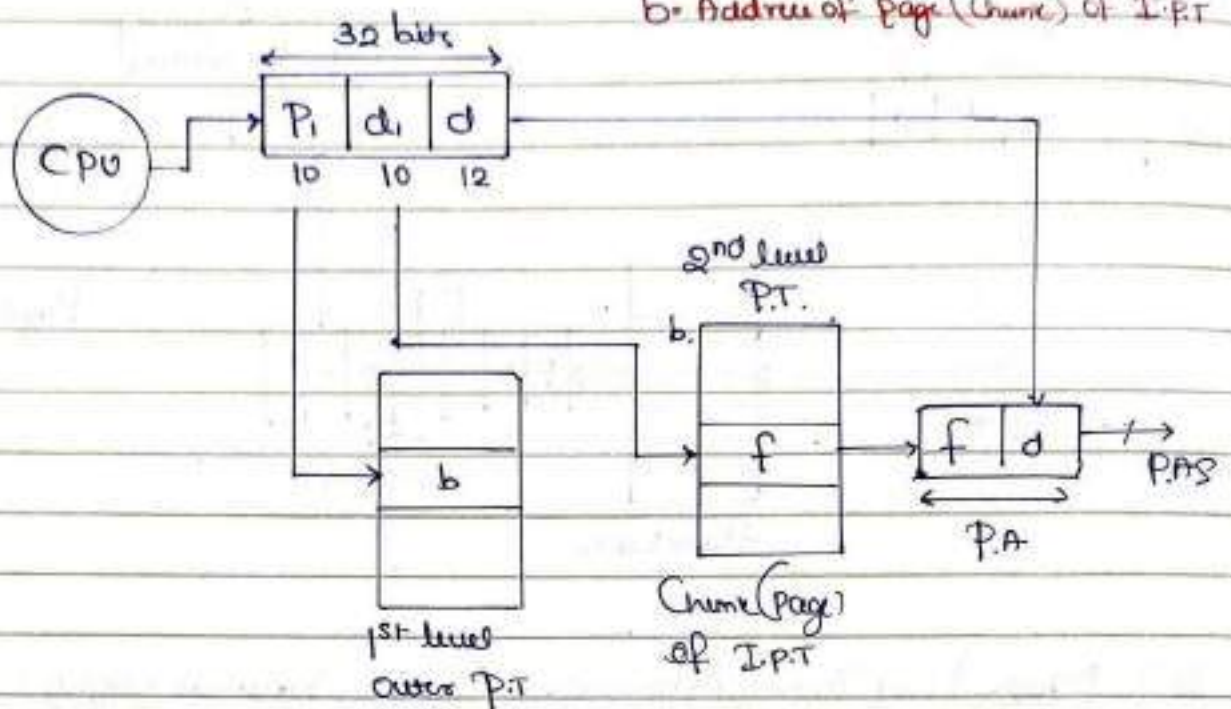
Performance: $T_{\text{MMT}} = m$

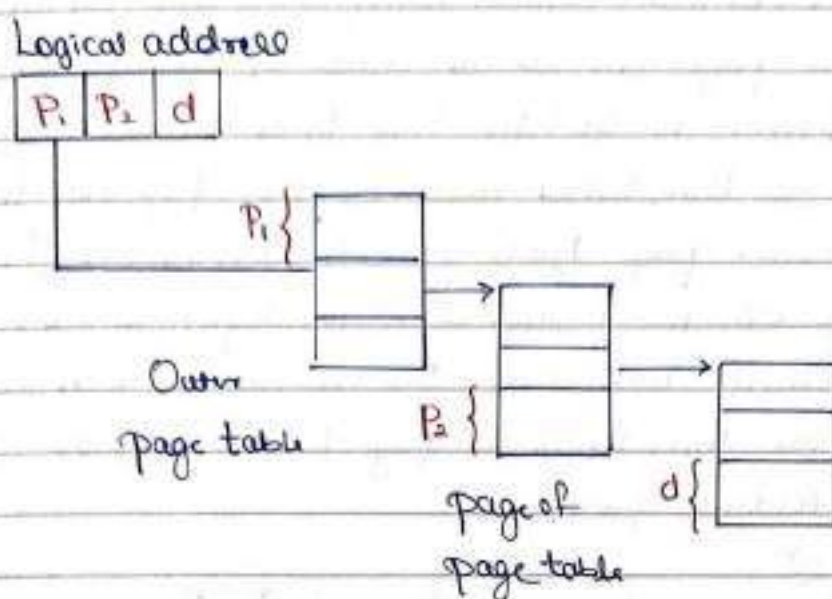
$E_{\text{MMT}} = 2m$

$E_{\text{MMT}} = (m+1) \cdot m$

eg: LA: 32 bits, P.S: 4KB.

b = Address of page (frame) of I.P.T





Q.1 In the context of operating systems, which of the following are correct w.r.t. Paging?

- * Paging helps solve the issue of external fragmentation
- * Paging incurs memory overheads.

Q.2. Consider a system using 2 level paging Architecture. The top level 9 bits of the Virtual Address are used to index into the outer Page Table. The next 7 bits of the address are used to index into next level page table. If the size of Virtual Address is 28 bits. Then

(i) How large are the pages and how many are there in virtual address space?

$$P \sim \text{No of page in VAS}, \quad N = 2^{16} = 2^6 \times 2^{10} = \underline{64 \text{ KB}}$$

$$P.S \text{ of VAS} = 2^{12} = \underline{4 \text{ KB}}$$

(ii) If P.T.E at both levels is 32 bit in size then what is the space overhead needed to translate Virtual Address to Physical Address of an instruction or Data unit?

$$P_1 \quad P_2 \quad d \quad 2 \text{ KB} + 512 \text{ B}$$

$$9 \quad 7 \quad 12 \quad \underline{25 \text{ KB}}$$

- Q3. Consider a Computer System using three level paging Architecture with uniform page size at all levels of paging. The size of virtual address is 46 bits. Page Table Entries at all levels of paging is 32 bits. What must be the Page size in Bytes such that the outer page table exactly fits in one frame of memory. Assume page size is power of 2 in Bytes. Show the virtual address format indicating the number of bits required to process all the three levels of page tables and the Page offset of Virtual Address Space.

$$VAS = 2^{46}$$

$$\text{Let } P.S = 2^x \text{ B} \rightarrow \left(\frac{2^{46}}{2^x} \right) = \left[2^{46-x+12} \right] \text{ B} \rightarrow \left[\frac{2^{46-x+12}}{2^x} \right] \text{ B}$$

$$\Rightarrow \left[2^{46-2x+2+2+2} \right] \text{ B}$$

$$x=13$$

$$\therefore 2^{13} = \underline{\underline{8 \text{ KB}}}$$

Derivation:

$$\text{Let } VAS = 2^S \text{ Bytes } VA = S \text{ bits}$$

$$P.S = 2^x \text{ Bytes, } P.T.E = 2^C \text{ Bytes}$$

levels of paging = 'l'

$$\rightarrow \text{Size of outer level PT} = \left[2^{S-l \cdot x + l \cdot C} \right] \text{ Bytes}$$

$$\rightarrow \frac{2^S}{2^x} \cdot 2^C = \left[2^{S-lx+C} \right]$$

- Q4. Consider a computer system with 57 bit Virtual Addressing, using multi level tree structured page tables with 1 level for Virtual to Physical address translation. Page size is 4KB and Page Table Entry is of 8 Bytes at all level. The value of L is 5

VA = 57 bit

P.S = 4 KB

OPT = 1 Page / Frame

No. of levels = '1'

P.T.E = 8B

OPT = 4 KB

$$OPT = \left[\frac{57 - 1 \cdot 12 + 1}{2} \right] \text{ Bytes}$$

$$= 2^{12} \text{ B}$$

$$57 - 91 = 12$$

$$1 - 5$$

HW

Q5.

Consider a three level page table to translate a 39-bit virtual address to a physical address as shown below.

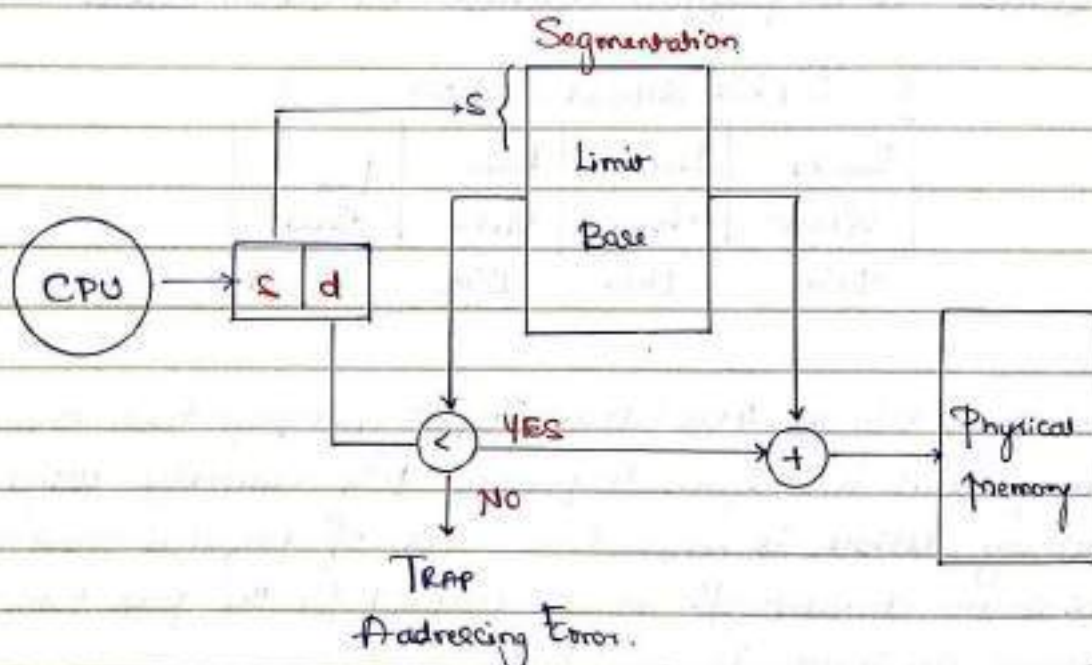
← 39 bit virtual address →

Level 1	Level 2	Level 3	Page
offset	offset	offset	offset
9 bit	9 bit	9 bit	12 bit

The page size is 4 KB ($1 \text{ KB} = 2^{10} \text{ Bytes}$) and page table entry size at every level is 8 bytes. A process P is currently using 2 KB virtual memory which is mapped to 2 KB of physical memory. The minimum amount of memory required for the page table of P across all levels is — KB.

2. Segmentation

- * Paging does not pressure user's view of memory allocation to programs
- * As per user's view of memory allocation, Program is divided into logical units, known as segments (function, block, procedure, D.S, class).
- * These segments are assumed to be stored in their entirety entirely at non contiguous locations.

HW

Q1. Consider the following segment table:

Segment	Base	Length
0	1219	600
1	3300	14
2	90	100
3	2327	580
4	1952	96

What are the physical addresses for the following logical addresses?

Performance of Segmentation1. Time : $M_{MAT} = 'm'$ $E_{MAT} = '2m'$ $T_{LBAT} = C$ $T_{LB} \text{ Hit ratio} = \alpha$ $T_{LB} \text{ Miss ratio} = 1-\alpha$

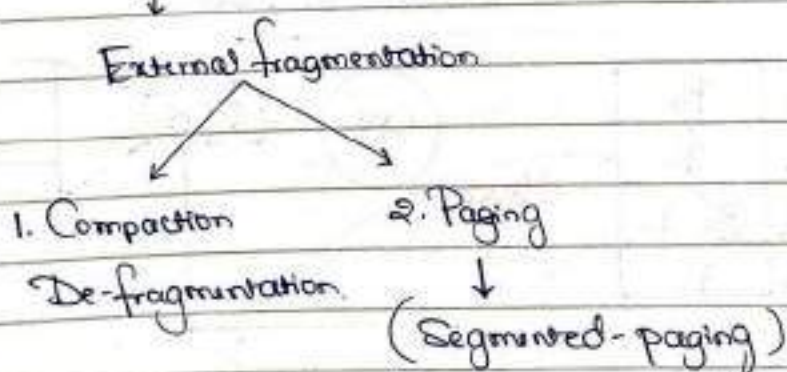
$$E_{MAT} = \alpha(C+m) + (1-\alpha)(C+2m)$$

2. Space :

When Segment table become large, apply paging on Segment table

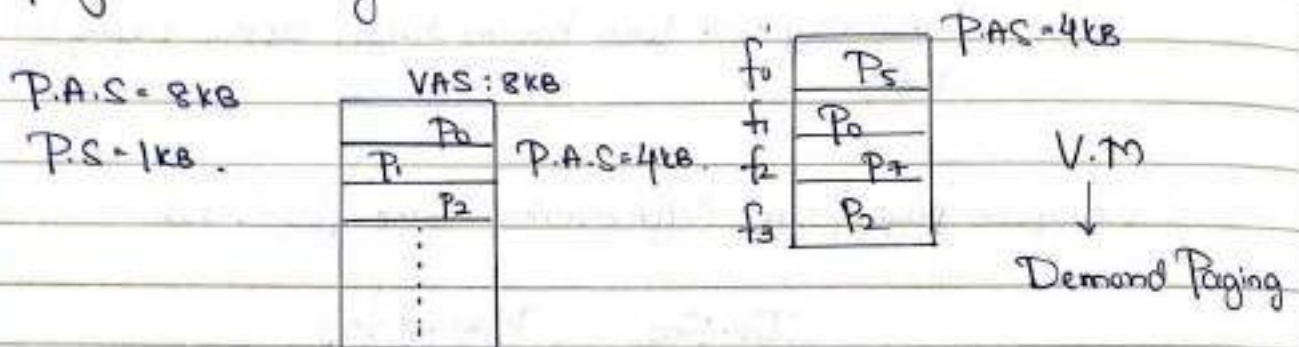
Compare paging and Segmentation w.r.t fragmentation

	Int. frag.	External frag.
Paging	✓ <last page>	X
Segment	X	✓ in P.A.S.

Segmentation

Virtual Memory (vm)

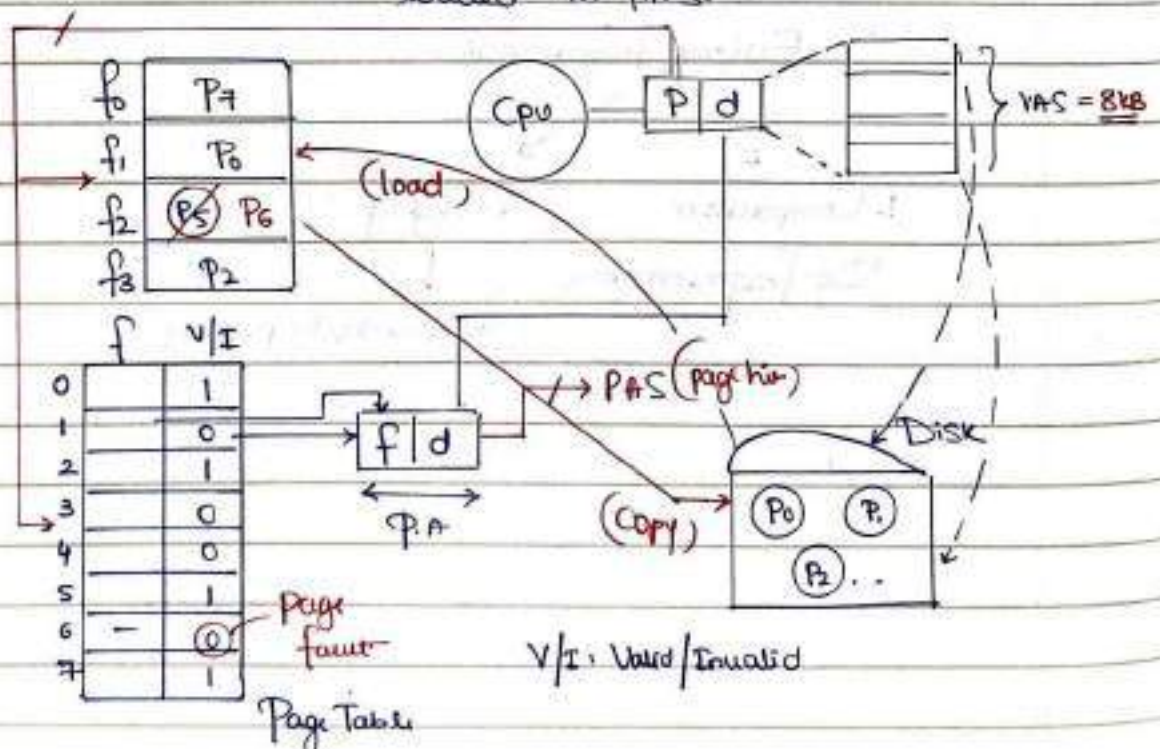
Virtual memory gives an illusion to the programmer that a huge/large amount of memory is available for executing programs, that are larger than the available/given physical memory



Demand paging: * Loading the page from disk to memory on demand at runtime.

* Virtual memory is implemented through "Demand Paging".

* Program (8KB/8 Pages): stored initially on disk, required pages are loaded in P.A.S.



Demand paging

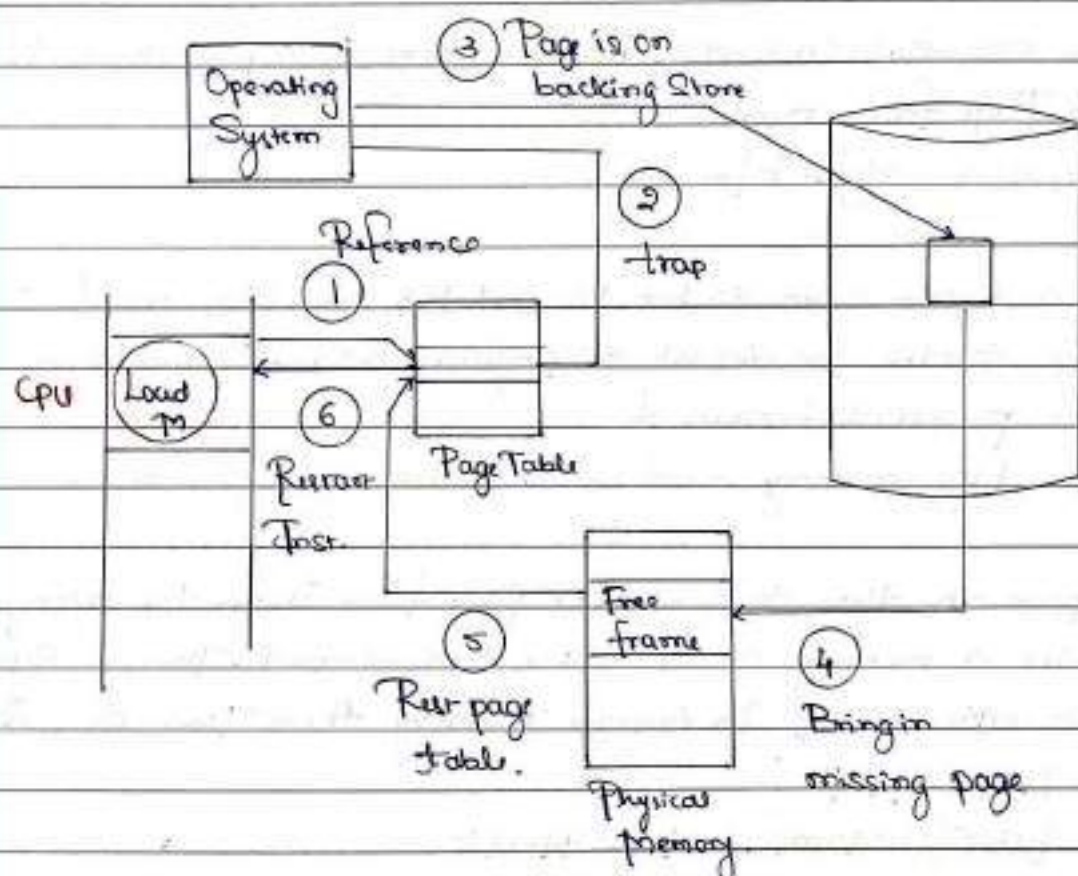
Pure DP

Pre Fetched D.P

< Execution of the
Process will start
with empty frame >

Note: * Size of virtual memory is
limited by size of disk.

* $P.A.S \leq V.A.S \leq D.A.S$

Steps taken to service page fault:

* Amount of time taken by O.S (VMM) to service a page fault is known as "Page fault service time." (ms)

Q1. In a virtual memory system the address space specified by the address line of the CPU must be Larger than the physical memory size and Smaller Secondary Storage size.

Performance of Virtual Memory:

1. Time Issue (Temporal)

$$M.M.A.T = 'm' [ns | \mu s]$$

$$PFST = 's' [s \gg m]$$

$$\text{Page fault rate} = 'P'$$

$$\text{Page hit rate} = '1-P'$$

$$EMAT = (1-P) \cdot m + P \cdot s$$

$$D.P$$

Q2 The total size of address space in virtual memory system is limited by:

Ans. Size of Secondary Storage

Q3 The essential content(s) in each entry of a page table is/are
Ans (i) Page frame number
(ii) Access right information

Q4 In a system with 32 bit VA and 1KB page size, use of one level page tables for virtual to physical address translation is not practical because of

Ans the large memory overhead in maintaining page tables

Q5 Suppose the time to service a page fault is on the average 10 ms, while a memory access takes 1 microsecond. Then a 99.99% hit ratio results in Average Memory Access Time of 2 μ s

Ans

$$M.M.A.T = 1 \mu s$$

$$PFST (s) = 10 ms$$

$$1-P = 99.99\%$$

$$= (0.9999)$$

$$P = 0.01\%$$

$$= 0.0001$$

$$EMAT = (1-P) \cdot m + P \cdot s$$

$$= 0.9999 \times 1 \times 10^{-6} + 0.0001 \times 10 \times 10^{-3}$$

$$= 0.9999 \times 1 \times 10^{-6} + 1 \times 10^{-6}$$

$$= 1.9999 \text{ ms } \mu s \sim \underline{2 \mu s}$$

Q6. If an instruction takes ' i ' microseconds and a Page fault takes an additional ' j ' microseconds, the Effective Instruction Time if on the average a page fault occurs every ' k ' instructions is _____

Ans. Instruction time without Page fault: ' i ' μ s
Instruction time with Page fault: ' $i+j$ ' μ s

$$P = 1/k$$

$$(1-P) = \left(1 - \frac{1}{k}\right)$$

$$\text{Effective Inst. Time} = \frac{1}{k}(i+j) + \left(1 - \frac{1}{k}\right)i$$

$$= \cancel{\frac{i}{k}} + \frac{j}{k} + \cancel{i} - \cancel{\frac{i}{k}}$$

$$= \underline{i + j/k}$$

HW

Q7. Assume that we have a Demand-paged memory. It takes 8 milli-seconds to service a page fault if an empty frame is available or if the rejected page is not modified, and 20 milliseconds if the replaced page is modified. Memory-access time is 100 nano sec. Assume that the page to be replaced is modified 70 percent of the time. What is the acceptable page fault rate for an effective access time of no more than 2000 nanoseconds?

Ans. $S \xrightarrow{30\%} 8(\text{ms}) : \text{EF + Clean page [70\%]}$

$\searrow 20(\text{ms}) : \text{Dirty page [30\%]}$

$$m = 100 \text{ ns}$$

$$P = ?$$

$$E_{\text{MAT}} = 2000 \text{ ns}$$

$$2000 \text{ ns} = (1-P) \cdot 100 \text{ ns} + P (0.7 \cdot 20 \text{ ms} + 0.3 \cdot 8 \text{ ms})$$

$$P =$$

Q8. Consider a process executing on an Operating System that uses demand paging. The average time for a memory access in the system is m units if the corresponding memory page is available in memory, and D units if the memory access caused a page fault. It has been experimentally measured that the average time taken for a memory access in the process is X units. Which one of the following is correct expression for the page fault rate experienced by the process?

Ans

$$m = m \quad X = (1-P)m + P \cdot D$$

$$S = D \quad = P = \frac{(X-m)}{(D-m)}$$

Q9. Consider a paging system that uses 1-level page table residing in main memory and a TLB for address translation. Each main memory access takes 100 ns and TLB lookup takes 20 ns. Each page transfer to/from the disk takes 5000 ns. Assume that the TLB hit ratio is 95%, page fault rate is 10%. Assume that for the 20% of the total page faults, a dirty page has to be written back to disk before the required page is read in from disk. The update time is negligible. The average memory access time in ns (round off to 1 decimal place) is 154.5

Ans

$$m = 100 \text{ ns} \quad \text{Disk R/W} = 5000 \text{ ns}$$

$$C = 20 \text{ ns} \quad \text{TLB hit rate} = 95\%$$

$$\text{Page fault rate} = 10\%$$

80% - Clean page
20% - Dirty page

TLB hit
TLB miss

$$EMAT = 0.95 [20 \text{ ns} + 100 \text{ ns}] + 0.05 [20 \text{ ns} + 0.9 (100 \text{ ns} + 100 \text{ ns}) + 0.1 \text{ PF} (100 \text{ ns} + 0.2 \times (5000 + 5000 \text{ ns}) + 0.8 (500 \text{ ns}))]$$

page hit
Clean page

$$EMAT = 154.54 \approx 155$$

Q10. The minimum number of page frames that must be allocated to a running process in a virtual memory environment is determined by Instruction Structure Architecture.

Ans

Page Replacement: One important concept of Virtual Memory implement.

(a) Reference String (Page-Ref string): Set of successively unique pages (Page Nos.) referred in given list of V.A's.

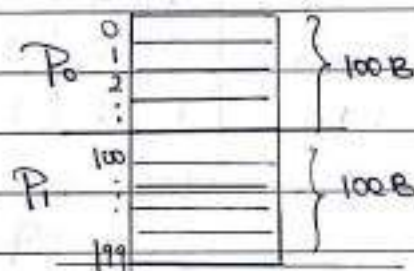
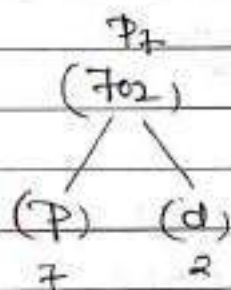
eg:: $\langle 7, 1, 6, 4, 0, 1, 7, 9, 6, 7, \dots \rangle$
 $\text{Pri} \langle 702, 704; 123; 654; 483; 012; 122; 124; 180; 785; 934; \dots \rangle$

Let P.S = 100B

$$P = \frac{VA}{PS} \quad d = VA \cdot \frac{1}{PS}$$

Length = 10

Unique pages = 6



$\langle 0 \dots 99 \rangle : P_0$
 $\langle 100 \dots 199 \rangle : P_1$
 $\langle 200 \dots 299 \rangle : P_2$

(b) Frame allocation Policies

* n = no. of processes ($P_1 \dots P_m$)

* S_i = Demand (frames) of process P_i

* D = Total Demand
 $= \sum_{i=1}^n S_i$

* m = Available frames

* a_i = Allocated frames

$n=5 \quad \langle P_1 \dots P_5 \rangle; \quad m=40$

P_i	Demand S_i	Eq. alloc. $a_i = m/n$	Prop. alloc. $a_i = \lceil S_i / d \rceil$	$a_i \cdot S_i / 2$
P_1	10	8	5	
P_2	5	8	3	
P_3	35	8	17	
P_4	18	8	9	
P_5	12	8	6	

D. 50

* Min. no of frames to be allocated to a process:

→ < Process cannot execute without this minimum number of frames >

→ Process should be able to execute minimum one instruction.

Page Replacement Techniques:

1. Ref string: $\langle 7; 0; 1; 2; 0; 3; 0; 4; 2; 3; 0; 3; 2; 1; 2; 0; 1; 7; 0; 1 \rangle$
length = 20, $m = 6$

(1) FIFO

	7	2	2	2	4	4	4	0	0	0	7	7	7
Criteria	0	0	3	3	3	2	2	2	1	1	1	0	0
: Tol	1	1	1	0	0	0	3	3	3	2	2	2	1

3 Frames: 15

- frames -

$$P = \frac{15^3}{20^4} = 75.10$$

eg::? Ref string: $\langle 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 \rangle$

Fifo: pure Demand paging

With 3 Frames:

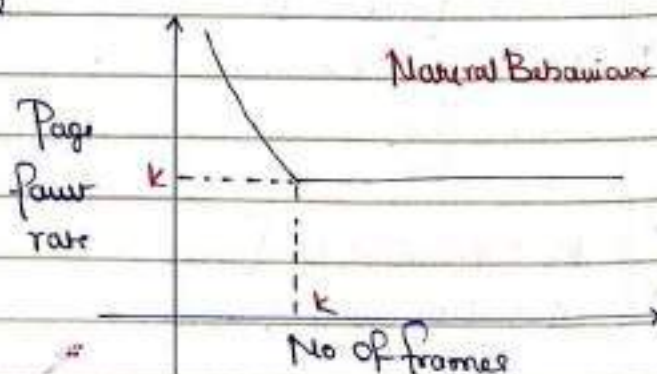
3 frames: 9

4 frame: 10

(tlw)

1	4	4	1	5	5	5
2	2	1	1	1	3	3
3	3	3	2	2	2	4

Frame



Belady's Anomaly :

As the number of frames allocated to the process increases, Page fault also Sometimes increases.

Only FIFO and LRU based suffers from Belady's Anomaly.

- ② Optimal Replacement : replace that page which will not be used for the longest duration of time in future reference.

eg:: Ref string $\langle 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 \rangle$

3 Frames : 9	7	2	2	2	2	7	7
4 Frames : ?	0	0	0	4	0	0	0
(Hw)	1	1	3	3	3	1	1
	Frames.						

* We use this algorithm as a benchmark to check performance of other replacement strategies. Optimal replacement is not implemented practically.

- ③ Least Recently used (LRU) : replace that page which has not been used for the longest duration of time in the past.

Criteria : Time of Reference.

eg:: $\langle 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 7, 7, 0, 1 \rangle$

3 Frames : 12	7	2	2	4	4	7	0	1	1	1
4 Frames : ?	0	0	0	0	0	3	3	3	0	0
(Hw)	1	1	3	3	2	2	2	2	2	7

- ④ Most Recently used (MRU) : Symmetrical opposite of least recently used, replace that page which has been used for longest duration of time.

HWeg:: $\langle 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 \rangle$

3 Frames:

4 Frames:

⑤ Counting Algorithms:

a). L.F.U [least frequently used]

b). M.F.U [most frequently used]

* Criteria: Count of reference

eg:: $\langle 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 \rangle$

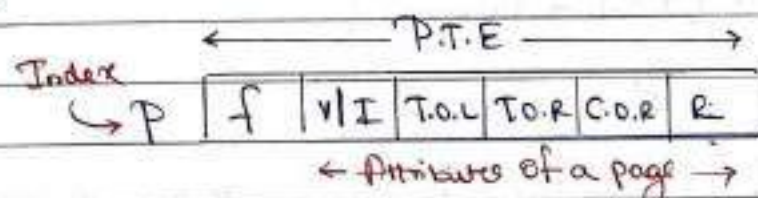
(HW: Solve with 3 frames (both)!)

Note: Among all the algorithms, Optimal is having least page fault rate.

- * LRU ~ Optimal
- * Many OS implement either LRU / LRU approximately
- * The best method to implement LRU is by "stack method".

LRU Approximations < These algorithms are not really LRU, but they approximate to the behaviour of LRU >.

They are based on a concept called < Reference bit (R) >: Each page in the page table will be associated with a reference bit 'R'.



a. > Reference bit (R):

Criteria: 'R' <

- 0 : Page is not referred so far during present Epoch.
- 1 : Page has been referred atleast once during present Epoch.

eg::

	P	f	V/I	T.O.L	R
	①	a	1	2	1
	1	e	1	3	0
	2	d	1	0	1
P.F	③	-	0	-	-
	4	c	1	4	10
	⑤	b	1	1	0

Reference bit fails when all R value are '1'.

Page Table

b. > Additional Reference bits: Each page is associated with more than one reference bits.

	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8 ↓ Current Epoch
P_i	1	0	0	1	0	1	1	1
P_j	0	0	1	1	1	0	1	1
P_k	0	0	1	1	0	1	1	1

(P_i) $\times$: Cause Page Fault
When current Epoch gets over

↓
[Shift left operations]

c). Second Chance [clock algorithm]

Criteria : [T.O.L + Reference]

eg.:

← P.T.E →

P	V/E	To.L	R	f
0	1	2	1	e
1	1	3	0	a
2	1	0	$\times^0$	b
3	0	-	-	-
4	1	1	$0^{\checkmark}$	c
5	1	4	1	d

When all pages 'R' value is '1' then FIFO page gets selected

* Second ~ FIFO
Chance

Page Table

↓
Belady's Anomaly

d). Enhanced Second Chance / NRU (Not recently used).

Criteria: Reference + modified

R_m

I. 00 → Not referenced and not modified

II. 01 → Not referenced but modified

III. 10 → Referenced but clean

IV. 11 → Referenced and modified

eg.:

← P.T.E. Structure →

P	f	V/I	Tol	R	m
0	c	1	4	1	1
1	a	1	3	0	0
2	-	0	-	-	-
3	b	1	0	1	1
4	d	1	2	1	0
5	e	1	1	0	1

(i) FIFO: P_0 (ii) R: P_1 (iii) Second Chance: P_5 (iv) Enhanced Second
Chance: P_1

Q1. Consider a system with V.A.S = P.A.S = 2^{16} Bytes. Page size is 512 Bytes. The size of the page table entry is 32 bits. If the page table entry contains beside information 1 V/I bit, 1 Reference, 1 modified bit, 3 bits for page protection. How many bits can be assigned for storing other attributes of the page. Also compute page table size in Bytes?

Ans:

← P.P.S = 32 bits →

P	f	V/I	R	m	PR.	Other
	1	1	1	1	3	2

$$n_{\text{frames}} = \frac{2^{16}}{2^9} = 2^7$$

$$\begin{aligned} \text{P.T.S} &= N \times e \\ &= 2^7 \times 2^9 = 2^{16} \rightarrow 512 \text{B} \end{aligned}$$

Q2. Consider a virtual memory system with FIFO page replacement policy, for an arbitrary page access pattern, increasing the number of page frames in main memory will.

Ans. Sometimes increases the no. of page faults (Belady's Anomaly)

Q3. A memory page containing a heavily used variable that was initialised very early and is in constant use is removed when

Ans. FIFO page replacement algorithm is used.

Q4. Recall that Belady's anomaly is that the page fault rate may increase as the number of allocated frames increases. Now consider the following statements:

S1: Random page replacement algorithm (where a page chosen at random is replaced) suffers from Belady's anomaly.

S2: LRU page replacement algorithm suffers from Belady's anomaly.

Ans: S1 is true and S2 is false.

Q5. Consider a process having reference string of length 'L' in which n unique pages occur. 'Z' frames are allocated to the process. Calculate the lower bound and upper bound of the no. of page faults.

a) maximum = 'L' [$Z=1$]
(Upper bound)

b) minimum = 'n' [$Z \geq n$] - if we use pure demand paging
(Lower bound)

= 0 - if we use pre-fetched demand paging

Thrashing: Excessive / high paging activity (high page fault rate)

↓
(The act of causing a page fault leading to loading and saving the pages on disc.)

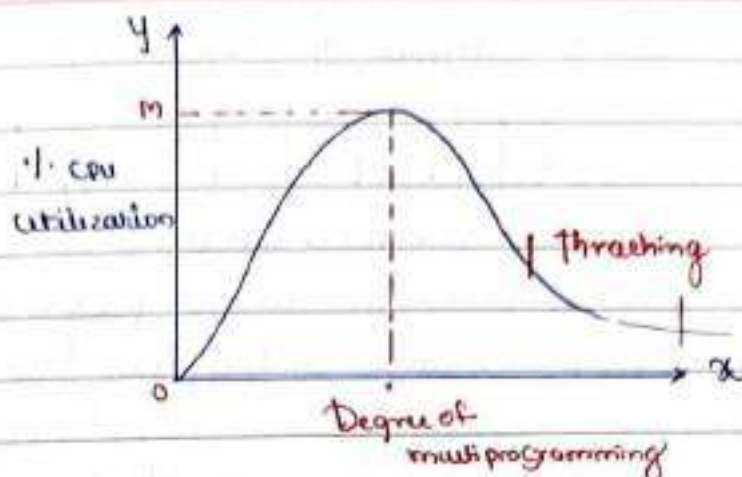
* Thrashing like deadlock is also an undesirable feature of OS.

Reasons for Thrashing: (Main)

1. High degree of multiprogramming
2. Frame allocation declines (lack of memory)

Other Reasons:

1. Page replacement policy

2. Page size:
large (less pages)↓
Less page fault3. Programming techniques
and data structureThrashing Control Strategies

1) Prevention

- * Thrashing never occur.
- * By controlling degree of multi-programming, thrashing can be controlled.

(Long term Scheduler)

2) Detection and Recovery.

- * Thrashing occurs
- * Thrashing can be detected when there is low CPU utilization.
- * Max no. of processes getting blocked
- * High degree of multi-programming
- * High disk utilization
- * Recovery: Process suspension

(Medium Term Scheduler)

Case I: Integer $A[1...128][1...128]$; P.S = 128 W; Row major order

1. for $i \leftarrow 1$ to 128
 for $j \leftarrow 1$ to 128
 $A[j, i] = 1$;

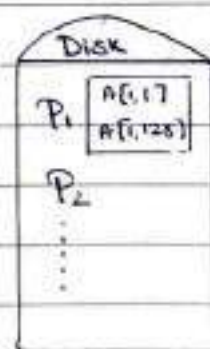
2. for $i \leftarrow 1$ to 128
 for $j \leftarrow 1$ to 128
 $A[i, j] = 1$;

No. of page
faults:

- a) Pure DP: 128^2
b) FIFO: 128

Locality
of
Reference

$$128 \times 128 = 2^{14} = 16K$$



Ans Array, because of less page fault.

Q1. Which is better for demand page environment?

Ans Linear Search may generate less page fault, (locality of reference).

Conclusion: A programming technique or a data structure is said to be good in demand page environment, if it satisfies locality model.

Working Set Strategy (model): To minimise page fault rate and also utilise memory effectively.

< Principle of locality of reference >

↑ 10 KB	↑ 2 KB	↑ 28 KB	↑ 15 KB	
<code>main()</code>	<code>f()</code>	<code>g()</code>	<code>h()</code>	Prog. Size = 55 KB Page Size = 1 KB N. of pages = 55
{ : : : : : : : : }	{ : : : <code>g();</code> : : : } :	{ : : : : <code>h();</code> : : : }	{ : : : : : <code>scanf();</code> : : : }	
				SOT Rule → 26 frames.

The basic idea of working Set model is to estimate the size of locality in which the program is executing and only ask for those many frames.

W.S.S follows dynamic type memory allocation (allocation of frames)

Working set window (WSW_i): Set of unique pages referred in the reference string during the past Δ references.

eg: $P_{ri} = \langle 5, 4, 8, 2, 9, 12, 54, 58, 56, 53, 51, 52, 58, 56, 57, 54 \rangle$

$\Delta = 10$
 (Sliding window)

$WSW_i = \{ 51, 52, 53, 54, 56, 57, 58 \}$
 $\Delta = 10$

$\Delta = \text{integer (value)}$

$WSW_i = 7$
 $\Delta = 10$

→ Let, No. of processes = 'n' $\langle P_1, \dots, P_n \rangle$

→ Total available frames = m

→ Demand of each = $S_i = WSW_i^+$
 $\Delta = 10$

→ Process for frames

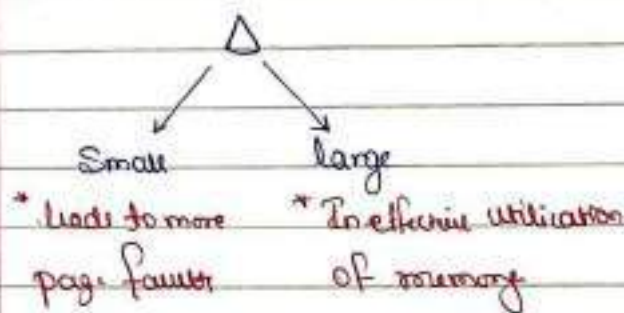
→ Total demand @ 't' = $\sum_{i=1}^n S_i = D$

I. $D \geq m$ [no thrashing]

II. $D < m$ [no thrashing]

↓
The degree of
multi-programming

III. $D > m$ [system is thrashing]



Q1. Let the page reference and the Delta (Δ) be "c c d b c e c e a d" and 4 respectively. The initial working set at time $t=0$ contains the pages {a, d, e, c} where 'a' was referenced at time $t=0$, 'd' was referenced at time $t=-1$, and 'e' was referenced at time $t=-2$. Determine the total number of page faults and the average number of page frames used by computing the working set at each reference.

$t < -2 \quad -1 \quad 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10 >$
 $e \quad d \quad a \quad c \quad c \quad d \quad b \quad c \quad e \quad c \quad e \quad a \quad d$

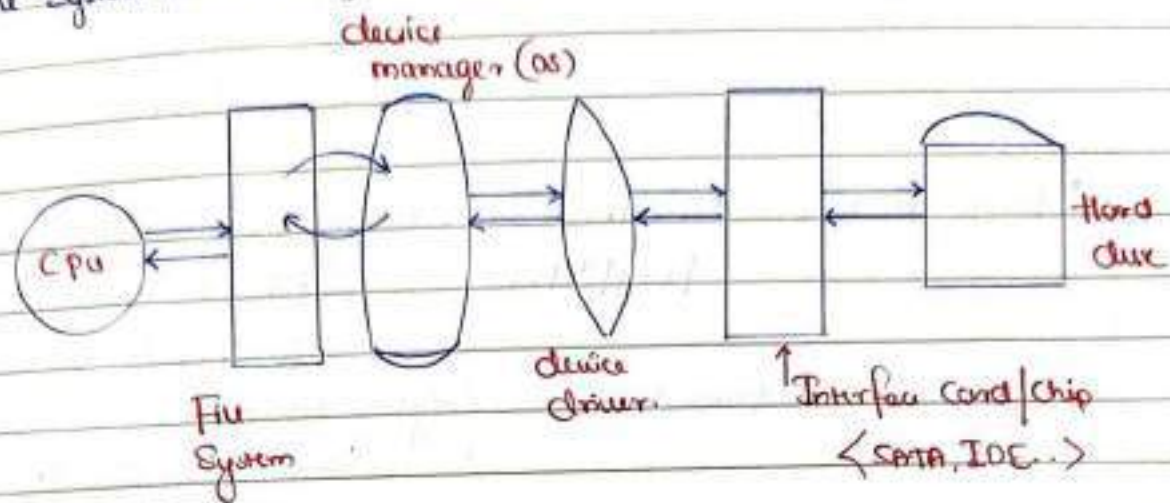
$\Delta = 4$ $t_0 = \langle e d a \rangle$ $t_1 = \langle e d a c \rangle : 4$ $t_2 = \langle d a c \rangle : 3$ $t_3 = \langle a c d \rangle : 3$ $t_5 = \langle c d b \rangle : 3$ $t_6 = \langle d b c \rangle : 3$ $t_6 = \langle d b c e \rangle : 4$ $t_7 = \langle b e c \rangle : 3$ $t_8 = \langle c, e \rangle : 2$ $t_9 = \langle c e a \rangle : 3$ $t_{10} = \langle c e a d \rangle : 4$

No. of page faults = 5

Avg. WSR = $\frac{7}{10}$

File System and Device Management

* File System is the only visible part of the operating system.

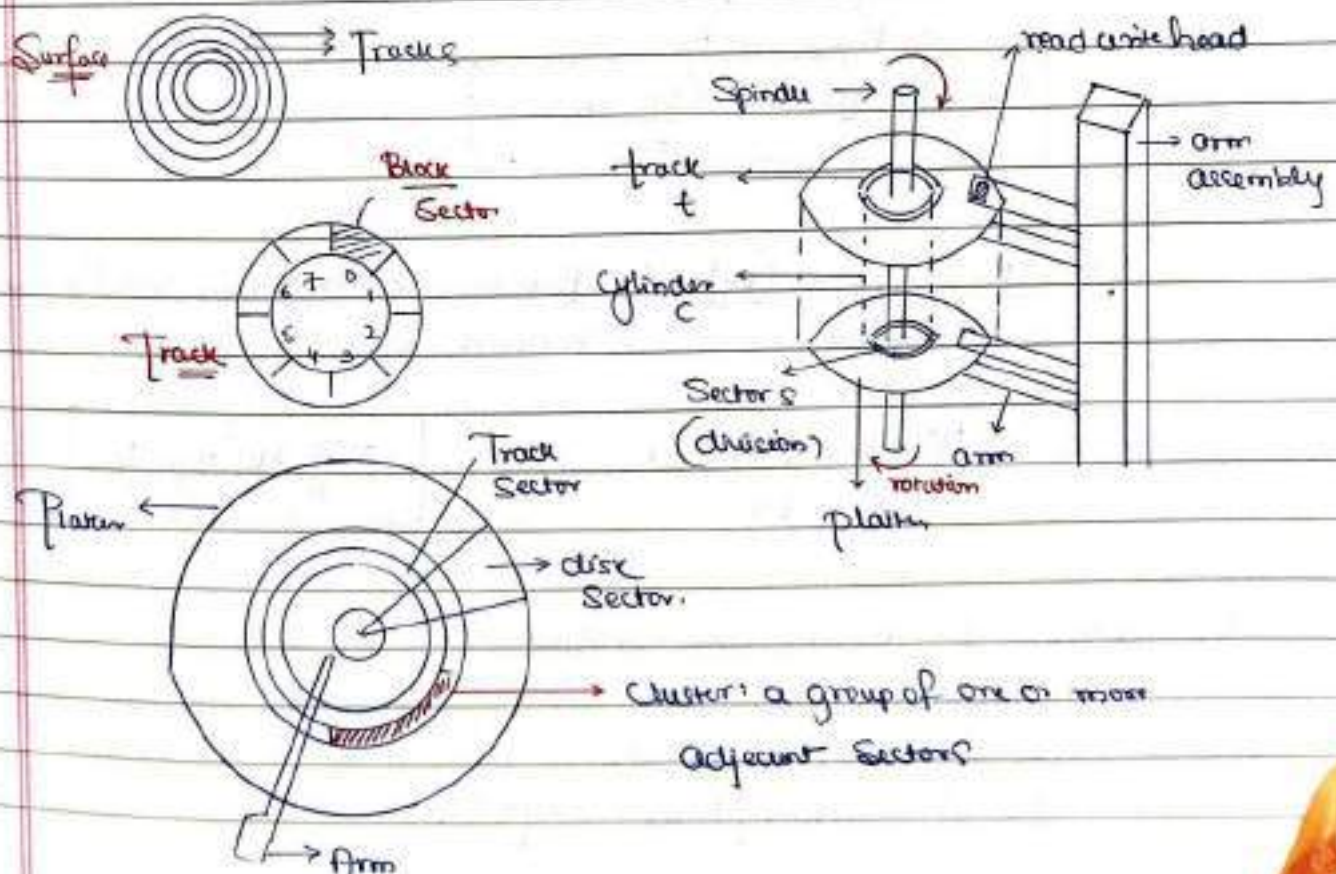


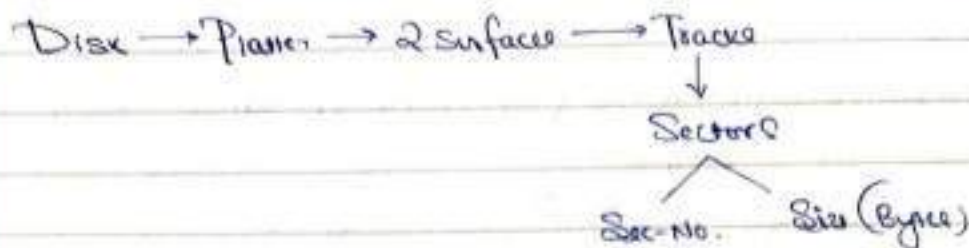
* Before storing the data on disk, it must be formatted in some file system (NTFS, Ext FAT32, UFS)

* Disk is a collection of circular/elliptical platters.

* Every platter has 2 track surfaces, and consists of circular tracks.

* Each track consists of sectors, Sector is measured in bytes.





* Disk IO Time = Seek-time (ST) + Rotational latency time + Transfer time

$(|x-y| \times TTT)$ $(RT = \frac{R}{2})$

R - time for one rotation.

TTT = Track-track time

Eg: 3600 rpm

3600 rotations - 60s

1 rotation - ?

$$R = \frac{60}{3600} = 16.67 \text{ ms}$$

→ Track Size = 'Z' bytes

→ Sector Size = 'S' bytes

→ Rotation time = 'R'

→ In one rotation, we can read one complete time Track.

'Z' Bytes - R (ms)

'S' Bytes - ?

$$\text{Transfer time} = \frac{(SR)}{Z} \text{ ms}$$

Data Transfer Rate (By/sec): Rate at which the data transfer takes place (speed of disk) is called data transfer rate.

'Z' Bytes - $R \times 10^{-3}$ (s)

? - 1s.

$$= \frac{Z}{R} \times 10^3 \text{ Bytes/sec}$$

Q1. Consider the following disk specifications

Number of platters = 16

No. of tracks / Surface = 512

No. of Sectors / track = 2048 (2K)

$$\text{Sector offset} = 12 \text{ bit} = 2^{12} = 4 \text{ KB}$$

$$\text{Average seek time} = 30 \text{ ms}$$

$$\text{Disk rpm} = 3600$$

Calculate:

(i) Unformatted Capacity of the disk:

$$= 32 \times 512 \times 2K \times 4KB = 2^9 \times 2^{11} \times 2^{12} = 2^{31} \text{ (missed to write)} \\ = 128 \text{ GB}$$

(ii) IO-time / Sector:

$$ST + LT + TT = 30 \text{ ms} + 8.3 \text{ ms} + 0 = 39 \text{ ms}$$

$$\text{Track size} = 2K \times 4KB \\ = 8 \text{ MB}$$

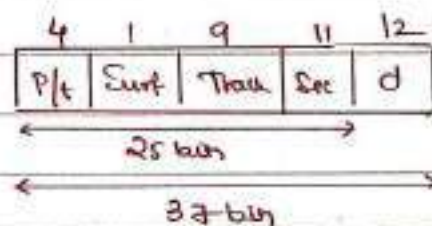
$$\left(\frac{4KB}{8 \text{ ms}} \times 16.6 \text{ ms} \right) \leftarrow TT$$

(iii) Data transfer rate:

$$\frac{TS}{8 \text{ MB}} = \frac{R}{16.6 \times 10^{-3} \text{ s}} \Rightarrow \frac{8 \text{ MB}}{16.6} \times 10^3 \text{ s} = \underline{\underline{1/2 \text{ GB/s}}}$$

$$? = 1 \text{ s}$$

(iv) Sector addresses = 25 bits required to select one sector.



$$\text{Disk capacity} = 2^{37} = 128 \text{ GB}$$

(another approach)

Ans

Q2.

Consider a disk with following specs:

$$\text{No. of surfaces} = 64$$

$$\text{Outer diameter} = 16 \text{ cm}, \text{ Inner dia} = 4 \text{ cm}$$

$$\text{Inner track space} = 0.1 \text{ mm}$$

$$\text{Track density} = 8000 \text{ bits/cm}$$

Calculate the unformatted Capacity of disk.

$$\text{No. of tracks} = \frac{6 \text{ cm}}{0.1 \text{ mm}} = 600$$

$$\text{Inner Track Circumference} = 2\pi r = \pi d$$

$$= 3.14 \times 4 = 12.56 \text{ cm}$$

$$8000 \text{ bits} = 1 \text{ cm}$$

$$? = 12.56 \text{ cm}$$

$$\text{Track Size} = \frac{8000 \times 12.56}{8}$$

$$= 1000 \times 12.56 \text{ B}$$

$$\text{Disk Capacity} = 64 \times 640 \times 12.56 \text{ KB}$$

$$= 64 \times 6 \times 12.56 \text{ KB}$$

$$= 64 \times 6 \times 1.256 \text{ MB}$$

$$= 482.2 \text{ MB} = 0.48 \text{ GB}$$

$$\therefore \text{Disk Cap} = 0.48 = 0.5 \text{ GB}$$

Q3. How long does it take to load a 64KB program from a disk whose avg. seek time is 30ms, Rotation time is 20ms, Track size is 32 Kbytes, Page size is 4kbytes. Assume that pages of program are distributed randomly around the disk. What will be the % saving in time if set of the pages of program are contiguous?

$$S.T = 30 \text{ ms}$$

$$R = 20 \text{ ms}$$

$$T.S = 32 \text{ KB}$$

$$P.S = 4 \text{ KB}$$

$$N = \frac{64 \text{ KB}}{4 \text{ KB}} = N_{pg}$$

$$\text{Time to load program} = \text{Time to load a page} \times N$$

$$\text{Time to load page} = S.T + L.T + T.T$$

$$= 30 \text{ ms} + 10 \text{ ms} + 2.5 \text{ ms}$$

$$= 42.5 \text{ ms}$$

$$\text{Prog. loading (T)} = 42.5 \times 16$$

$$= 680 \text{ ms} = 0.68 \text{ s}$$

$$32 \text{ KB} = 20 \text{ ms}$$

$$4 \text{ KB} = ?$$

$$\frac{4 \text{ KB}}{32 \text{ KB}} \times 20 = 2.5 \text{ ms}$$

When pages are contiguous then only one seek and one latency:

$$= (30 \text{ ms} + 10 \text{ ms} + 2.5 \text{ ms}) + 8 \times 42.5 \text{ ms}$$

$$(Cg) \quad (Ncg)$$

$$= 60 \text{ ms} + 340 = 400 \text{ ms} = 0.4 \text{ s}$$

$$(i) 0.68 \text{ s}$$

$$(ii) 0.4 \text{ s} = 0.28 \text{ s (diff.)}$$

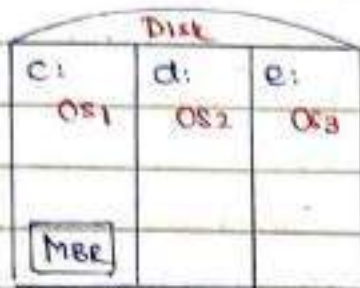
$$\therefore \text{Savings} = \left(\frac{0.28}{0.68} \right) = 41\% \text{ Percentage}$$

$$\text{Savings} = 41\%$$

File System and Device Management (continued)Logical Structure of disk: [Formatting Process]

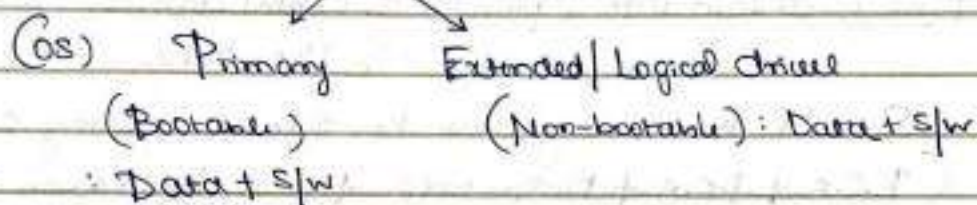
→ Placing infrastructure on disk

[for efficient storage and faster retrieval]

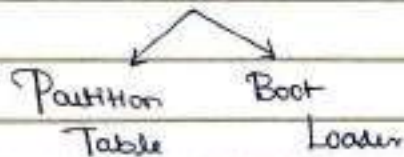


"Multi boot Computer"

→ Disk divided into partitions (volumes/minidisk)

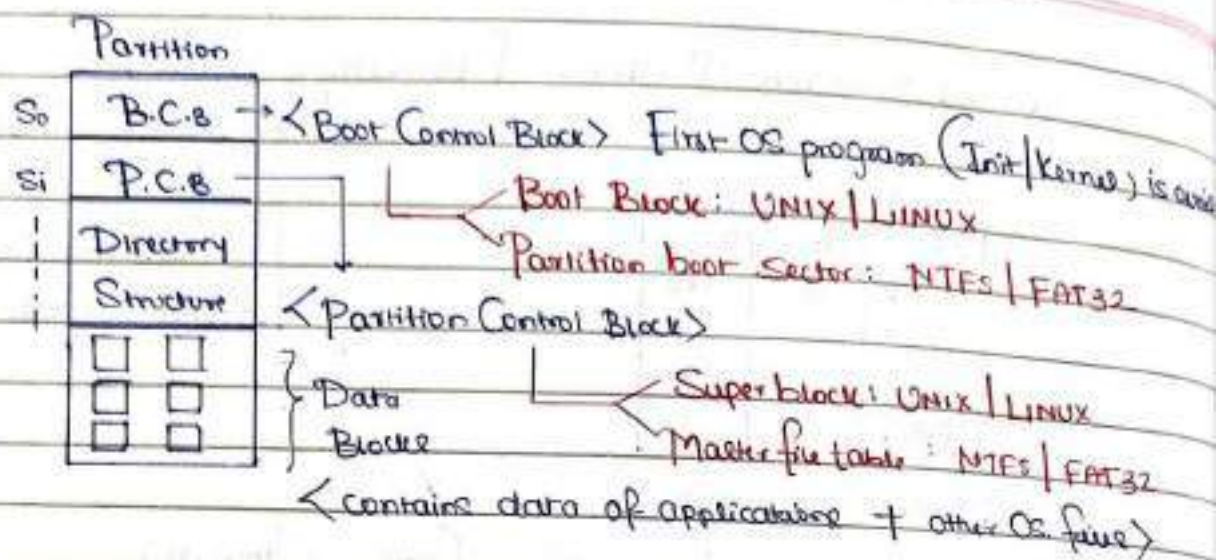


MBR < Master Boot record >

Steps in Booting process:

1. Power On or Switch on.
2. POST < Power on Self test >
3. BIOS < Basic Input/Output System >
4. Boot Strap loads MBR into RAM and hands over the control to boot loader.
5. Boot loader reads partition table and displays options if available.
6. Boot loader will load the kernel program from disk into memory.

Partition Structure [What data structure (infrastructure) are embossed on the disk/partition after formatting]



Disk is divided into 2 parts:

1. Control data
2. User data.

Some of the memory will be taken as memory overheads (B.C.B + P.C.B + D. Structure), that is the reason in Pen drives the size mentioned will be 128 Gb but memory actual avail is 114 Gb.

File v/s Directory:

File: File is a collection of logically related records of entity. It is an abstract data type < Definition, Representation, Operations, Attributes >

Operations: Create, Open, read, write, truncate, seek, Close, Copy, move, delete, ..

Flat
(Series of Bytes)

Record
(Series of records)
- Hierarchical Tree
(B-tree, B+..)

Attributes: Name, identifier, type, protection, time, date, location... (different OS have diff attributes)

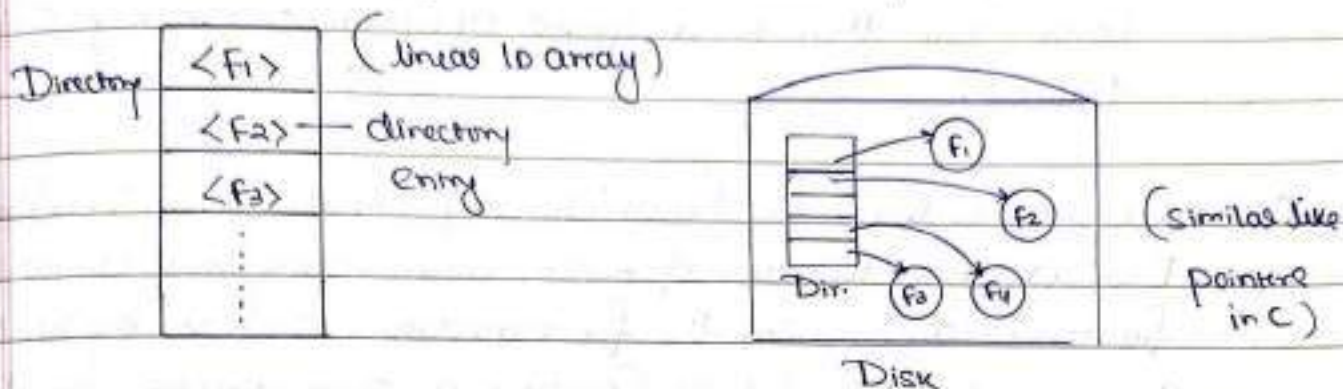
* File's attributes are stored in File Control Block (FCB)

UNIX
(I-node)

Windows
(Directory entry)

Directory: It is a collection of files. It is also a special file (data about other files $\rightarrow$ "Meta data")

Directory contains meta data of files.



Directory Structure

* "Single level directory"

Directory	cat	bo	a	test	data	mail	Cont	hex	records
-----------	-----	----	---	------	------	------	------	-----	---------

(Single directory for all users)

- * no sub directory
- * simple to implement
- * Search time is more
- * Name conflict

* Two level directory

- * Separate directory for each user
- * path name
- * Can have the same file name for diff. user
- * efficient searching
- * no grouping capability

* Tree structured directories (Multi level)

* Acyclic graph directories (File sharing)

- also called directed acyclic graph directory for file sharing purposes
- Link count: counts the no. of people sharing a file.

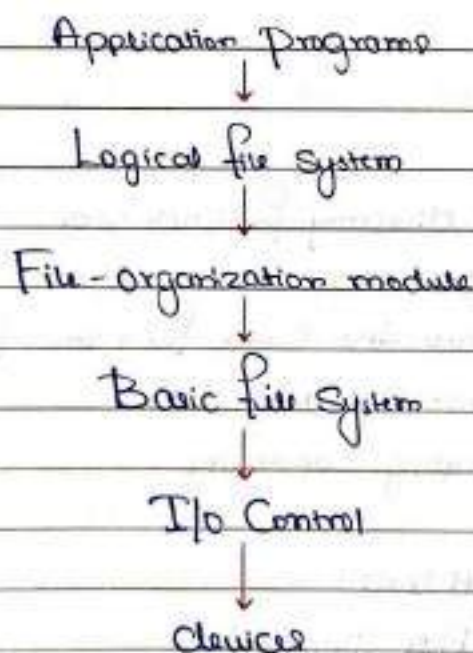
* General graph directory (Cycle)

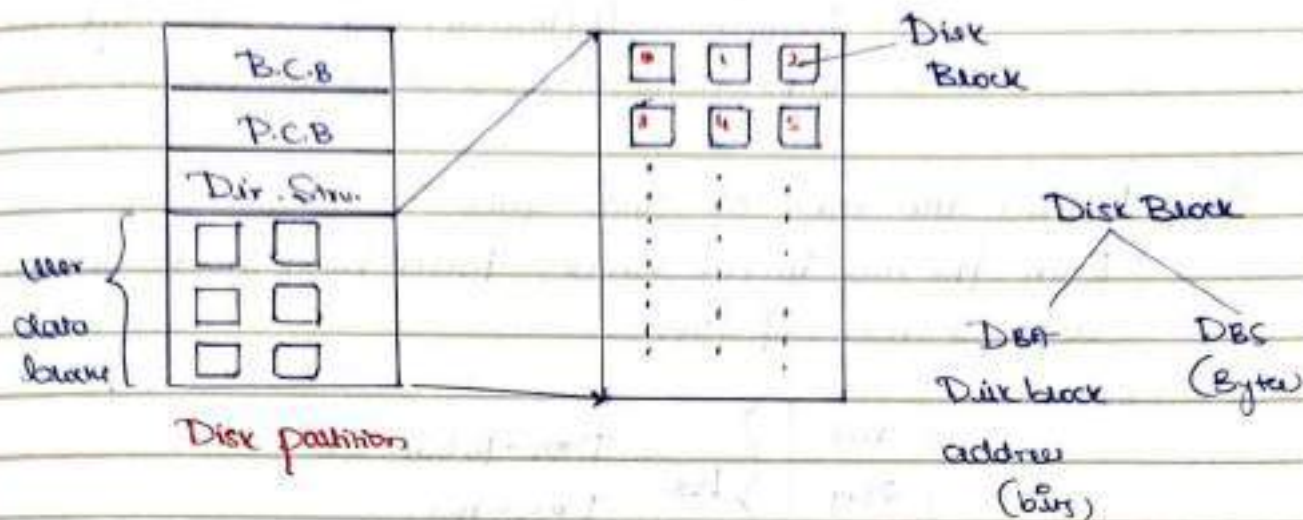
- * To search a file, we may have to traverse the directory, for that we may use Graph traversal like DFS, BFS.
- * Directory as a file also supports operations like open, read, write but there is a special operation on directory called "traverse".

Q1. Consider a linear list based directory implementation in a file system. Each directory is a list of nodes, where each node contains the file name along with the file metadata, such as the list of pointers to data blocks. Consider a given directory 'foo' which of the following operations will necessarily require a full scan of 'foo' for successful completion?

Ans. Creation of a new file in 'foo'.
Renaming of an existing file in 'foo'.

Layered File System



Allocation Methode

* $DBA = 16 \text{ bits}$

$DBS = 1 \text{ KB}$

* Max possible file size on the disk = Max possible (given) disk size,

eg: no. of blocks = $2^{16} = 64k$

Disk size = $64k \times 1KB = 64MB$

* Maximum Possible disk size = $(2^{DBA}) \times DBS$

① Contiguous Allocation of disk space

Performance:

1. internal fragmentation: Yes (last block)
2. external fragmentation: Yes
3. Increasing file size: Inflexible
4. type of access: Sequential / Random/direct access
↓
(Faster)

② Linked allocation of disk space

Performance:

1. Internal fragmentation: Yes (last block)
2. External fragmentation: No
3. Increasing file size: Flexible.

4. Type of access : Only sequential (clusters)
5. Performance / Efficiency : ptrs consume disk space
6. Vulnerability of pointers : pointers can get broken.

③ Indexed allocation of disk space

- * Each file has index block, index block holds address of data blocks of file.

eg::

x.2B	} 1kb.
2B.y	
⋮	
⋮	

Index block

$$DBA = 16 \text{ bits} = 2B$$

$$DBS = 1kB$$

$$\text{No. of addresses} = \frac{1kB}{2B} = \underline{512}$$

eg:: $DBA = 32 \text{ bits}$

$$DBS = 4kB$$

Max possible file size with index block : ?

$$\text{No. of addresses} = \frac{4kB}{4B} = \underline{1K}$$

$$\text{File size} = 1K \times 4kB = \underline{4MB}$$

Q1. A file system supports a $DBS = 4kB$, each block can hold 512 addresses (ptrs). What is the size of DBA in bits?

$$DBA = \frac{DBS}{N \text{ addresses}}$$

N addresses.

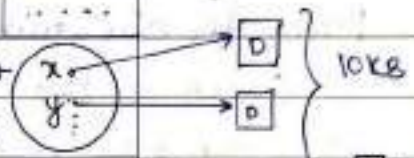
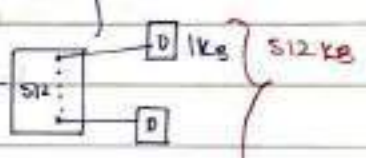
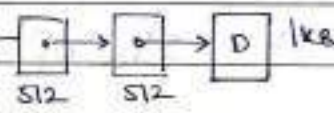
$$DBA = \frac{4kB}{512} = \frac{2^{12}}{2^9} = 2^3 = 8B = \underline{64 \text{ bits}}$$

Case Studies of O.S

① UNIX / LINUX : each file is associated with a I-node

File no	I-node
kxc	23

I-node Structure :

General Attribute	Type	Owner	23 (Dev)
	Date and Time	Sub	
	Permissions	mode	
	...	...	
	10 direct DBA's		10KB
	Single Indirect DBA		512KB
	Double Indirect pointer		1KB

Block is use by the file.

→ With double indirect

$$512 \times 512 \times 1KB = 2^{28} = 512MB$$

→ Total file size = 256 + 512 MB

② DOS / WINDOWS

Block info.

Master File Table

File Name	(Other Attributes)	First DBA	Last DBA
kxc	...	5	10

Directory

0	
1	
2	<2>
3	
4	
5	<6>
6	
7	
8	
9	
10	...
11	
12	
13	
14	
15	
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	
26	
27	
28	
29	
30	
31	

Linked allocation

* No. of entries in FAT (MFT) = no. of blocks

* Fat entry contains address (DBA) of next data block
FAT (32) Fat entry size DBA

Q1. In a file allocation system, which of the following allocation schemes(s) can be used if no external fragmentation is allowed?

Ans (i) Linked allocation
(ii) Indexed allocation

Q2. Consider a Unix-Inode structure that has 8 direct disk block address and 3 indirect block address, namely single, double and triple. Disk block size is 1KB and each block can hold 128 DBA. Calculate:

(i) Max file size with this Inode structure: 16.136 MB

$$= 8 \times 1KB + 128 \times 1KB + 128 \times 128KB + 128 \times 128 \times 128KB$$

$$= 8KB + 128KB + 16MB + 2GB$$

$$16.136MB = 0.016GB + 2GB = 2.016GB$$

$$= \underline{\underline{2GB}}$$

(ii) Size of the disk block address:

$$DBA = \frac{1KB}{128} = \frac{2^{10}}{2^7} = 2^3 = \underline{\underline{64 \text{ bits}}}$$

(iii) Is this file size possible on our hard disk?

Q3. The index node of a Unix-like file system has 12 direct, one single-indirect and one double-indirect pointers. The disk block size is 4KB, and the disk block address is 32 bit long. The max possible file size is 4GB GB.

Ans $N_{\text{addresses}} = \frac{4KB}{4B} = 1K$

$$\begin{aligned} \text{File Size} &= 12 \times 4KB + 1K \times 4KB + 1K \times 1K \times 4KB \\ &= 48KB + 4MB + 4GB \\ &= 4.004GB \sim 4GB \end{aligned}$$

Q4 A file system with 300GB disk uses a file descriptor with 8 DBA, 1 indirect block address and 1 doubly indirect block address. The size of each disk block is 128B and size of each DBA is 8 bytes. Max possible file size in the file system is

Ans

$$DBA = 128B$$

$$DBA = 8B$$

$$\text{No. addresses} = \frac{128}{8} = 16$$

$$\text{File Size} = 8 \times 128B + 16 \times 128B + 16 \times 16 \times 128B$$

$$= 1KB + 2KB + 32KB$$

$$= \underline{35KB}$$

Q5. The data blocks of a very large file in the Unix file system are allocated using:

Ans An extension of indexed allocation

Q6 Using a larger block size in a fixed block size file system leads to

Ans Better disk throughput and poor disk space utilization

Q7 A FAT based file system is being used and total overhead of each entry in the FAT is 4 bytes in size. Given a 100×10^6 bytes disk on which the file system is stored and data block size is 10^3 Bytes, the maximum size of a file that can be stored on this disk in units of 10^6 Bytes is 99.6

$$N = \frac{100 \times 10^6}{10^3} = 100 \times 10^3$$

$$\text{Max. file size} = \text{Given disk size} - \text{FAT size}$$

$$= 100 \times 10^6 - 0.4 \times 10^6$$

$$= \underline{99.6 \times 10^6}$$

$$\text{FAT size} = 100 \times 10^3 \times 4B$$

$$= 400 \times 10^3 B$$

$$= 4 \times 10^5$$

$$= 0.4 \times 10^6 B$$

Q8 A file system with one level directory structure is implemented on a disk with disk block size of 4KB. The disk is used as follows:

Disk block 0 : Boot control block

Disk block 1 : File allocation system/table, consisting of 10-bit entry per data block, representing the data block address of next data block in file.

Disk block 2, 3 : Directory with 32-bit entry per file.

Disk block 4 : Data block

Disk block 5 : Data block 2, 3, etc.

(i) What is the max possible number of file?

(ii) What is the max possible file size in bytes?

Ans. Number of file (max) = no. of entries

$$= \frac{8 \text{ KB}}{4 \text{ B}} = 2^k = \underline{2048}$$

$$\text{DBA} = 10 \text{ bits}$$

$$\text{DBS} = 4 \text{ KB}$$

$$\text{Max file size} = \text{Disk size} - (\text{Control overheads})$$

$$\text{No. of blocks} = 2^{10}$$

$$= 1024 - 4 = 1020 \times 4 \text{ KB}$$

$$= 1024 - 4 = \underline{1020}$$

$$= 4080 \text{ KB}$$

$$= \underline{\underline{4.08 \text{ MB}}}$$

Q9. Consider a file system that stores 128 DBA in the index table of the directory. Disk block size is 4 KB. If the file size is less than 128 blocks, then these addresses act as direct data block addresses.

However, if the file size is more than 128 blocks, then these 128 addresses in the index table point to next level index blocks, each of which contains 256 data block's address. What is the max file size in the system?

Ans. Min file size = $128 \times 4 \text{ KB}$

$$= \underline{512 \text{ KB}}$$

$$128 \times 256 \times 4 \text{ KB}$$

$$= 2^7 \times 2^8 \times 2^{12} = 2^{27} = \underline{\underline{128 \text{ MB}}}$$

Disk Free Space management algorithms:

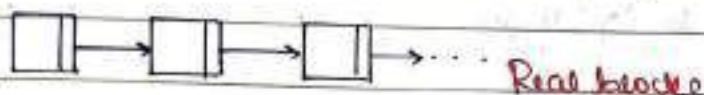
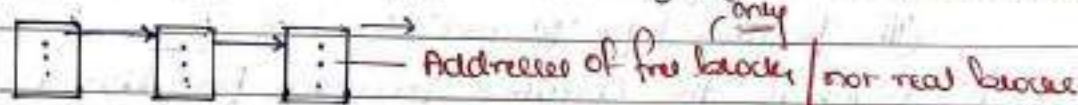
Assume: disk - 20 TB;

DBS : 1 KB

DBA : 16 bits

$$N. \text{ block} = \frac{20 \text{ TB}}{1 \text{ KB}} = \underline{20k}$$

Given disk Size	$\leq$	Max. possible D.S
		$\downarrow$
		$2^{DBA} \times DBS$

1. Free linked list: linked list of free blocks.2. Free list: keeps track of free blocks only, faster compared to bit map.

Q. How many blocks are (Free list) are needed to store 20k free DBS?

Ans: 1 block - 512 free DBS

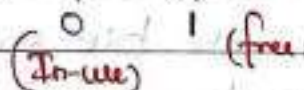
$$? - 20k \text{ free} = \frac{20k}{512} = \underline{40}$$

3. Bit-map / Vector

: Associate a Binary bit with each block

: Bit map keeps track of all blocks.

: Slower compared to bit-map.



Q. How many blocks of disk are needed to store our bit-map?

Ans: DBS = 1 KB

DBA = 16 bits

$$1 \text{ Block} \rightarrow 8k \text{ bits}$$

$$? \rightarrow 20k \text{ bits}$$

$$= \frac{20k}{8k} = 2.5 = \underline{3 \text{ Blocks}}$$

4. Counter method: when we have many contiguous free blocks.

Size of DBA. No. of contiguous free blocks

1.	5	45
2.	85	200
3.	325	125
4.	600	25

eg: New File $F = 20$ blocks
Consumption can be done by
first fit, best fit, worst fit.

- Q1. Consider a disk with 'B' blocks, 'F' are free, DBA is 'D' bits.
Disk block size is 'X' Bytes.

(A) Calculate:

(i) Given disk size: $B \times X$ Bytes.

(ii) Max. possible disk size = $2^D \times X$ Bytes.

(iii) Relation b/w 'B' and 'D' = $B \leq 2^D$.

(B) What is the condition in which Free list uses less space than Bit-Map?

$$F \times D < B$$

- Q2. The beginning of a free space bit map looks like after the disk partition is first formatted: 1000 0000 0000 0000. The system always searches for free blocks starting at the lowest numbered block, so after writing file A, which uses 6 Kib blocks, the bit map looks like this: 1111 1110 0000 0000. Show the bit map after each of the following additional actions of the code.

- Ans.
- * File B is written, using 5 Blocks
 - * File deleted.

- Q3. A file system uses an in-memory cache to cache disk blocks. The miss rate of the cache is shown in the figure. The latency to read a block from cache is 1 ms and to read a block from disk is 10 ms. Assume that the cost of checking

whether a block exists in the cache is negligible. Available cache sizes are in multiples of 10 MB.

The smallest cache size required to ensure an average read latency of less than 6 ms is 30 MB.

Ans

$$G_{ms} = x(1ms) + (1-x)(10ms)$$

$$= x + 10 - 10x$$

$$-4 = -9x$$

$$x = 4/9 = 0.45$$

$$\text{Miss ratio} = 0.55$$

$$55\% \Rightarrow 25 \text{ ms}$$

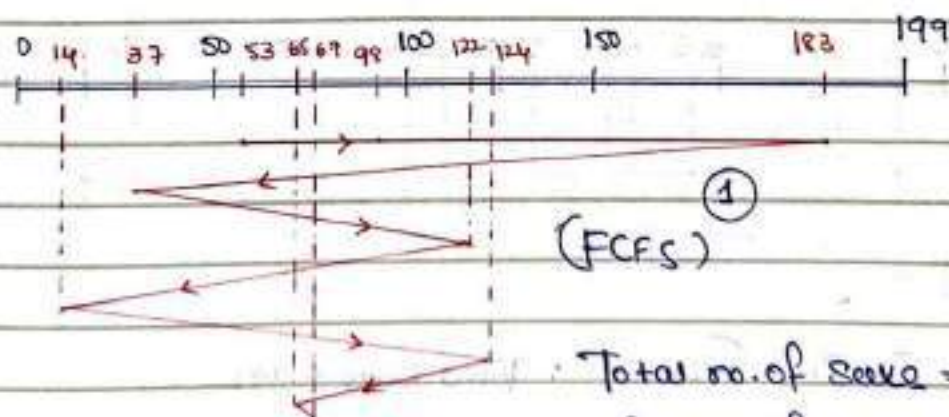
= 30 MB (for this miss ratio reduce, Error)

Q4. The amount of disk space that must be available for page storage is related to maximum number of processes 'N', the number of bytes in Virtual Address Space 'B' and the number of bytes in Ram 'R'. Give an expression for worst case disk space required.

Ans. $N \times S$ Bytes

Device Scheduling:

Process requests : 98, 183, 37, 122, 14, 124, 65, 67. (Track no / cylinder no.)



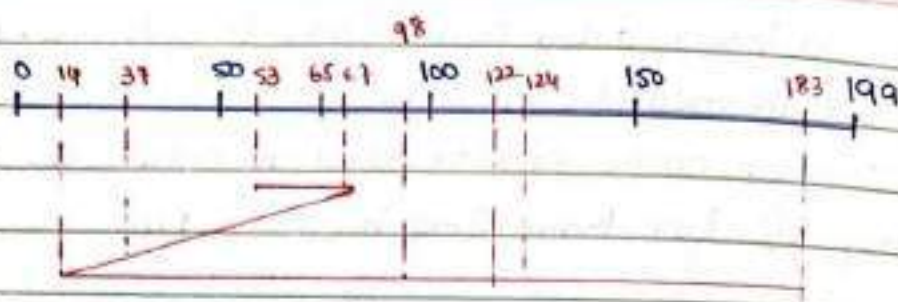
(FCFS)

$$\text{Total no. of seeks} = 640$$

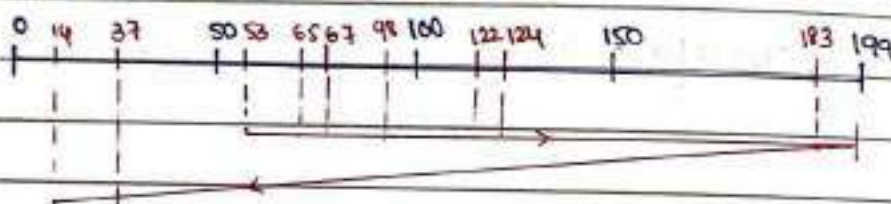
$$\text{Avg no. of seeks} = \frac{640}{80}$$

* Objective of disk scheduler is to reduce average seek-time.

$$= 80$$



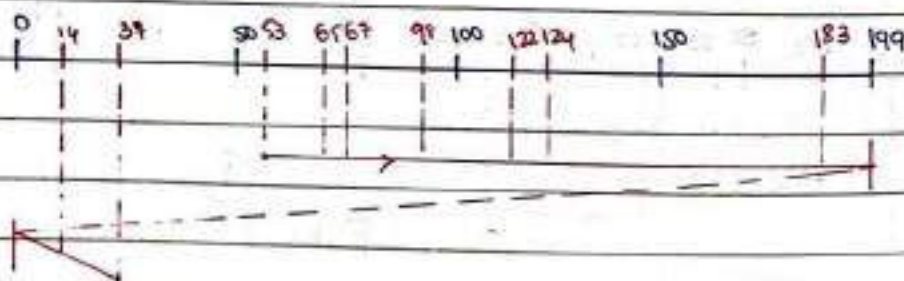
② Shortest Seek time first : Total Seek = 236



③ Scan/Elevator : Total Seek = 331



④ Look : Total Seek = 299

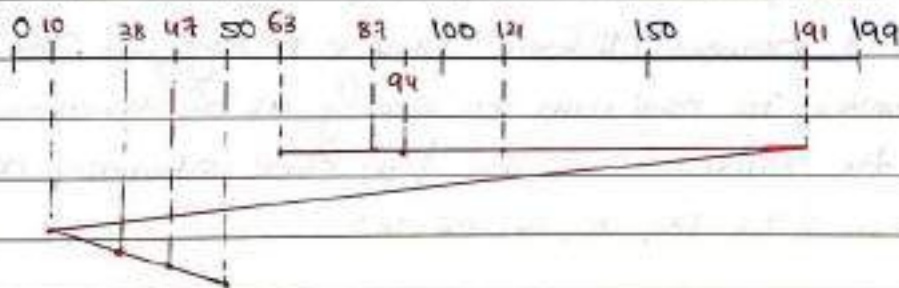


⑤ C-Scan (Circular) : Total Seek = 382



⑥ C-Look : Total Seek : 322

Q1. Consider a disk queue with requests for I/O to blocks on cylinder 47, 38, 121, 191, 87, 11, 92, 10. The C-look scheduling algorithm is used. The head is initially at cylinder no. 63, moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement incurred while servicing these requests is: 346



Q2. Disk requests come to disk driver for cylinders 10, 22, 20, 2, 40, 06 and 38, in that order at a time when the disk drive is regarding from cylinder 20. The seek time is 6ms per cylinder. Compute the total seek time if the disk arm scheduling algorithm is:

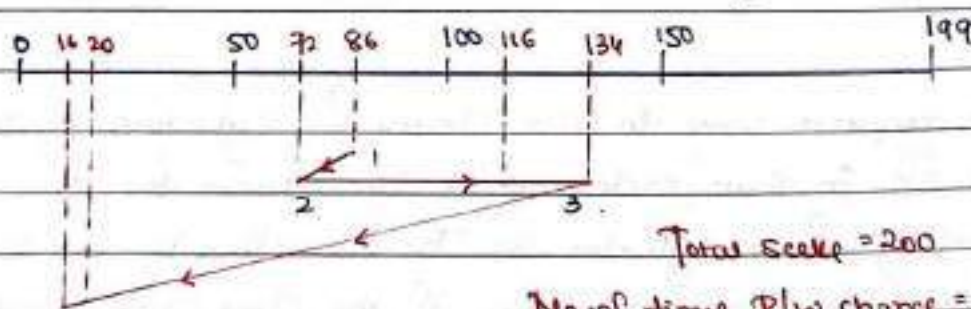
(a) First Come First Serve.

(b) Closest cylinder next:

Q4. Consider a storage disk with 4 platters (0-3) 200 cylinders (0-199) and 256 sectors per track (numbered as 0, 1, 255). The following 6 disk requests of the form [Sector no, cyl. no, platter no] are received by disk controller at same time [120, 72, 2], [134, 134, 1], [60, 20, 0], [212, 86, 3], [56, 116, 2], [118, 6, 1].

Currently head is at no. 100 of cylinder 80 and moving towards higher cylinder numbers. The average power dissipation in moving the head over 100 cylinders is 20 milliwatt and for reversing the direction of the head movement every once is 15 milliwatt. Power dissipation associated with rotational latency and switching of head between different platters is negligible. The total power consumption in milliwatts to satisfy all of the above disk request using the shortest seek time first disk scheduling algorithm is 85.

Ans. Requests < 72, 134, 20, 86, 116, 16 >



$$\text{Total Seek} = 200 - 20 \times 2 = 160$$

$$\text{No. of times P/W change} = 3 - 3 \times 15 = 45$$

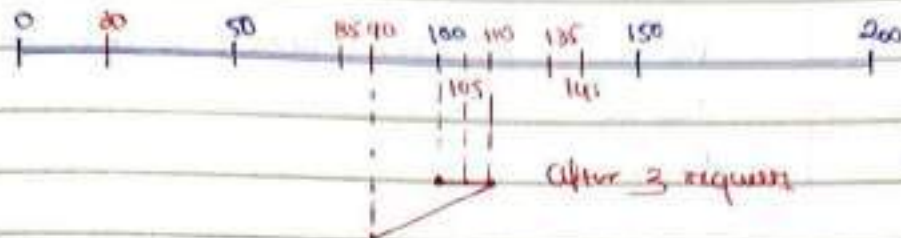
its direction.

$\therefore$ 85 milli watts

Q5. Suppose a disk has 201 cylinders, numbered from 0 to 200. At some time the disk arm is at cylinder 100, there is a queue of disk access requests for cylinders 30, 85, 90, 100, 105, 110, 135, 145. If SSTF is being used for scheduling the disk access

the request for cylinder 90 is serviced after servicing 3 no. of requests

Ans.



Q6. Consider an operating system capable of loading and executing a single sequential user process at a time. The disk head scheduling algorithm used is FCFS. If FCFS is replaced by SRTF, claimed by vendor to give 50% better benchmark results, what is the expected improvement in the I/O performance of user programs?

Ans. 0% (no improvement)

Q7. Consider the following five disk access requests of the form (req. id, cylinder number) that are present in the disk scheduler queue at a given time (P, 155), (Q, 85), (R, 110), (S, 30), (T, 115)

Assume the head is positioned at cylinder 100. The scheduler follows Shortest Seek time first to serve the requests. Which of statements is false?

Q8. Consider a disk with following details:

Track-to-track time = 1ms.

Current head position = 65

Direction: moving towards higher tracks with higher track = 199

Current clock time = 160ms.

Consider the following data set:

Q11W

Track no.	T. of arrival
12	65 ms
85	80 ms
40	110 ms
100	120 ms
75	175 ms

Calculate time of decision, Pending requests, head position, Selected request, Seek time using FCFS, C-scan, SSTF, SCAN, LOOK, C-look algorithms.

